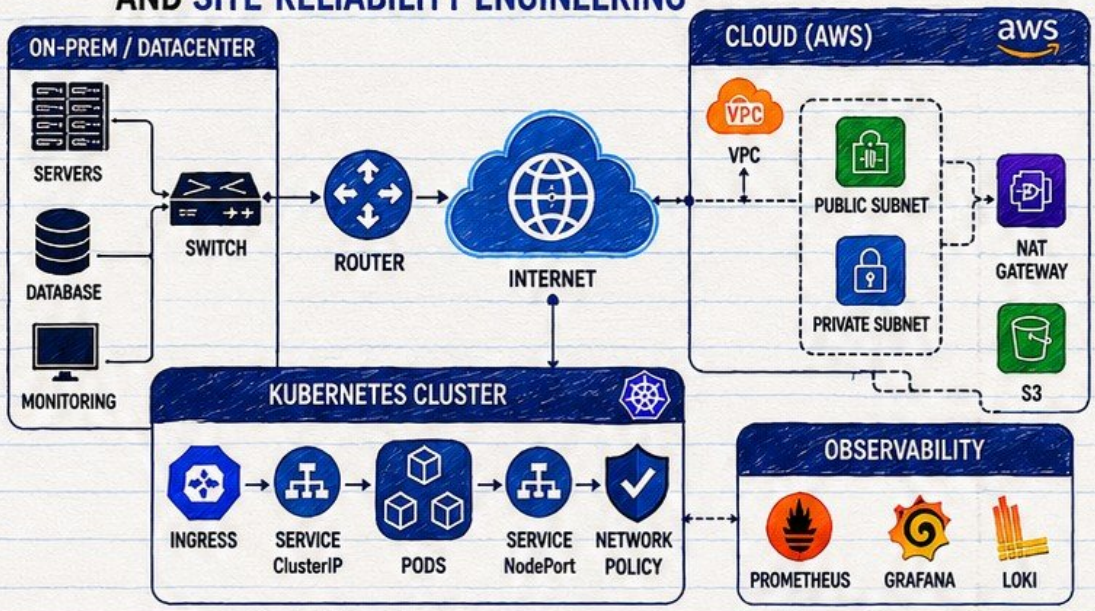
VERIQTA

NETWORKING FOR DEVOPS HANDBOOK

60 PAGES
FLAGSHIP HANDBOOK

THE COMPLETE GUIDE TO NETWORKING FOR CLOUD, KUBERNETES, PLATFORM ENGINEERING, AND SITE RELIABILITY ENGINEERING

- FROM FUNDAMENTALS TO PRODUCTION
- CLOUD, KUBERNETES & HYBRID NETWORKING
- SECURITY, RELIABILITY & BEST PRACTICES
- TROUBLESHOOTING, MONITORING & PERFORMANCE
- REAL-WORLD EXAMPLES & CASE STUDIES



NETWORK FUNDAMENTALS | LINUX NETWORKING | ENTERPRISE NETWORKING | CLOUD NETWORKING | KUBERNETES NETWORKING | PRODUCTION OPERATIONS

BUILD, SECURE, OPERATE, AND TROUBLESHOOT NETWORKS LIKE A PRO

WHO THIS HANDBOOK IS FOR

DEVOPS ENGINEERS | SITE RELIABILITY ENGINEERS (SRE) | PLATFORM ENGINEERS | CLOUD ENGINEERS | KUBERNETES ENGINEERS | INFRASTRUCTURE ENGINEERS | TECHNICAL ARCHITECTS

- ✓ PRODUCTION FOCUSED
- ✓ PRACTICAL
- ✓ HANDS-ON
- ✓ REAL WORLD READY

VERIQTA

1. INTRODUCTION TO PRODUCTION NETWORKING

"You can't secure what you don't understand."
You can't scale what you can't see."
– Network First, Everything Follows.

PRODUCTION NOTE

Most outages are not caused by the application. They are caused by networking.

WHY NETWORKING IS THE BACKBONE OF DEVOPS

- Every request starts and ends with the network.
- Compute, storage, databases, containers, APIs → All communicate over the network.
- No network = No systems.
- Reliability, performance, security, and scale all depend on great networking.
- DevOps is not just about code and CI/CD, it is about delivering systems that work anywhere, anytime.

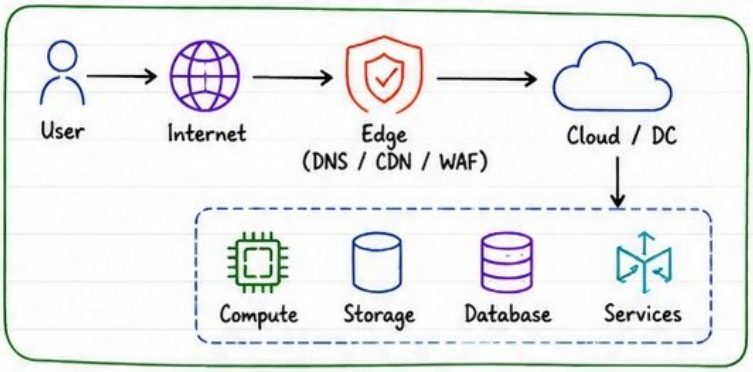
NETWORKING RESPONSIBILITIES FOR DEVOPS ENGINEERS

- ☑ Design and implement reliable network architectures.
- ☑ Automate network provisioning (IaC, APIs, Controllers).
- ☑ Manage IP addressing, routing, DNS, and load balancing.
- ☑ Secure networks (firewalls, ACLs, segmentation, zero trust).
- ☑ Ensure high availability and disaster recovery.
- ☑ Monitor network performance and troubleshoot issues.
- ☑ Collaborate with security, platform, and cloud teams.

HANDBOOK OBJECTIVES

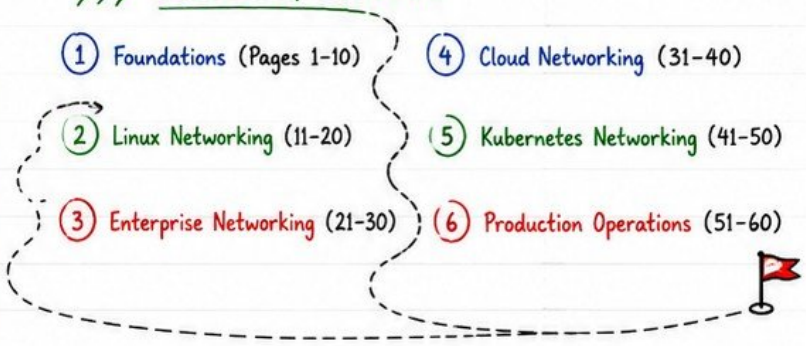
- ☆ Build a strong foundation in networking fundamentals.
- ☆ Master Linux networking for production systems.
- ☆ Understand enterprise networking concepts.
- ☆ Design and operate cloud networking (AWS focus).
- ☆ Deep dive into Kubernetes networking.
- ☆ Troubleshoot real-world networking problems.
- ☆ Adopt an enterprise networking mindset.

NETWORKING IN MODERN CLOUD ENVIRONMENTS



- Hybrid and multi-cloud are the new normal.
- Everything is distributed.
- Identity, routing, encryption, and segmentation happen at scale.
- Networking must be automated, observable, and secure by design.

LEARNING ROADMAP



ENTERPRISE NETWORKING MINDSET

THINK IN SYSTEMS
Understand how components communicate.

DESIGN FOR FAILURE
Networks fail. Build for resilience and recovery.

AUTOMATE EVERYTHING
Manual networking does not scale. Automate and version your network.

OBSERVE & MEASURE
You can't improve what you don't measure.

SECURE BY DEFAULT
Security is not a feature. It is the foundation.

2. HOW DATA TRAVELS ACROSS NETWORKS

PACKETS AND FRAMES

- Data is broken into small chunks for efficient transmission.
- Packets (Layer 3) travel across networks.
- Frames (Layer 2) are used within local networks.

Type	Layer	Purpose	Example
Segment	L4 - Transport	Breaks data into transport units	TCP Segment UDP Datagram
Packet	L3 - Network	Logical addressing and routing	IP Packet (IPv4 / IPv6)
Frame	L2 - Data Link	Local delivery (MAC addressing)	Ethernet Frame
Bits	L1 - Physical	Actual electrical/ optical signals	0s and 1s

ENCAPSULATION (SENDING HOST)

Data moves down the stack

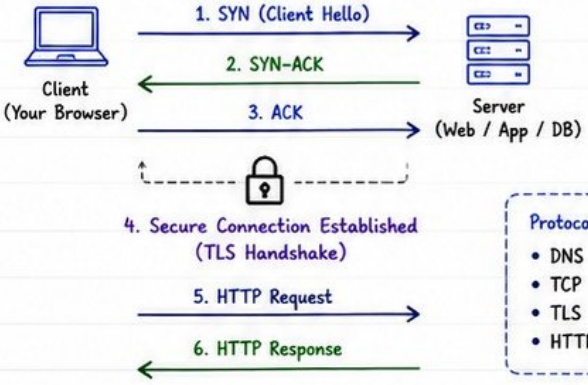
- Application Data → Your data (HTTP request, DB query, file, etc.)
- L4 Header (TCP/UDP) → Adds port numbers, reliability, checksums
- L3 Header (IP) → Adds IP addresses, routing information
- L2 Header (MAC) → Adds MAC addresses, error detection (FCS)
- Bits on the Wire 0101010101... → Converted to electrical, optical or radio signals

DECAPSULATION (RECEIVING HOST)

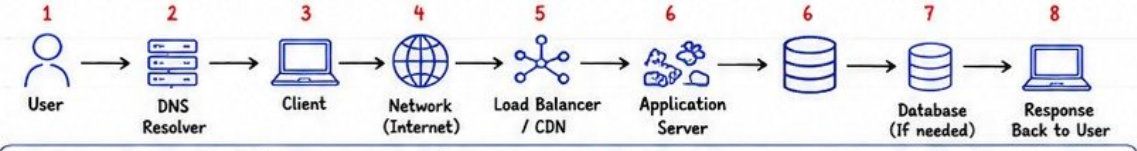
Data moves up the stack

- Bits on the Wire 0101010101... → Physical signals are received by NIC
- L2 Header (MAC) → Frame is validated (MAC + FCS checked)
- L3 Header (IP) → Packet is validated (IP header checked)
- L4 Header (TCP/UDP) → Segments are reassembled, acknowledged
- Application Data

CLIENT-SERVER COMMUNICATION



END-TO-END REQUEST LIFECYCLE

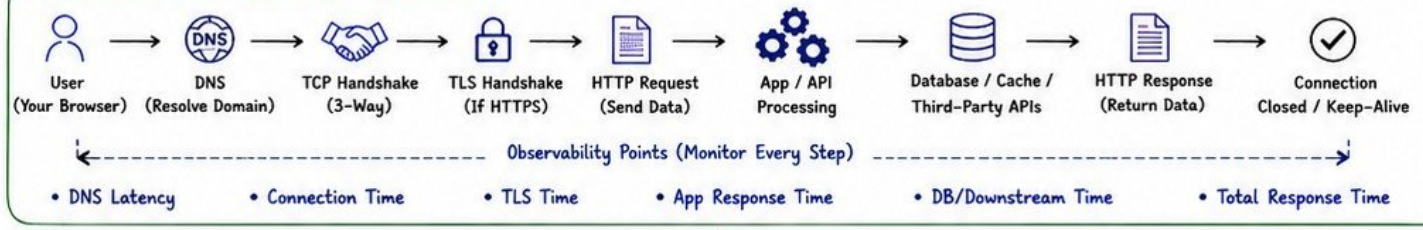


1. User initiates a request (e.g., opens a website).
2. DNS resolves the domain to an IP address.
3. Client opens a connection to the IP (TCP handshake).
4. Packets travel across the internet to the destination.
5. Load Balancer / CDN routes traffic to a healthy server.
6. Application server processes the request.
7. Database or downstream services are queried (if needed).
8. Response travels back through the same path to the user.

PRODUCTION NOTE

Every delay, drop, or misconfiguration in any of these steps can cause high latency, timeouts, or complete outages.

PRODUCTION REQUEST FLOW (DETAILED)



SENIOR ENGINEER TIP

When debugging network issues, trace the entire path. The problem is rarely where the error appears. Use layered visibility: DNS → Network → TCP → TLS → HTTP → Application.

COMMON PRODUCTION ISSUES

- Slow DNS = slow everything
- Packet loss → retries → high latency
- MTU mismatch → strange connection drops
- Firewall/NACL blocking
- SNAT port exhaustion
- Load balancer misconfiguration

VERIQTA

3. THE OSI MODEL

"The OSI Model is a conceptual framework that standardizes how data communicates across a network."

PRODUCTION NOTE

In real production systems, issues can occur at any layer. A senior engineer isolates the layer first, then digs deeper.

THE SEVEN LAYERS OF THE OSI MODEL

LAYER (No.)	LAYER NAME	PRIMARY RESPONSIBILITIES	PDU (Protocol Data Unit)
7	Application Layer	• Provides network services to user applications. • Handles protocols that applications use. • Examples: HTTP, HTTPS, DNS, SMTP, SSH	Data
6	Presentation Layer	• Translates, encrypts, compresses data. • Ensures data is in a usable format. • Examples: TLS/SSL, JPEG, ASCII, MPEG	Data
5	Session Layer	• Manages sessions between applications. • Establishes, maintains, and terminates sessions. • Examples: NetBIOS, RPC, SIP, PPTP	Data
4	Transport Layer	• End-to-end communication. • Segmentation, flow control, error recovery. • Examples: TCP, UDP	Segment (TCP) / Datagram (UDP)
3	Network Layer	• Logical addressing and routing. • Path selection and packet forwarding. • Examples: IP (IPv4, IPv6), ICMP	Packet
2	Data Link Layer	• Node-to-node delivery. • Framing, MAC addressing, error detection. • Examples: Ethernet, VLAN, PPP	Frame
1	Physical Layer	• Transmits raw bits over the physical medium. • Defines electrical, optical, mechanical specs. • Examples: Cables, Hubs, Repeaters	Bits

MAPPING TO REAL TECHNOLOGIES

OSI LAYER	REAL-WORLD TECHNOLOGIES / PROTOCOLS
7. Application	HTTP/HTTPS, DNS, SMTP, SSH, RDP, SNMP, NTP, REST APIs
6. Presentation	TLS/SSL, JSON, XML, ASCII, JPEG, MPEG, GZIP
5. Session	RPC, NetBIOS, SIP, PPTP, SMB Session, SQL Session
4. Transport	TCP, UDP, SCTP
3. Network	IP (v4/v6), ICMP, IGMP, IPSec, OSPF, BGP
2. Data Link	Ethernet, Wi-Fi (802.11), VLAN (802.1Q), PPP, STP
1. Physical	Copper, Fiber, Radio, Hubs, Repeaters, Patch Panels

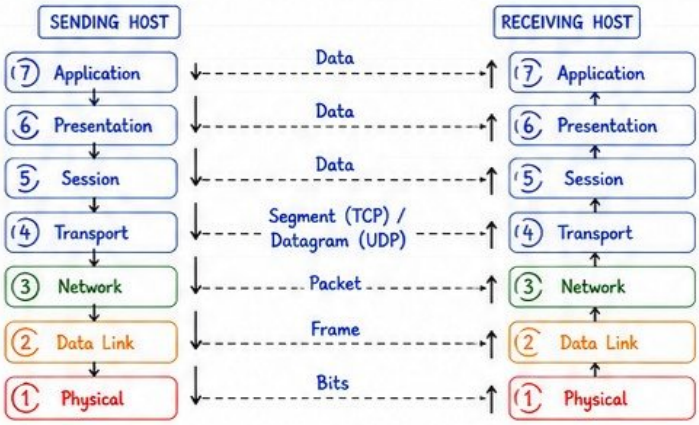
TROUBLESHOOTING USING OSI

7. Is the application working as expected? Check service, protocol, and data format.
6. Is the data in the correct format? Check encryption, compression, encoding.
5. Is the session established and stable? Check for timeouts and session drops.
4. Is the end-to-end transport healthy? Check ports, TCP state, retransmits.
3. Is routing and IP connectivity working? Check IP, routes, firewalls, NAT.
2. Is the frame reaching the target device? Check MAC, VLAN, switching, ARP.
1. Is the physical connection good? Check cables, NIC, signal, link up.

↑ Start from the Top (Application)

↓ Go Down until you find the Break.

HOW DATA MOVES ACROSS THE OSI MODEL



⟶ Encapsulation (Down the stack) ⟶ Decapsulation (Up the stack)

PRODUCTION EXAMPLES BY LAYER

7	App down due to DNS failure (e.g., cannot resolve API domain).
6	TLS certificate expired causing HTTPS failure.
5	Database session idle timeout causing disconnects.
4	TCP port blocked by firewall, causing connection refused.
3	Wrong route or missing NAT causing packets to drop.
2	Incorrect VLAN tagging causing devices cannot communicate.
1	Faulty cable or NIC down causing link failure.

SENIOR ENGINEER TIP

Don't jump to conclusions. Use OSI as your mental checklist. Layered thinking saves time, reduces guesswork, and helps you communicate clearly during incidents.

COMMON MISTAKES

- Checking Layer 7 without verifying Layer 3/4.
- Ignoring physical layer issues (cables/NIC).
- Assuming it's an application problem.
- Not using tools that map to OSI layers.

QUICK RECALL

7-6-5-4-3-2-1
Top to Bottom (Send)
1-2-3-4-5-6-7
Bottom to Top (Receive)

VERIQTA

4. TCP/IP MODEL

"TCP/IP is the foundation of how the internet works.
It is practical, scalable, and the standard everywhere."

PRODUCTION NOTE

You don't design networks around theory.
You design them around what actually works at scale.

INTERNET ARCHITECTURE

WHAT HAPPENS?

1. Your device sends a request over the internet.
2. Routers forward the packets across multiple networks.
3. The server receives the request and sends a response.
4. The response travels back the same (or different) path.
5. Your device receives and displays the data.

TCP/IP LAYERS

LAYER (No.)	LAYER NAME	PRIMARY FUNCTIONS	EXAMPLES / PROTOCOLS	PDU
4	Application Layer	• Provides network services to applications. • Handles user applications and protocols.	HTTP/HTTPS, DNS, SMTP, SSH, FTP, RDP, SNMP, NTP, REST APIs	Data
3	Transport Layer	• End-to-end communication. • Segmentation, reliability, flow control, error recovery.	TCP, UDP, SCTP	Segment (TCP) / Datagram (UDP)
2	Internet Layer	• Logical addressing and routing. • Path selection and packet forwarding across networks.	IP (IPv4, IPv6), ICMP, IGMP, IPsec	Packet
1	Network Access (Link) Layer	• Node-to-node delivery. • Framing, MAC addressing, error detection on local link.	Ethernet, Wi-Fi (802.11), PPP, VLAN, ARP	Frame

COMPARISON WITH OSI MODEL

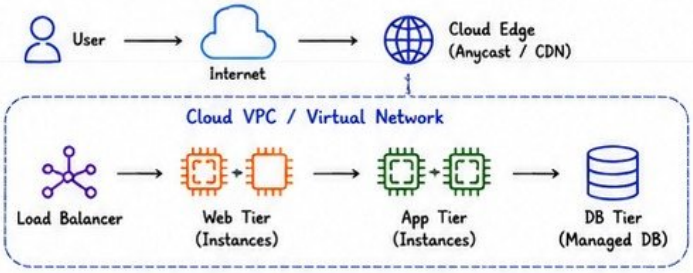
OSI MODEL (7 LAYERS)	TCP/IP MODEL (4 LAYERS)	HOW THEY MAP
7. Application 6. Presentation 5. Session	→ Application Layer	All application and user interaction functions are combined.
4. Transport	→ Transport Layer	Same core function: End-to-end communication, reliability, flow control.
3. Network	→ Internet Layer	Same core function: Logical addressing and routing.
2. Data Link 1. Physical	→ Network Access (Link) Layer	Combined into one layer handling local delivery and framing.

WHY TCP/IP DOMINATES MODERN INFRASTRUCTURE

- ✓ Built for the real world (Internet), not just theory.
- ✓ Simple and practical (4 layers vs 7 layers).
- ✓ Vendor neutral and universally adopted.
- ✓ Scales from small networks to global infrastructure.
- ✓ Supports multiple transports (TCP, UDP, SCTP).
- ✓ Works across all devices, OS, and cloud platforms.
- ✓ Easier troubleshooting and faster implementation.
- ✓ Designed for reliability, performance, and interoperability.

TCP/IP is not just a model. It is the language of the internet and modern cloud systems.

CLOUD NETWORKING AND TCP/IP



Under the hood: Everything uses TCP/IP

☑ IP Addressing ☑ Routing ☑ TCP/UDP ☑ Security (IPsec)

Global Reach
TCP/IP enables communication across regions, continents, and cloud providers.

Scalability
IP routing and stateless communication make it easy to scale workloads and services.

Security
TCP/IP supports encryption (TLS), VPNs (IPsec), firewalls, and micro-segmentation.

Performance
Efficient protocols (TCP for reliability, UDP for speed) power modern applications, streaming, and APIs.

Cloud Agnostic
Whether AWS, Azure, GCP, or on-prem — TCP/IP remains the universal foundation.

SENIOR ENGINEER TIP

When troubleshooting, think in TCP/IP layers:
Start at Layer 4 (Application) and go down.
Don't guess. Validate each layer step by step.

KEY TAKEAWAYS

- TCP/IP is the real-world model behind the internet.
- 4 layers, practical and scalable.
- Used in every device, cloud, and data center.
- Understand it deeply to design, secure, and scale modern systems.

COMMON PROTOCOLS BY LAYER

Application : HTTP, DNS, SMTP, SSH, RDP, SNMP
Transport : TCP, UDP, SCTP
Internet : IP (v4/v6), ICMP, IPsec
Link : Ethernet, Wi-Fi, ARP, VLAN

VERIQTA

5. IPV4 ADDRESSING

"IP addressing is the foundation of every network."
If you design it right, everything scales. If you design it wrong, everything breaks."

PRODUCTION NOTE

Poor IP design causes:

- ☑ IP conflicts
- ☑ Routing issues
- ☑ Security gaps
- ☑ Outages at scale

BINARY FUNDAMENTALS

- IPv4 address = 32 bits (4 octets)
- Each octet: 8 bits (0 – 255 in decimal)

DECIMAL	BINARY (8 BITS)	RANGE
0	00000000	0
1	00000001	1
127	01111111	127
128	10000000	128
255	11111111	255

EXAMPLE: 192.168.1.10

Decimal	192	168	1	10
Binary	11000000	10101000	00000000	00001010

32 Bits

Tip: Learn binary for faster subnetting and troubleshooting.

PUBLIC AND PRIVATE IP ADDRESSES

Public IPs

- Routable on the internet
- Assigned by ISP or Cloud Provider
- Used for services that must be accessible from the internet

Private IPs (RFC 1918)

- Not routable on the internet
- Used inside private networks
- Save public IPs and add security

PRIVATE RANGE	CIDR	USABLE ADDRESSES
10.0.0.0 – 10.255.255.255	10.0.0.0/8	~16.7 Million
172.16.0.0 – 172.31.255.255	172.16.0.0/12	~1.0 Million
192.168.0.0 – 192.168.255.255	192.168.0.0/16	~65 Thousand

NETWORK ID AND HOST ID

- An IP address is divided into two parts: Network ID and Host ID.

32 Bits

NETWORK ID	HOST ID
n bits	(32 – n) bits

EXAMPLE: 192.168.1.10/24

IP Address: 11000000.10101000.00000001.00001010
Subnet Mask: 11111111.11111111.11111111.00000000

Network ID	Host ID
192.168.1.0	0.0.0.10

CIDR BASICS

- CIDR (Classless Inter-Domain Routing)
- Notation: a.b.c.d/n where n = number of network bits

CIDR	SUBNET MASK	USABLE HOSTS
/8	255.0.0.0	16,777,214
/16	255.255.0.0	65,534
/24	255.255.255.0	254
/25	255.255.255.128	126
/26	255.255.255.192	62
/27	255.255.255.224	30
/28	255.255.255.240	14
/30	255.255.255.252	2

QUICK CIDR EXAMPLES

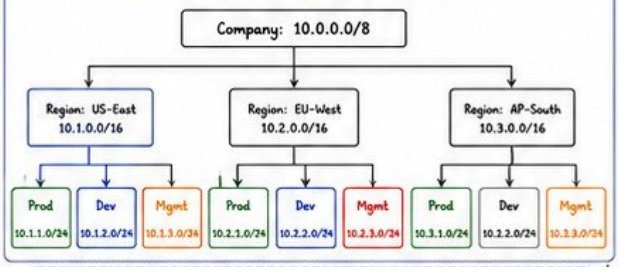
- 10.0.0.0/8 → 16,777,214 hosts
- 172.16.0.0/16 → 65,534 hosts
- 192.168.1.0/24 → 254 hosts
- 192.168.1.0/25 → 126 hosts
- 192.168.1.0/26 → 62 hosts
- 192.168.1.0/30 → 2 hosts

Remember: Usable Hosts = 2^(32–n) – 2 (Subtract network & broadcast)

ENTERPRISE ADDRESSING BEST PRACTICES

1. **Plan Before You Deploy**
   Design IP ranges, subnets, and VLANs before implementation.
2. **Use Hierarchical Design**
   Group by region → site → environment → tier.
3. **Reserve and Document**
   Reserve ranges for future growth and critical systems.
4. **Avoid IP Waste**
   Right-size your subnets. Don't use a /24 where a /26 is enough.
5. **Separate Network Functions**
   Management, production, development, and backups should be in different subnets/VLANs.

EXAMPLE: ENTERPRISE IP HIERARCHY



RESERVED IPs (Don't Use for Hosts)

- Network ID (first IP)
- Broadcast ID (last IP)
- Loopback: 127.0.0.0/8
- Link-Local: 169.254.0.0/16
- Multicast: 224.0.0.0/4

REAL-WORLD EXAMPLES

AWS VPC
Default VPC: 172.31.0.0/16
Custom VPCs use private CIDRs.

Kubernetes Pods
Pod CIDRs: 10.244.0.0/16 (Flannel default)

On-Prem Data Center
10.10.0.0/16 for Corp LAN
10.20.0.0/16 for Servers
10.30.0.0/16 for Storage

Hybrid Networks
Use non-overlapping CIDRs across cloud and on-prem.

SENIOR ENGINEER TIP: Great networks are invisible when they work and obvious when they don't. Design them for scale, security, and simplicity.

VERIQTA

6. SUBNETTING

"Good subnet design improves performance, security, scalability, and troubleshooting."

PRODUCTION NOTE
- ☑ Plan before you deploy
- ☑ Use the right size subnet
- ☑ Avoid overlaps
- ☑ Document everything

1. CIDR CALCULATIONS (CLASSLESS INTER-DOMAIN ROUTING)

HOW CIDR WORKS

1. Start with a base network (e.g., 192.168.1.0/24)
2. Borrow bits from the host portion to create subnets.
3. More bits borrowed = more subnets, fewer hosts per subnet.
4. Fewer bits borrowed = fewer subnets, more hosts per subnet.

EXAMPLE: 192.168.1.0/24

Network Portion | Host Portion

11000000 · 10101000 · 00000001 · 00000000

8 bits | 8 bits | 8 bits | 8 bits

/24 (24 bits network, 8 bits host)

256 total addresses (2^{8})

254 usable hosts

TIP: Learn binary. It makes subnetting faster and troubleshooting easier.

2. SUBNET MASKS QUICK REFERENCE

CIDR	SUBNET MASK	USABLE HOSTS
/8	255.0.0.0	16,777,214
/16	255.255.0.0	65,534
/17	255.255.128.0	32,766
/18	255.255.192.0	16,382
/19	255.255.224.0	8,190
/20	255.255.240.0	4,094
/21	255.255.248.0	2,046
/22	255.255.252.0	1,022
/23	255.255.254.0	510
/24	255.255.255.0	254
/25	255.255.255.128	126
/26	255.255.255.192	62
/27	255.255.255.224	30
/28	255.255.255.240	14
/29	255.255.255.248	6
/30	255.255.255.252	2

3. SUBNETTING WORKED EXAMPLE

Subnet 192.168.1.0/24 into 4 equal subnets.

Step 1: Borrow 2 bits (2^{2} = 4 subnets) → New prefix /26

Original Network	192.168.1.0/24 11000000.10101000.00000001.00000000
New Prefix	/26 11111111.11111111.11111111.11000000
Borrowed Bits	↑ ↑ 2 bits from host portion
Subnets Created	4 subnets Each subnet has 2^{6} - 2 = 62 usable hosts

SUBNET TABLE

Subnet #	Network ID	Usable Range	Broadcast
1	192.168.1.0/26	192.168.1.1 - 192.168.1.62	192.168.1.63
2	192.168.1.64/26	192.168.1.65 - 192.168.1.126	192.168.1.127
3	192.168.1.128/26	192.168.1.129 - 192.168.1.190	192.168.1.191
4	192.168.1.192/26	192.168.1.193 - 192.168.1.254	192.168.1.255

4. NETWORK PLANNING STEPS

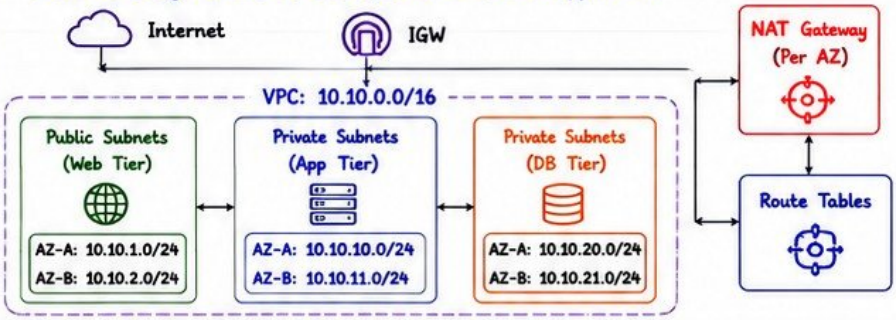
1. **1. Identify Requirements**
   Determine number of subnets and hosts.
2. **2. Choose IP Block**
   Select a private IP range large enough.
3. **3. Design Subnet Structure**
   Decide how many subnets and their sizes.
4. **4. Allocate Subnets**
   Assign subnets to departments, tiers, or services.
5. **5. Document the Plan**
   Maintain an IP addressing plan.

5. ADDRESS ALLOCATION GUIDELINES

TYPE	USE CASE	EXAMPLE	NOTES
Large Subnet (/24 -/20)	User networks, large departments	10.0.0.0/24	Many hosts, less subnets
Medium Subnet (/25 - /26)	Application tiers, server pools	10.0.1.0/25	Balanced subnets and hosts
Small Subnet (/27 - /28)	Point-to-point, VPN, Mgmt	10.0.1.128/27	Fewer hosts, more subnets
Very Small Subnet (/29 - /30)	P2P links, transit links	10.0.1.248/30	2-6 usable addresses

6. PRODUCTION SUBNET DESIGN EXAMPLE

Scenario: Design a VPC 10.10.0.0/16 for a 3-Tier Application



SUBNET TYPE	CIDR EXAMPLE	PURPOSE	NOTES
Public (Web)	10.10.1.0/24	Load balancers, web servers	Internet facing
Private (App)	10.10.10.0/24	Application servers	No direct internet
Private (DB)	10.10.20.0/24	Databases	Highly restricted
Mgmt	10.10.100.0/24	Bastion, monitoring, tools	Admin access only

7. SUBNETTING TIPS

- ☑ Plan for future growth.
- ☑ Keep similar systems in the same subnet.
- ☑ Separate critical systems (DB, Mgmt).
- ☑ Use /28 or /30 for point-to-point links.
- ☑ Avoid overlapping IP ranges.
- ☑ Document everything.

8. COMMON MISTAKES

- Using too many small subnets.
- Not leaving space for growth.
- Ignoring broadcast address.
- Ignoring broadcast limit.
- Overlapping IP ranges.
- Overlapping IP ranges.
- Poor documentation.

9. QUICK FORMULAS

Total Subnets = 2^{n} (n = bits borrowed)

Usable Hosts = 2^{h} - 2 (h = host bits)

Block Size = 256 - mask value in the interesting octet

10. PRIVATE IP RANGES (RFC 1918)

10.0.0.0/8	(10.0.0.0 - 10.255.255.255)
172.16.0.0/12	(172.16.0.0 - 172.31.255.255)
192.168.0.0/16	(192.168.0.0 - 192.168.255.255)

SENIOR ENGINEER TIP: Great subnetting is invisible to users but critical to the success and security of your infrastructure.

VERIQTA

7. MAC ADDRESSES & ETHERNET

"Layer 2 is where devices talk on the same network."
MAC addresses, Ethernet frames, and switches make local communication possible."

PRODUCTION NOTE
- ☑ Know your MACs.
- ☑ Understand your traffic.
- ☑ Control your broadcasts.
- ☑ Document everything.

1. MAC ADDRESSES BASICS

- MAC = Media Access Control address
- Layer 2 (Data Link Layer) identifier
- Unique to each network interface
- Usually permanently assigned by vendor
- 48 bits (6 bytes) written as 12 hex digits

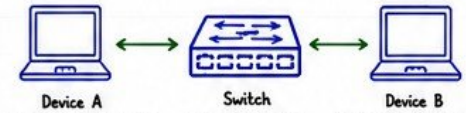
MAC ADDRESS FORMAT

`01 : 23 : 45 : 67 : 89 : AB`

- `01 : 23 : 45` — OUI (First 24 bits) — Assigned by IEEE
- `67 : 89 : AB` — Device Identifier (Last 24 bits) — Assigned by Vendor

MAC addresses never change (unless manually spoofed). They stay the same even if the device moves.

2. LAYER 2 COMMUNICATION



Device A
MAC: 00:11:22:33:44:55

Switch
Layer 2 Device

Device B
MAC: 00:11:22:33:44:66

- Layer 2 communication happens within the same network / VLAN.
- Devices use MAC addresses to deliver frames.
- Routers do NOT forward Layer 2 frames (they break broadcast domains).

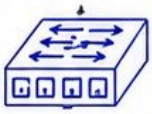
KEY POINT
Layer 2 = Local communication
Layer 3 = Routed communication

3. ETHERNET FRAME STRUCTURE

6B	6B	2B	46 – 1500B	4B
Destination MAC	Source MAC	Type /Length	Payload (Data)	FCS (CRC)

- **Destination MAC (6 bytes):** MAC of the intended receiver or broadcast.
- **Source MAC (6 bytes):** MAC of the sender.
- **Type/Length (2 bytes):** Indicates protocol type (e.g., 0x0800 = IPv4) or length of data.
- **Payload (46 – 1500 bytes):** Actual data being carried.
- **FCS (4 bytes):** Frame Check Sequence for error detection.

4. SWITCH FORWARDING PROCESS



HOW A SWITCH WORKS
1. Learns MAC addresses from source.
2. Builds a MAC address table (CAM table).
3. Looks up destination MAC.
4. Forwards frame out correct port.
5. If unknown MAC, floods to all ports except incoming port.

MAC ADDRESS TABLE (CAM TABLE EXAMPLE)

MAC ADDRESS	PORT	TYPE	AGE
00:11:22:33:44:55	Gi0/1	Dynamic	10s
00:11:22:33:44:66	Gi0/2	Dynamic	15s
00:11:22:33:44:77	Gi0/3	Dynamic	8s
00:11:22:33:44:88	Gi0/4	Dynamic	20s

Note: Entries age out if not seen for a period of time.

FORWARDING BEHAVIOR
- ☑ Known unicast → Forward only to destination port
- ☑ Unknown unicast → Flood all ports except incoming
- ☑ Broadcast → Flood all ports in the VLAN
- ☑ Multicast → Flood (unless IGMP Snooping is enabled)

5. ARP BASICS

ARP (Address Resolution Protocol) maps IP addresses to MAC addresses within a network. ARP is used before sending an IP packet.

ARP REQUEST FLOW

Host A
192.168.1.10

Host B
192.168.1.20

1. Host A needs to send data to 192.168.1.20 but doesn't know the MAC.
2. Host A sends ARP Request (broadcast):
   `Who has 192.168.1.20?`
   `Tell 192.168.1.10`
   `(ff:ff:ff:ff:ff:ff)`
3. Host B replies with ARP Reply (unicast):
   `192.168.1.20 is at`
   `00:11:22:33:44:66`
4. Host A caches the MAC and sends the data.

ARP CACHE EXAMPLE (Host A)

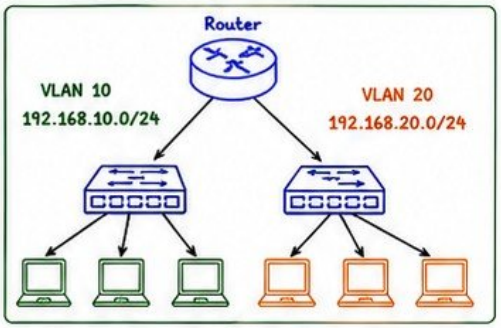
IP ADDRESS	MAC ADDRESS	AGE
192.168.1.20	00:11:22:33:44:66	3m
192.168.1.1	00:11:22:33:44:01	5m

View ARP Cache (Linux): `ip neigh show`
View ARP Cache (Windows): `arp -a`

6. BROADCAST DOMAINS

- A broadcast domain is a network segment where broadcast frames are received by all devices.
- Switches extend the broadcast domain.
- Routers break broadcast domains.
- VLANs create separate broadcast domains.

BROADCAST EXAMPLE



Broadcasts stay inside VLAN 10 only.

Broadcasts stay inside VLAN 20 only.

COMMON BROADCAST SOURCES
- ☑ ARP Requests
- ☑ DHCP Discovery
- ☑ Unknown Unicast Traffic
- ☑ L2 Loops (Spanning Tree prevents this)
- ☑ Misconfigurations

7. KEY TAKEAWAYS

- ☑ MAC addresses are Layer 2 identities.
- ☑ Ethernet frames carry data between devices.
- ☑ Switches learn MACs and forward intelligently.
- ☑ ARP maps IP to MAC within the local network.
- ☑ Routers break broadcast domains.
- ☑ VLANs help control broadcasts and improve performance.

COMMON ETHERNET TYPES (ETHERTYPE VALUES)

ETHERTYPE	PROTOCOL
0x0800	IPv4
0x0806	ARP
0x86DD	IPv6
0x8847	MPLS (Unicast)
0x8100	802.1Q VLAN Tag

PRODUCTION TIP
- ☑ Monitor broadcast traffic.
- ☑ Use VLANs to segment networks.
- ☑ Enable Port Security on access ports.
- ☑ Use ARP inspection on untrusted networks.
- ☑ Document MACs and IPs in critical systems.

VERIQTA

8. ROUTING FUNDAMENTALS

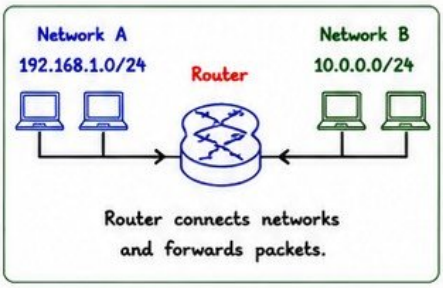
"Routers connect networks. Routing decides the best path for packets to reach their destination."

PRODUCTION NOTE

- ☑ Good routing = availability
- ☑ Bad routing = outages
- ☑ Always know your paths
- ☑ Document your routes

1. ROUTERS – THE TRAFFIC DIRECTORS

- Operate at Layer 3 (Network Layer).
- Connect different networks.
- Examine IP addresses and make forwarding decisions.
- Use routing tables to choose the best path.



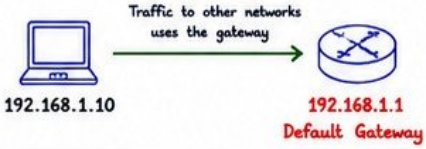
Router connects networks and forwards packets.

2. DEFAULT GATEWAY

- A router interface that hosts use to reach remote networks.
- If the destination IP is not in the local network, traffic is sent to the default gateway.
- Every device needs a default gateway (except routers).

EXAMPLE

Host IP	: 192.168.1.10
Subnet Mask	: 255.255.255.0 (/24)
Default Gateway	: 192.168.1.1



3. STATIC ROUTING

- Routes are manually configured by the admin.
- Simple, secure, predictable.
- Best for small networks or specific paths.

CISCO EXAMPLE

```
ip route 10.0.0.0 255.255.255.0 192.168.1.2
ip route 172.16.0.0 255.255.0.0 192.168.1.2
ip route 0.0.0.0 0.0.0.0 192.168.1.2
```

↑ Default route (route of last resort)

STATIC ROUTE TABLE EXAMPLE

Destination Network	Subnet Mask	Next Hop	Interface
10.0.0.0	255.255.255.0	192.168.1.2	Gi0/0
172.16.0.0	255.255.0.0	192.168.1.2	Gi0/0
0.0.0.0	0.0.0.0	192.168.1.2	Gi0/0

4. DYNAMIC ROUTING OVERVIEW

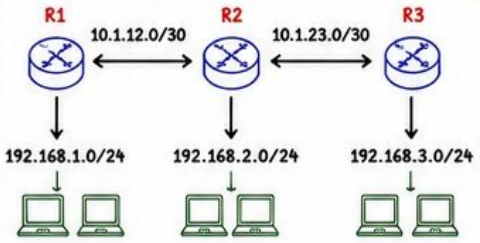
- Routers automatically learn routes from each other.
- Routing protocols exchange information.
- Adapts to network changes.
- Best for medium to large networks.

COMMON DYNAMIC ROUTING PROTOCOLS

PROTOCOL	TYPE	ADMIN DISTANCE	NOTES
RIP	Distance Vector	120	Simple, hop count (max 15)
OSPF	Link State	110	Fast, scalable, widely used
EIGRP	Advanced DV	90	Cisco proprietary, fast
BGP	Path Vector	20	Used between organizations (Internet routing)

Administrative Distance (AD): Lower value = more trusted route.

DYNAMIC ROUTING EXAMPLE (OSPF AREA 0)



Routers exchange routes and build a complete map of the network.

5. ROUTE SELECTION (LONGEST PREFIX MATCH)

- Router chooses the most specific match.
- If multiple routes match, the longest prefix wins.
- If same prefix length, use Administrative Distance.
- If same AD, use metric (cost).

ROUTE SELECTION EXAMPLE

Destination	Mask / Prefix	Next Hop	Metric
10.0.0.0	255.0.0.0 /8	192.168.1.2	1
10.10.0.0	255.255.0.0 /16	192.168.1.3	5
10.10.10.0	255.255.255.0 /24	192.168.1.4	10
10.10.10.5	255.255.255.255 /32	192.168.1.5	1

if destination is 10.10.10.5, router uses /32 route (most specific).

6. TYPES OF ROUTES

TYPE	DESCRIPTION	EXAMPLE
Connected	Directly connected networks	C 192.168.1.0/24
Static	Manually configured routes	S 10.0.0.0/8
Dynamic	Learned from routing protocols	O 172.16.0.0/16
Default	Route of last resort (0.0.0.0/0)	S* 0.0.0.0/0

ROUTE CODES (CISCO)

C = Connected	D = EIGRP
S = Static	R = RIP
O = OSPF	B = BGP

7. ROUTING TABLE EXAMPLE (SHOW IP ROUTE)

```
R1# show ip route
Codes: C - connected, S - static, O - OSPF, D - EIGRP, B - BGP, * - candidate default
Gateway of last resort is 192.168.1.2 to network 0.0.0.0

C    192.168.1.0/24 is directly connected, Gi0/0
C    10.1.12.0/30 is directly connected, Gi0/1
O    192.168.2.0/24 [110/10] via 10.1.12.2, 00:00:12, Gi0/1
S    10.10.0.0/16 [1/0] via 192.168.1.2
S*   0.0.0.0/0 [1/0] via 192.168.1.2
```

S* = Static default route (used when no other route matches).

8. COMMON ROUTING ISSUES

- ⚠ No route to host → Missing route or wrong gateway
- ⚠ Routing loop → Incorrect configuration
- ⚠ Asymmetric routing → Different paths in and out
- ⚠ Blackhole route → Traffic goes nowhere
- ⚠ Slow convergence → Poor routing design

9. KEY TAKEAWAYS

- ☑ Routers connect different networks and make path decisions.
- ☑ Hosts use a default gateway to reach remote networks.
- ☑ Static routes are manual, dynamic routes are automatic.
- ☑ Routers always choose the best path based on rules.
- ☑ Good routing design = stable and fast networks.

VERIQTA

9. DNS FUNDAMENTALS

"DNS translates human friendly names into IP addresses.
It is the phonebook of the Internet."

PRODUCTION NOTE

- ☑ Fast DNS = Fast Users
- ☑ Reliable DNS = High Availability
- ☑ Secure DNS = Protected Users
- ☑ Monitor DNS Always

1. NAME RESOLUTION PROCESS

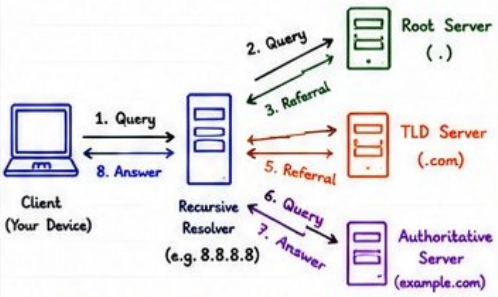
Converts a domain name (example.com) to an IP address (93.184.216.34)

1. User enters: www.example.com
2. DNS resolver is queried
3. Resolver asks other DNS servers
4. IP address is returned
5. Connection to IP is established

Without DNS, users must remember IP addresses instead of names!

2. RECURSIVE QUERIES EXPLAINED

- The client asks a recursive resolver.
- The resolver does the work on behalf of the client.
- It can contact multiple servers until it gets the answer.
- The final answer is returned to the client.



Client asks once (recursive).
Resolver asks many (iterative).

3. AUTHORITATIVE SERVERS

- Hold the actual DNS records for a domain.
- Provide definitive answers.
- Example: example.com authoritative server knows the real IP.

TYPES OF DNS RECORDS

TYPE	PURPOSE	EXAMPLE
A	Maps domain to IPv4	example.com → 93.184.216.34
AAAA	Maps domain to IPv6	example.com → 2606:2800::1
CNAME	Alias of another name	www → example.com
MX	Mail exchange server	10 mail.example.com
NS	Nameserver for domain	ns1.example.com
TXT	Text information	v=spf1 include:_spf.example.com
PTR	Reverse DNS lookup	34.216.184.93.in-addr.arpa

4. DNS HIERARCHY

ROOT (.)
13 Root Server Names
(a.root-servers.net)
Top of the DNS tree. Knows all TLD servers.

↓

TOP LEVEL DOMAIN (TLD)
.com .org .net .io .ng .uk
Knows authoritative servers for domains under it.

↓

AUTHORITATIVE NAMESERVER
example.com Nameservers
(ns1.example.com , ns2.example.com)
Knows all records for the domain.

↓

SUBDOMAINS & HOST RECORDS
www.example.com api.example.com mail.example.com
Actual hosts and services.

5. DNS QUERY TYPES

TYPE	DESCRIPTION	EXAMPLE
Recursive	Resolver gets the answer on behalf of client.	Client → Resolver
Iterative	Resolver returns referral and client asks again.	Resolver → Root → TLD → Authoritative
Forward	Domain name to IP	example.com → 93.184.216.34
Reverse	IP to Domain name	93.184.216.34 → example.com

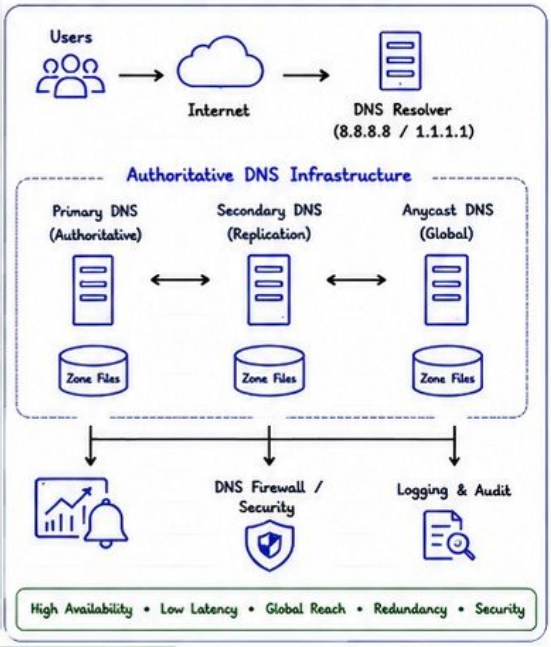
6. DNS CACHE & TTL

- DNS responses are cached to improve speed.
- TTL (Time To Live) defines how long the record is cached.
- Lower TTL = Faster changes, Higher load
- Higher TTL = Better performance, Slower changes

EXAMPLE TTL VALUES

A / AAAA Records	300 – 3600 seconds
MX Records	3600 seconds
NS Records	86400 seconds
TXT Records	Varies

7. PRODUCTION DNS ARCHITECTURE



COMMON DNS ISSUES

- DNS not resolving → Check DNS server
- Slow resolution → Check TTL, network, recursion
- Wrong IP returned → Check records, cache
- Intermittent failures → Check connectivity, firewalls
- NXDOMAIN errors → Domain does not exist
- SERVFAIL errors → Authoritative or network issue

9. ESSENTIAL DNS COMMANDS

```
nslookup example.com

dig example.com

# Detailed lookup
dig @8.8.8.8 example.com
# Query specific DNS
dig -x 8.8.8.8
# Reverse DNS lookup

host example.com
# Simple lookup
```

```
example.com          93.184.216.34
;; ANSWER SECTION:
example.com.         300   IN   A   93.184.216.34
Non-authoritative answer:
Name:    example.com
Address: 93.184.216.34

8.8.8.8.in-addr.arpa domain name pointer dns.google.

example.com has address 93.184.216.34
```

10. KEY TAKEAWAYS

- ☑ DNS makes the Internet usable.
- ☑ Resolvers perform the heavy lifting.
- ☑ Authoritative servers hold the truth.
- ☑ Caching improves performance.
- ☑ Good DNS design = reliability, speed, security.
- ☑ Always monitor your DNS.

VERIQTA

10. PORTS AND PROTOCOLS

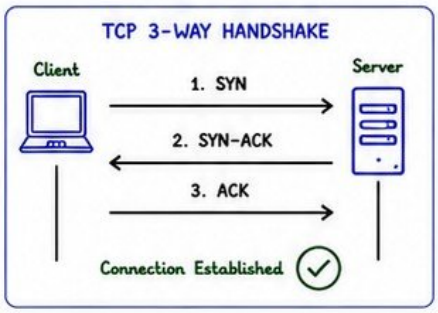
"Ports are the doors. Protocols are the rules.
Together they enable secure communication."

PRODUCTION NOTE

- ☑ Know your ports.
- ☑ Understand your protocols.
- ☑ Secure what you expose.
- ☑ Monitor every connection.
- ☑ Document everything.

1. TCP (TRANSMISSION CONTROL PROTOCOL)

- Connection-oriented (3-way handshake).
- Reliable, ordered delivery.
- Error checking and retransmission.
- Flow control and congestion control.
- Used where reliability is critical.



2. UDP (USER DATAGRAM PROTOCOL)

- Connectionless (no handshake).
- Faster, lightweight, lower overhead.
- No guarantee of delivery or order.
- Used for speed-sensitive applications.
- Best-effort delivery.

Use TCP for reliability.
Use UDP for speed.

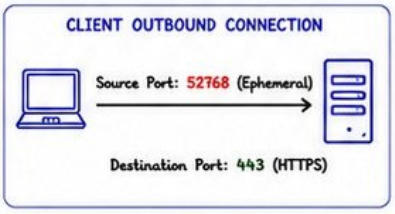
3. WELL-KNOWN PORTS (0 - 1023)

- Assigned by IANA.
- Used by system and common services.
- Do not require special configuration.
- Examples:

PORT	PROTOCOL	SERVICE	USE CASE
20	TCP	FTP Data	File transfer
21	TCP	FTP Control	File transfer
22	TCP	SSH	Secure remote access
53	UDP/TCP	DNS	Name resolution
80	TCP	HTTP	Web (insecure)
443	TCP	HTTPS	Web (secure)
25	TCP	SMTP	Email sending
110	TCP	POP3	Email retrieval
143	TCP	IMAP	Email retrieval
3306	TCP	MySQL	Database
5432	TCP	PostgreSQL	Database

4. EPHEMERAL (DYNAMIC/PRIVATE) PORTS (49152 - 65535)

- Used by clients for outbound connections.
- Assigned dynamically by the OS.
- Short-lived and released after session ends.
- Example range: 49152 - 65535 (varies by OS).



5. COMMON DEVOPS PROTOCOLS

PROTOCOL	PORT	TRANSPORT	DESCRIPTION / USE
SSH	22	TCP	Secure remote access
HTTP	80	TCP	Web traffic (insecure)
HTTPS	443	TCP	Secure web traffic
DNS	53	UDP/TCP	Name resolution
SMTP	25	TCP	Email sending
NTP	123	UDP	Time synchronization
SNMP	161/162	UDP	Monitoring & metrics
Syslog	514	UDP/TCP	Logging
LDAP	389	TCP	Directory services
LDAPS	636	TCP	Secure LDAP
Kubernetes API	6443	TCP	K8s API server
Docker Registry	5000/443	TCP	Image registry
Prometheus	9090	TCP	Metrics scraping

6. PORT CATEGORIES

Well-Known Ports (0 - 1023)	System services and standard applications.
Registered Ports (1024 - 49151)	Registered by organizations or vendors.
Ephemeral Ports (49152 - 65535)	Used temporarily by clients for outbound connections.

PORT CHECK EXAMPLE

```
# Check if port 22 is open on a server
$ nc -vz 192.168.1.10 22
Connection to 192.168.1.10 22 port [tcp/ssh] succeeded!

# Check listening ports on localhost
$ ss -tulnp
Netid   State    Recv-Q Send-Q   Local Address:Port
tcp     LISTEN   0      128      0.0.0.0:22
udp     UNCONN   0      0        0.0.0.0:53
```

7. TROUBLESHOOTING CONNECTIONS

1. CHECK CONNECTIVITY
   - ping <IP/Domain>
   - Test basic reachability.
2. CHECK PORT
   - nc -vz <IP> <port>
   - Test if port is open.
3. CHECK FIREWALL
   - iptables -L / nft list ruleset
   - firewall-cmd --list-all
4. CHECK SERVICE
   - systemctl status <service>
   - Ensure service is running.
5. CHECK LOGS
   - journalctl -u <service>
   - /var/log/syslog
6. VERIFY ROUTING
   - ip route
   - traceroute <IP>
7. DNS RESOLUTION
   - dig <domain>
   - nslookup <domain>

Always isolate the problem: Network → Port → Service → Application.

8. COMMON CONNECTION PROBLEMS

PROBLEM	POSSIBLE CAUSE	SOLUTION
Connection Refused	No service listening	Start service, check port
Connection Timed Out	Firewall / Network block	Check firewall, routes
Host Unreachable	No route to host	Check routing, gateway
DNS Lookup Failed	DNS server issue	Check DNS config, resolver
SSL Handshake Failed	Cert / TLS mismatch	Check certificate, time

9. ESSENTIAL PORT COMMANDS

ss -tulnp	Show listening ports
lsof -i :<port>	Show process using port
netstat -tulnp	(Legacy) show ports
nc -vz <IP> <port>	Test TCP port
nc -vu <IP> <port>	Test UDP port
telnet <IP> <port>	Test TCP connection
iptables -L -n	List firewall rules

10. KEY TAKEAWAYS

- ☑ Ports identify services. Protocols define how data is sent.
- ☑ TCP = reliable. UDP = fast.
- ☑ Well-known ports are standardized. Ephemeral ports are temporary.
- ☑ Secure only what is necessary.
- ☑ Always test, monitor and document your connections.

VERIQTA

11. LINUX NETWORK STACK

"The network stack is the path your packet takes from your application to the wire and back."

PRODUCTION NOTE

- ☑ Understand the stack.
- ☑ Know your tools.
- ☑ Observe before you change.
- ☑ Document everything.

1. WHAT IS THE LINUX NETWORK STACK?

- A layered model that handles network communication.
- Each layer has a specific responsibility.
- Data flows down the stack when sending, and up the stack when receiving.

NETWORK STACK OVERVIEW

7. Application Layer (HTTP, DNS, SSH)	SEND ↓
6. Presentation Layer (TLS/SSL, Encoding)	
5. Session Layer (Sessions, NetBIOS)	
4. Transport Layer (TCP, UDP)	
3. Network Layer (IP, ICMP)	
2. Data Link Layer (Ethernet, ARP)	
1. Physical Layer (Bits on the wire)	↑ RECEIVE

2. HOW DATA FLOWS (SEND PATH)

7	Application	Your application creates data
6	Presentation	Data is formatted, encrypted, compressed
5	Session	Session is created and managed
4	Transport (TCP/UDP)	Data is segmented and ports are added
3	Network (IP)	IP header added, routing decision
2	Data Link (Ethernet)	MAC header added (MAC addresses)
1	Physical	Bits are sent as electrical signals

↓ To the network...

3. HOW DATA FLOWS (RECEIVE PATH)

7	Application	Data delivered to the application
6	Presentation	Decrypt, decompress, convert format
5	Session	Session handling
4	Transport (TCP/UDP)	Reassemble segments, check ports
3	Network (IP)	Validate IP, check routing & TTL
2	Data Link (Ethernet)	Validate MAC, remove Ethernet header
1	Physical	Bits are received from the wire

↑ From the network...

KEY POINT

The same stack is used on both sides. One side sends, the other side receives.

COMMON OSI vs TCP/IP

OSI LAYER	TCP/IP LAYER
7. Application	Application
6. Presentation	Application
5. Session	Application
4. Transport	Transport
3. Network	Internet
2. Data Link	Network Access
1. Physical	Network Access

4. KEY LINUX NETWORK COMPONENTS

COMPONENT	DESCRIPTION
Network Interfaces	Connect the system to the network (eth0, ens33, wlan0)
IP Address	Logical address assigned to interface
Subnet Mask / CIDR	Defines network and host portions
Default Gateway	Next hop for external networks
Routing Table	Tells the kernel where to send packets
ARP Table	Maps IP addresses to MAC addresses
Sockets	Endpoint for network communication
Netfilter / iptables	Controls traffic flow and filtering

5. NETWORK INTERFACE INFORMATION

```
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever  preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
state UP group default qlen 1000
    link/ether 00:0c:29:1a:2b:3c brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.10/24 brd 192.168.1.255 scope global noprefixroute ens33
       valid_lft forever  preferred_lft forever
    inet6 fe80::20c:29ff:fe1a:2b3c/64 scope link noprefixroute
       valid_lft forever  preferred_lft forever
```

lo	= Loopback Interface	UP	= Interface is up
ens33	= Ethernet Interface	BROADCAST	= Supports broadcast
inet	= IPv4 Address	MULTICAST	= Supports multicast
inet6	= IPv6 Address		

6. ROUTING TABLE EXAMPLE

```
$ ip route
Destination     Gateway         Genmask         Interface
default         192.168.1.1     0.0.0.0         ens33
192.168.1.0     0.0.0.0         255.255.255.0   ens33
10.0.0.0        192.168.1.254   255.255.0.0     ens33
```

- default : Route for all unknown destinations.
- Gateway : Next hop router.
- Genmask : Subnet mask.
- Interface : Outgoing interface.

CHECK ROUTES

```
$ ip route show
$ route -n
```

7. ARP TABLE EXAMPLE

```
$ ip neigh
192.168.1.1   dev ens33   lladdr 00:50:56:c0:00:08   STALE
192.168.1.20  dev ens33   lladdr 00:0c:29:9d:6f:2a   REACHABLE
192.168.1.30  dev ens33   lladdr 00:0c:29:8c:1e:10   REACHABLE
```

REACHABLE	= Valid MAC
STALE	= Not used recently
FAILED	= Unreachable

CHECK ARP TABLE

```
$ ip neigh show
$ arp -n
```

8. IMPORTANT FILES AND PATHS

FILE / PATH	PURPOSE
/etc/network/interfaces	Interface configuration (Debian/Ubuntu)
/etc/sysconfig/network-scripts/	Interface config (RHEL/CentOS)
/etc/resolv.conf	DNS configuration
/etc/hosts	Static host mappings
/proc/net/	Network stack information
/var/log/syslog	Network and system logs

EDIT INTERFACE (TEMPORARY)

```
$ ip addr add 192.168.1.100/24 dev ens33
$ ip link set ens33 up
$ ip addr del 192.168.1.100/24 dev ens33
```

9. TROUBLESHOOTING LAYER BY LAYER

Application	Test with: curl, wget, nslookup, dig
Transport (TCP/UDP)	Test with: nc, telnet, ss -tulnp
Network (IP)	Test with: ping, traceroute, ip route
Data Link (Ethernet)	Test with: ip link, ethtool, tcpdump
Physical	Check cable, NIC status, switch port

QUICK COMMANDS CHEAT SHEET

```
ip a              # Show IP addresses
ip r              # Show routes
ip neigh          # Show ARP table
ss -tulnp         # Show listening ports
ping              # Test connectivity
traceroute        # Trace the path
tcpdump -i ens33  # Capture packets
ethtool ens33     # NIC information
```

KEY TAKEAWAYS

- ☑ The network stack is layered and works the same on both sides.
- ☑ Linux gives you powerful tools to observe every layer.
- ☑ Always test from the application down to the physical layer.
- ☑ Understanding the stack helps you solve problems faster.
- ☑ Measure, observe, document and then change.

VERIQTA

12. NETWORK INTERFACES

"Interfaces are the endpoints that connect your Linux system to the network."

PRODUCTION NOTE

- ☑ Know your interfaces.
- ☑ Monitor their health.
- ☑ Document configurations.
- ☑ Test changes safely.

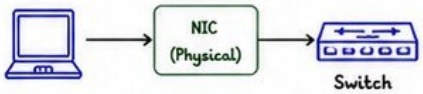
1. PHYSICAL INTERFACES

- Real hardware interfaces on the system.
- Connected to network devices via cables or modules.
- Names: eth0, enp0s3, ens33, eno1, etc.
- Managed by the OS and drivers.

EXAMPLE

```
$ ip link show
```

```
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
 link/ether 00:0c:29:3e:1a:b2 brd ff:ff:ff:ff:ff:ff
 inet 192.168.1.100/24 brd 192.168.1.255 scope global ens33
    valid_lft forever preferred_lft forever
```



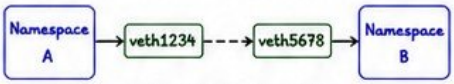
2. VIRTUAL INTERFACES

- Software-created interfaces.
- Not tied to physical hardware.
- Used for advanced networking, containers, VPNs, and overlays.
- Examples: veth, tun/tap, bridges, macvlan.

EXAMPLE

```
$ ip link show type virtual
```

```
5: veth1234@if4: <BROADCAST,MULTICAST,UP,LOWER_UP> ...
7: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> ...
9: cni0: <BROADCAST,MULTICAST,UP,LOWER_UP> ...
11: tun0: <POINTTOPOINT,MULTICAST,UP,LOWER_UP> ...
```



Virtual Link (veth pair)

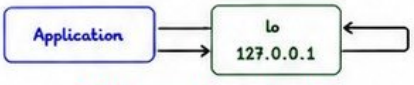
3. LOOPBACK INTERFACE

- Special virtual interface for local communication.
- Always present: lo
- IP: 127.0.0.1/8
- Used for testing and local services.

EXAMPLE

```
$ ip addr show lo
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
```



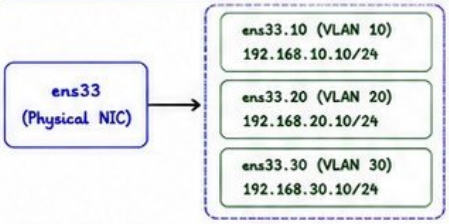
Traffic never leaves the host.

4. VLAN INTERFACES

- Logical interfaces mapped to a VLAN ID.
- Allow a single physical NIC to carry multiple VLANs.
- Syntax: <parent>.<vlan-id> (e.g., ens33.10)
- Common in servers, cloud, and multi-tenant setups.

EXAMPLE

```
$ ip link add link ens33 name ens33.10 type vlan id 10
$ ip addr add 192.168.10.10/24 dev ens33.10
```

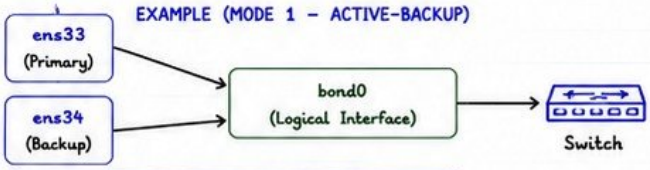


One physical interface, many logical networks.

5. BONDING OVERVIEW

- Combines multiple physical interfaces into one logical interface.
- Provides redundancy (failover) and/or increased throughput.
- Modes: 0 (balance-rr), 1 (active-backup), 4 (802.3ad/LACP), 6 (balance-alb)

EXAMPLE (MODE 1 – ACTIVE-BACKUP)



EXAMPLE CONFIG (RHEL/CENTOS)

```
DEVICE=bond0
TYPE=Bond
BONDING_MASTER=yes
BOOTPROTO=static
IPADDR=192.168.1.200
PREFIX=24
BONDING_OPTS="mode=1 miimon=100"
```

Benefits

- ☑ High Availability
- ☑ Link Aggregation
- ☑ No Single Point of Failure

6. INTERFACE STATES (ip link)

STATE	DESCRIPTION
UP	Interface is enabled and active.
DOWN	Interface is administratively down.
LOWER_UP	Physical link is detected (carrier).
LOWER_DOWN	Physical link is not detected.
UNKNOWN	Interface status is unknown.

CHECK INTERFACE STATUS

```
$ ip link show ens33
$ cat /sys/class/net/ens33/operstate
```

7. COMMON INTERFACE COMMANDS

`ip link show`	Show all interfaces
`ip addr show`	Show IP addresses on interfaces
`ip link set ens33 up`	Bring interface up
`ip link set ens33 down`	Bring interface down
`ip addr add 192.168.1.10/24 dev ens33`	Add IP address
`ip addr del 192.168.1.10/24 dev ens33`	Remove IP address
`ip link add link ens33 name ens33.10 type vlan id 10`	Create VLAN interface
`ip link del ens33.10`	Delete VLAN interface
`cat /proc/net/dev`	View interface statistics

8. KEY TAKEAWAYS

- ☑ Interfaces connect your system to networks.
- ☑ Linux supports physical and virtual interfaces.
- ☑ Loopback is essential for local communication.
- ☑ VLANs provide logical network segmentation.
- ☑ Bonding increases reliability and performance.
- ☑ Always monitor interface status in production.

BEST PRACTICES

- ☑ Use meaningful names (predictable names).
- ☑ Monitor link state and traffic.
- ☑ Document interface configurations.
- ☑ Test changes in non-production first.
- ☑ Use bonding for critical services.
- ☑ Secure interfaces and limit access.

INTERFACE LOCATION

Linux systems: /sys/class/net/
Config files: /etc/sysconfig/network-scripts/ (RHEL)
/etc/network/interfaces (Debian/Ubuntu)

VERIQTA

13. IP CONFIGURATION

"Correct IP configuration is the foundation of reliable network communication."

PRODUCTION NOTE

- ☑ Validate changes
- ☑ Test connectivity
- ☑ Document everything
- ☑ Have a rollback plan
- ☑ Monitor after changes

1. THE ip COMMAND (iproute2)

- Modern tool for network configuration.
- Replaces old ifconfig.
- Powerful, scriptable, and precise.
- Works for IP, routes, links, and more.

COMMON ip SYNTAX

```
ip [ OPTIONS ] OBJECT { COMMAND | help }
ip -br address show
ip -br link show
ip route show
ip -br addr show dev ens33
```

2. NetworkManager (nmcli)

- Service that manages network connections.
- Used on most servers and desktops.
- Simplifies connection management.
- Supports profiles for different networks.

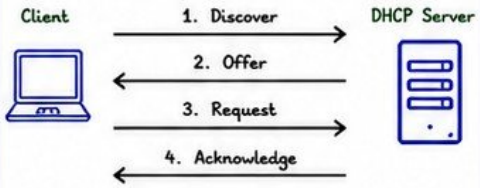
nmcli EXAMPLES

```
nmcli connection show
nmcli device status
nmcli connection modify ens33
  ipv4.addresses 192.168.10.10/24
nmcli connection up ens33
nmcli connection down ens33
```

3. DHCP (DYNAMIC IP)

- Automatically assigns IP and network info.
- Uses DORA process: Discover, Offer, Request, Acknowledge.
- Common in cloud, offices, and datacenters.

DHCP DORA PROCESS



4. STATIC IP ADDRESSING

- Manually assign IP, subnet, gateway, DNS.
- Used for servers, critical systems, appliances.
- Provides stability and predictability.

EXAMPLE: Static IP Settings

IP Address	192.168.10.20
Subnet Mask	255.255.255.0
Default Gateway	192.168.10.1
DNS Server	8.8.8.8, 1.1.1.1

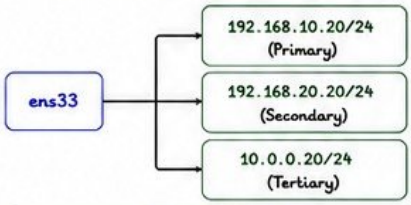
SET STATIC IP WITH ip COMMAND

```
# Add IP address
ip addr add 192.168.10.20/24 dev ens33
# Set default gateway
ip route add default via 192.168.10.1
# Add DNS (edit /etc/resolv.conf)
nameserver 8.8.8.8
nameserver 1.1.1.1
```

5. MULTIPLE IP ADDRESSES

- An interface can have multiple IPs.
- Useful for hosting multiple services or networks.
- Each IP must be in the same or different subnet.

EXAMPLE: Multiple IPs on ens33



ADD MULTIPLE IPs

```
ip addr add 192.168.10.20/24 dev ens33
ip addr add 192.168.20.20/24 dev ens33
ip addr add 10.0.0.20/24 dev ens33
ip addr show dev ens33
```

6. IP CONFIGURATION FILES

DISTRIBUTION	CONFIG FILE (STATIC IP)
RHEL / CentOS 7/8+	/etc/sysconfig/network-scripts/ifcfg-ens33
Rocky / AlmaLinux	/etc/sysconfig/network-scripts/ifcfg-ens33
Ubuntu / Debian	/etc/netplan/01-netcfg.yaml
openSUSE	/etc/sysconfig/network/ifcfg-ens33

EXAMPLE: ifcfg-ens33 (RHEL/CentOS)

```
TYPE=Ethernet
BOOTPROTO=static
NAME=ens33
DEVICE=ens33
ONBOOT=yes
IPADDR=192.168.10.20
PREFIX=24
GATEWAY=192.168.10.1
DNS1=8.8.8.8
DNS2=1.1.1.1
```

APPLY CHANGES

```
# RHEL/CentOS
nmcli connection
reload
nmcli connection
up ens33
or
systemctl restart
NetworkManager
```

7. VIEW IP INFORMATION

ip addr show	Show all IP addresses
ip -br addr	Brief IP summary
ip addr show dev ens33	Show IPs on specific interface
ip route show	Show routing table
hostname -I	Show all IP addresses

SAMPLE OUTPUT (ip addr show dev ens33)

```
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
 link/ether 00:0c:29:3e:1a:b2 brd ff:ff:ff:ff:ff:ff
 inet 192.168.10.20/24 brd 192.168.10.255 scope global ens33
    valid_lft forever preferred_lft forever
 inet 192.168.20.20/24 brd 192.168.20.255 scope global secondary ens33
    valid_lft forever preferred_lft forever
 inet 10.0.0.20/24 brd 10.0.0.255 scope global secondary ens33
    valid_lft forever preferred_lft forever
 inet6 fe80::20c:29ff:fe3e:1ab2/64 scope link
    valid_lft forever preferred_lft forever
```

8. TROUBLESHOOTING IP ISSUES

PROBLEM	POSSIBLE CAUSE	SOLUTION
No IP address	Interface down or DHCP failed	ip link set up dev ens33 Check DHCP server
Wrong IP	DHCP issue or wrong config	Release & renew DHCP Check settings
Cannot reach gateway	Wrong gateway or route missing	Check route table Verify gateway
No internet	DNS or upstream issue	Test DNS Check upstream route
Duplicate IP	IP conflict	Use unique IP Check DHCP lease

QUICK TESTS

ping 8.8.8.8	Test connectivity
ping google.com	Test DNS resolution
traceroute 8.8.8.8	Trace network path
ip route get 8.8.8.8	See route decision

9. KEY TAKEAWAYS

- ☑ Use ip command for precise control.
- ☑ NetworkManager simplifies management.
- ☑ DHCP is convenient but not always reliable.
- ☑ Static IPs provide stability for servers.
- ☑ Multiple IPs allow flexibility.
- ☑ Always test and validate changes.
- ☑ Document every change for rollback.
- ☑ Monitor interfaces and network health.

BEST PRACTICE

Plan your IP addressing scheme before deployment. A clean plan prevents conflicts and downtime.

VERIQTA

14. ROUTING TABLES

"Routing tables tell your system where to send packets. Good routes build reliable networks."

PRODUCTION NOTE

- ☑ Know your routes.
- ☑ Validate default gateway.
- ☑ Monitor route changes.
- ☑ Document network paths.
- ☑ Test failover paths.

1. WHAT IS A ROUTING TABLE?

- A routing table is a list of routes.
- Each route tells the system where to send traffic.
- Kernel uses the best route match.
- Routes can be static or dynamic.
- IPv4 and IPv6 have separate tables.

ROUTE ENTRY FIELDS

Destination	– Network or subnet
Gateway	– Next hop IP
Genmask	– Subnet mask
Interface	– Outgoing interface
Metric	– Route preference (lower is better)

2. THE ip route COMMAND

- Used to view and modify the routing table.
- Works with both IPv4 and IPv6.
- Changes are immediate but not permanent unless saved.

COMMON ip route COMMANDS

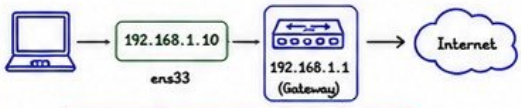
```
ip route show                  # Show routing table
ip route show table main       # Show main table
ip route add <dest> via <gw> dev <iface> metric<m>
ip route del <dest> via <gw> dev <iface>
ip route replace <dest> via <gw> dev <iface>
ip route add default via <gw> dev <iface>
ip route flush dev <iface>     # Clear routes on iface
```

3. DEFAULT ROUTES

- Default route (0.0.0.0/0) is used when no specific route matches.
- Points traffic to the gateway of last resort.
- Essential for internet access.

ADD DEFAULT ROUTE

```
ip route add default via 192.168.1.1 dev ens33
```



All traffic without a specific route uses the default route.

4. VIEW ROUTING TABLE (EXAMPLE)

```
$ ip route show
```

Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
default	192.168.1.1	0.0.0.0	UG	100	0	0	ens33
192.168.1.0	0.0.0.0	255.255.255.0	U	100	0	0	ens33
10.0.0.0	192.168.1.254	255.0.0.0	UG	100	0	0	ens33
172.16.0.0	192.168.1.2	255.240.0.0	UG	200	0	0	ens33

FLAGS EXPLAINED

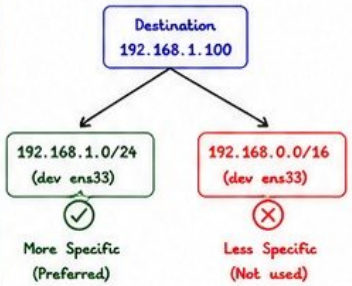
U	– Route is up
G	– Uses gateway
H	– Target is a host
D	– Installed by redirect
M	– Modified
!	– Reject route

EXAMPLE: UNDERSTANDING THE TABLE

default via 192.168.1.1	→ Internet traffic
192.168.1.0/24 dev ens33	→ Local network
10.0.0.0/8 via 192.168.1.254	→ Remote network
172.16.0.0/12 via 192.168.1.2	→ Another network

5. MULTIPLE ROUTES

- You can have many routes to different destinations.
- More specific routes (longer prefix) are preferred.
- Example: /24 is preferred over /16.



6. ROUTE METRICS

- Metric (cost) decides preference when multiple routes exist.
- Lower metric = higher priority.
- Used in static and dynamic routing.

EXAMPLE: SAME DESTINATION, DIFFERENT METRIC

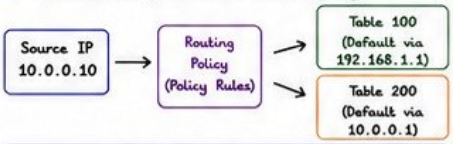
```
ip route add 10.0.0.0/8 via 192.168.1.1 dev ens33 metric 100
ip route add 10.0.0.0/8 via 192.168.1.2 dev ens33 metric 200
```

7. ADDING & DELETING ROUTES (EXAMPLES)

```
# Add route to a network
ip route add 10.10.10.0/24 via 192.168.1.1 dev ens33
# Add host route
ip route add 10.10.10.5/32 via 192.168.1.1 dev ens33
# Add route with metric
ip route add 10.20.0.0/16 via 192.168.1.2 dev ens33 metric 150
# Delete a route
ip route del 10.10.10.0/24 via 192.168.1.1 dev ens33
# Replace a route
ip route replace 10.20.0.0/16 via 192.168.1.254 dev ens33
# Flush all routes on an interface
ip route flush dev ens33
```

8. POLICY ROUTING OVERVIEW

- Policy routing allows different rules based on source, user, or interface.
- Uses multiple routing tables.
- Advanced but powerful for multi-homing.



BASIC POLICY ROUTING COMMANDS

```
ip rule show                                  # Show rules
ip rule add from 10.0.0.10 table 100
ip rule add from 192.168.1.50 table 200
ip route add default via 192.168.1.1 table 100
ip route add default via 10.0.0.1 table 200
```

9. KEY TAKEAWAYS

- ☑ Routing tables decide where packets go.
- ☑ Use ip route to view and manage routes.
- ☑ Default route is essential for internet.
- ☑ Most specific route wins.
- ☑ Metrics control route preference.
- ☑ Multiple tables enable advanced routing.
- ☑ Always test and document your routes.

TROUBLESHOOT ROUTING

- ☑ Check routes: ip route show
- ☑ Check connectivity: ping <destination>
- ☑ Trace path: traceroute <destination>
- ☑ Check interface: ip addr show <iface>
- ☑ Check gateway: ping <gateway-ip>

COMMON DEFAULT ROUTES

Internet (IPv4):	0.0.0.0/0
Internet (IPv6):	::/0
Localhost (IPv4):	127.0.0.0/8
Localhost (IPv6):	::1/128

BEST PRACTICE

- ☑ Desing routes before deployment.
- ☑ Keep metrics consistent.
- ☑ Use specific routes for critical paths.
- ☑ Document routing for troubleshooting.
- ☑ Monitor route changes and failures.

VERIQTA

15. ARP AND NEIGHBOR DISCOVERY

"Before data can travel, devices must discover each other at the link layer."

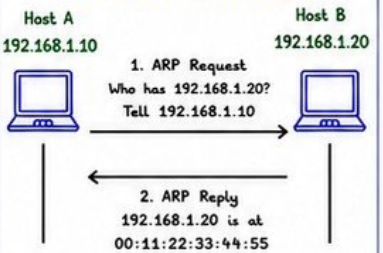
PRODUCTION NOTE

- ☑ Understand how ARP works.
- ☑ Monitor your ARP cache.
- ☑ Use static ARP for critical systems.
- ☑ Know IPv6 Neighbor Discovery.
- ☑ Document fixes and patterns.

1. WHAT IS ARP?

- Address Resolution Protocol.
- Resolves IPv4 addresses to MAC addresses.
- Operates at Layer 2.
- Required before sending packets to local network devices.

ARP REQUEST / REPLY FLOW



2. ARP CACHE (NEIGHBOR TABLE FOR IPv4)

- Devices store resolved IP → MAC mappings.
- Speeds up communication.
- Entries age out after a timeout.
- View and manage with ip command.

```
$ ip neigh show
192.168.1.1    dev ens33   lladdr 08:00:27:11:22:33   REACHABLE
192.168.1.20   dev ens33   lladdr 00:11:22:33:44:55   STALE
192.168.1.30   dev ens33   lladdr 00:11:22:33:44:66   DELAY
192.168.1.40   dev ens33   lladdr 00:11:22:33:44:77   FAILED
```

COMMON ARP STATES

REACHABLE	– Entry is valid and can be used
STALE	– Entry may be out of date, verify before use
DELAY	– Waiting before sending ARP probe
PROBE	– ARP probes are being sent
FAILED	– Resolution failed
INCOMPLETE	– Waiting for ARP reply

3. USEFUL ARP COMMANDS

COMMAND	DESCRIPTION
ip neigh show	Show ARP/neighbor table
ip neigh show dev ens33	Show table for specific interface
ip neigh add 192.168.1.20 lladdr 00:11:22:33:44:55 dev ens33	Add static ARP entry
ip neigh replace 192.168.1.20 lladdr 00:11:22:33:44:55 dev ens33	Replace or add ARP entry
ip neigh del 192.168.1.20 dev ens33	Delete ARP entry
ip neigh flush dev ens33	Clear ARP cache on interface

BEST PRACTICE

- ☑ Use static ARP for critical infrastructure.
- ☑ Monitor for unexpected ARP changes.
- ☑ Flush ARP cache during troubleshooting.

4. IPv6 NEIGHBOR DISCOVERY (ND)

- Replaces ARP in IPv6.
- Discovers neighbors and resolves addresses.
- Uses ICMPv6 messages.
- Also handles router discovery, prefix discovery, and more.

KEY ICMPv6 ND MESSAGES

Neighbor Solicitation (NS)	– Who has this IPv6?
Neighbor Advertisement (NA)	– This is my IPv6
Router Solicitation (RS)	– Ask for routers
Router Advertisement (RA)	– Router information
Redirect Message (RD)	– Better next hop
Multicast Listener Discovery (MLD)	– Group mgmt

5. IPv6 NEIGHBOR CACHE

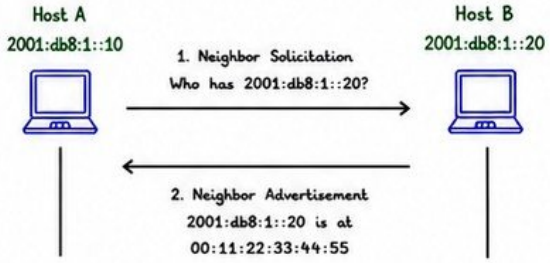
- Similar to ARP cache but for IPv6.
- Stores IPv6 → MAC mappings.
- View with ip -6 neigh.

```
$ ip -6 neigh show
fe80::1 dev ens33  lladdr 08:00:27:11:22:33   REACHABLE
fe80::a00:27ff:fe11:2233 dev ens33  lladdr
00:11:22:33:44:55                             STALE
fe80::abcd:ef12:3456:789a dev ens33  lladdr
00:11:22:33:44:66                             DELAY
```

COMMON ND STATES

REACHABLE	– Entry is valid
STALE	– Needs verification
DELAY	– Waiting before probing
PROBE	– Sending Neighbor Solicitation
FAILED	– Could not reach neighbor

6. ND PROCESS EXAMPLE



NOTE

IPv6 uses multicast (FF02::1:FFXX:XXXX) for ND, not broadcast like ARP.

7. TROUBLESHOOTING ARP AND NEIGHBOR DISCOVERY

COMMON PROBLEMS

- No ARP entry
- Stale or incorrect entry
- Duplicate IP addresses
- Gratuitous ARP not working
- ND cache not updating
- Firewall blocking ND/ICMPv6

DIAGNOSIS COMMANDS

☑ ip neigh show	Check ARP/ND cache
☑ arp -n	Traditional ARP table
☑ ip -s neigh show dev ens33	Stats on neighbors
☑ tcpdump -i ens33 arp	Capture ARP packets
☑ tcpdump -i ens33 icmp6	Capture ND packets
☑ ping -c 3 <ip>	Test reachability
☑ ip addr show dev ens33	Check IP on interface

ARP/ND CACHE STATES EXPLAINED

- REACHABLE: Entry confirmed, can be used.
- STALE: Entry may be outdated; first packet will trigger validation.
- DELAY: System waits before sending probe.
- PROBE: Probes are sent to confirm neighbor.
- FAILED: No response to probes.
- INCOMPLETE: Waiting for ARP reply.

FIXES

- ☑ Flush ARP cache: ip neigh flush dev ens33
- ☑ Add static ARP: ip neigh add <ip> lladdr <mac> dev ens33
- ☑ Renew DHCP lease: dhclient -r && dhclient
- ☑ Check for IP conflicts.
- ☑ Verify switch port/VLAN settings.
- ☑ Allow ICMPv6 (ND) through firewalls.
- ☑ Restart networking if needed.

8. ARP vs ND COMPARISON

FEATURE	ARP (IPv4)	ND (IPv6)
Protocol	ARP (RFC 826)	ICMPv6 (RFC 4861)
Purpose	IP → MAC resolution	Address + router discovery
Transport	Layer 2 (Broadcast)	Layer 2 (Multicast)
Messages	Request / Reply	NS / NA, RS / RA, RD
Security	No authentication	Supports SEND (IPsec)
Table Command	ip neigh show	ip -6 neigh show

9. QUICK REFERENCE

VIEW TABLES

```
ip neigh show
ip neigh show dev <iface>
ip -6 neigh show
```

ADD STATIC ENTRY (IPv4)

```
ip neigh add <ip>
lladdr <mac> dev <iface>
```

FLUSH TABLES

```
ip neigh flush dev <iface>
ip -6 neigh flush dev <iface>
```

ADD STATIC ENTRY (IPv6)

```
ip -6 neigh add <ip>
lladdr <mac> dev <iface>
```

10. KEY TAKEAWAYS

- ☑ ARP/ND is essential for local network communication.
- ☑ Monitor your neighbor tables regularly.
- ☑ Stale or wrong entries cause connectivity issues.
- ☑ IPv6 ND does more than ARP; it's critical for IPv6.
- ☑ Always test, validate, and document fixes.

"If devices can't find each other at Layer 2, no amount of Layer 3 engineering will help."

VERIQTA

16. LINUX DNS CONFIGURATION

"DNS is the phonebook of the internet."
Correct configuration = fast, reliable, and secure name resolution."

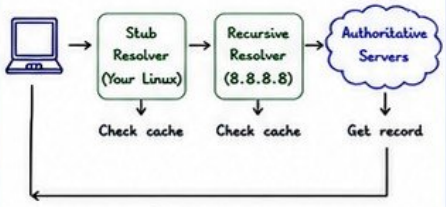
PRODUCTION NOTE

- ☑ Validate DNS before apps.
- ☑ Monitor DNS latency.
- ☑ Use redundant resolvers.
- ☑ Document DNS design.
- ☑ Test resolution from every network segment.

1. LINUX DNS OVERVIEW

- Translates domain names to IP addresses.
- Configured at OS level.
- Used by all applications and services.
- Components: resolvers, caches, and name servers.

DNS RESOLUTION FLOW



2. RESOLV.CONF (LEGACY METHOD)

- Main configuration file for DNS resolvers.
- Location: /etc/resolv.conf
- Contains nameserver, search, and options.
- May be managed by NetworkManager or systemd-resolved.

EXAMPLE /etc/resolv.conf

```
# Generated by NetworkManager
nameserver 8.8.8.8
nameserver 1.1.1.1
search veriqta.local corp.local
options ndots:5 timeout:2 attempts:2 rotate
# ndots:5 → how many dots before querying full FQDN
# rotate → round-robin between nameservers
```

IMPORTANT
Do not edit /etc/resolv.conf manually if your system uses NetworkManager or systemd-resolved. Your changes may be overwritten.

3. SYSTEMD-RESOLVED (MODERN METHOD)

- Local resolver service and DNS stub listener.
- Caches responses and manages DNS servers.
- Works with DNSSEC, LLMNR, mDNS.
- Configuration file: /etc/systemd/resolved.conf

CHECK STATUS

```
$ systemctl status systemd-resolved
```

COMMON COMMANDS

$ resolvectl status	Show resolver status
$ resolvectl dns	Show DNS servers
$ resolvectl domain	Show search domains
$ resolvectl query google.com	Test DNS query
$ resolvectl flush-caches	Flush DNS cache
$ resolvectl statistics	Show statistics

LISTENING ADDRESSES (DEFAULT)

- 127.0.0.53 → Stub resolver (for apps)
- ::1 (IPv6) → Stub resolver (for apps)

4. CONFIGURE DNS WITH SYSTEMD-RESOLVED

Temporarily (runtime)

```
$ sudo resolvectl dns eth0 8.8.8.8 1.1.1.1
$ sudo resolvectl domain eth0 veriqta.local corp.local
$ resolvectl status eth0
```

Permanently

```
# /etc/systemd/network/eth0.network
[Match]
Name=eth0
[Network]
DNS=8.8.8.8 1.1.1.1
Domains=veriqta.local corp.local
```

Apply changes:

```
$ sudo systemctl restart systemd-networkd
```

5. DNS TROUBLESHOOTING

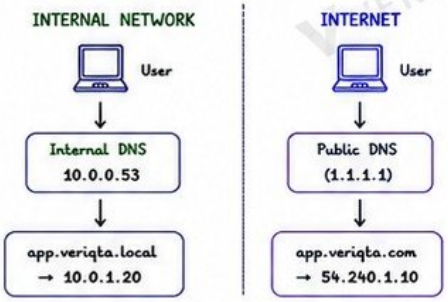
CHECK	COMMAND	WHAT IT SHOWS
Resolver status	resolvectl status	DNS servers, domains, mode
DNS servers	cat /etc/resolv.conf resolvectl dns	Current DNS configuration
Test resolution	dig google.com nslookup google.com host google.com	DNS query result
Check cache	resolvectl statistics	Cache hits, misses
Flush cache	resolvectl flush-caches	Clear local DNS cache
Route to DNS	ip route get 8.8.8.8	Network path to resolver
Connectivity	ping 8.8.8.8	Network reachability
Port check	nc -zv 8.8.8.8 53	DNS port (UDP/TCP 53)

COMMON DIG EXAMPLES

```
$ dig veriqta.com              # Basic A record query
$ dig veriqta.com +short       # Short output
$ dig veriqta.com MX           # MX records
$ dig veriqta.com NS           # Name servers
$ dig @1.1.1.1 veriqta.com     # Query specific DNS server
$ dig -x 8.8.8.8               # Reverse DNS lookup
```

6. SPLIT DNS (SPLIT-HORIZON DNS)

- Different DNS answers for internal vs external users.
- Internal users resolve private IPs.
- External users resolve public IPs.



IMPLEMENTATION OPTIONS

- Internal DNS server (BIND, CoreDNS, AWS Route 53 Private Zone)
- Conditional forwarders on DNS server
- Views on BIND
- Cloud provider private hosted zones

7. NETWORKMANAGER DNS CONTROL

- Manages DNS via connection profiles.
- Commands:

```
$ nmcli connection show
$ nmcli connection modify eth0 ipv4.dns "8.8.8.8 1.1.1.1"
$ nmcli connection modify eth0 ipv4.dns-search "veriqta.local"
$ nmcli connection up eth0
```

NOTE
If NetworkManager is managing DNS, manual edits to /etc/resolv.conf will be lost.

8. VERIFY DNS FROM DIFFERENT LAYERS

LAYER	TEST	EXAMPLE COMMAND
Application	Uses system resolver	ping veriqta.com
Stub Resolver	Local resolver	resolvectl query veriqta.com
Recursive Resolver	Upstream resolver	dig @8.8.8.8 veriqta.com
Authoritative Server	Final source	dig @ns1.veriqta.com veriqta.com
Network	Connectivity	traceroute 8.8.8.8

TIP
Always test from the server itself, not just from your laptop. Firewalls, routing, and DNS policies may differ.

9. KEY TAKEAWAYS

- ☑ DNS is critical for every service.
- ☑ Use systemd-resolved for modern systems.
- ☑ Never ignore DNS in troubleshooting.
- ☑ Validate from every network and segment.
- ☑ Split DNS improves security and performance.
- ☑ Monitor latency, errors, and cache hit ratio.

VERIQTA

17. LINUX FIREWALL FUNDAMENTALS

"A firewall controls what enters and leaves your system."
Default deny. Allow only what is necessary."

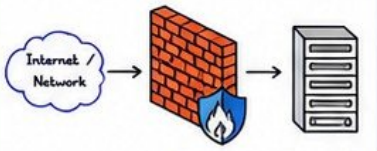
PRODUCTION NOTE

- ☑ Default policy should be DROP.
- ☑ Allow only what is required.
- ☑ Log and monitor firewall events.
- ☑ Test rules before enabling.
- ☑ Keep rules simple and clear.

1. FIREWALL OVERVIEW

- Firewalls filter network traffic.
- Protect systems from unauthorized access.
- Enforce security policies.
- Operate at Network Layer (L3/L4).
- Tools: nftables (modern), iptables (legacy), firewalld (management layer).

TRAFFIC FLOW THROUGH FIREWALL



2. nftables (MODERN FIREWALL)

- Successor to iptables.
- More powerful, fast, and consistent.
- Uses tables, chains, rules.
- Integrated with kernel.

```
# List rules
$ sudo nft list ruleset
```

nftables BASIC EXAMPLE

```
table inet filter {
chain input {
   type filter hook input priority 0; policy drop;
   iif "lo" accept
   ct state established,related accept
   tcp dport 22 accept
   tcp dport 80 accept
   tcp dport 443 accept

}
chain forward { type filter hook forward priority 0; policy drop; }
chain output { type filter hook output priority 0; policy accept; }
}
```

Common nftables commands

```
$ sudo nft list ruleset          # Show all rules
$ sudo nft add rule inet filter input tcp dport 22 accept
$ sudo nft delete rule inet filter input handle <ID>
$ sudo nft flush ruleset         # Remove all rules
```

3. iptables (LEGACY OVERVIEW)

- Traditional firewall tool.
- Still widely used.
- Uses tables: filter, nat, mangle, raw.
- Based on chains and rules.

```
# List rules
$ sudo iptables -L -n -v
```

iptables BASIC EXAMPLE (filter table)

```
# Accept established connections
$ sudo iptables -A INPUT -m conntrack --ctstate \
      ESTABLISHED,RELATED -j ACCEPT

# Allow SSH, HTTP, HTTPS
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
$ sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Drop all other incoming
$ sudo iptables -A INPUT -j DROP
```

Common iptables commands

```
$ sudo iptables -L -n -v              # List rules
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
$ sudo iptables -D INPUT <rule>       # Delete rule
$ sudo iptables -F                    # Flush rules
$ sudo iptables-save                  # Save rules
```

4. FIREWALLD (DYNAMIC MANAGEMENT)

- High-level firewall manager.
- Uses zones and services.
- Backend: nftables (modern systems).
- Easy runtime changes.

ZONES IN FIREWALLD

- public – Default, for public networks
- internal – Internal networks
- trusted – All traffic allowed
- dmz – Demilitarized zone
- block – Drops all incoming
- drop – Drops all incoming (no reject)
- external – For masqueraded networks

Common firewalld commands

```
$ sudo firewall-cmd --state
$ sudo firewall-cmd --list-all
$ sudo firewall-cmd --list-services
$ sudo firewall-cmd --add-service=ssh --permanent
$ sudo firewall-cmd --remove-service=ftp --permanent
$ sudo firewall-cmd --reload
$ sudo firewall-cmd --set-default-zone=public
```

5. PACKET FILTERING PROCESS (How it works)

FILTER TABLE CHAINS (Most important)

INPUT – For packets destined to the local system

FORWARD – For packets passing through the system

OUTPUT – For packets generated by the system

DEFAULT POLICIES (RECOMMENDED)

INPUT	DROP
FORWARD	DROP
OUTPUT	ACCEPT

BEST PRACTICE

- ☑ Deny everything by default.
- ☑ Allow only required traffic.
- ☑ Log dropped traffic.
- ☑ Review rules regularly.
- ☑ Test before applying permanently.

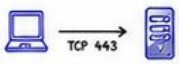
6. PACKEEN CRITERIA

Firewalls can match packets using:

- ☑ Source IP / Destination IP
- ☑ Protocol (TCP, UDP, ICMP)
- ☑ Port numbers
- ☑ Interface
- ☑ Connection state
- ☑ MAC address
- ☑ Packet size
- ☑ Time, rate limiting, and more

EXAMPLE MATCH

Allow TCP traffic from 10.0.0.0/24 to port 443 only



7. LOGGING DROPPED PACKETS

- Logging helps in troubleshooting.
- Example with nftables:

```
$ sudo nft add rule inet filter input \
ct state invalid counter log prefix "DROP: " flags all drop
```

Example with iptables:

```
$ sudo iptables -A INPUT -m limit --limit 5/min \
  -j LOG --log-prefix "IPTABLES-DROP: " --log-level 4
$ sudo iptables -A INPUT -j DROP
```

8. QUICK REFERENCE

TASK	nftables	iptables	firewalld
List rules	nft list ruleset	iptables -L -n -v	firewall-cmd --list-all
Allow SSH	nft add rule inet filter \ input tcp dport 22 accept	iptables -A INPUT -p tcp --dport 22 -j ACCEPT	firewall-cmd --add-service=ssh \ --permanent
Drop all incoming	Set policy drop in input chain	iptables -P INPUT DROP	firewall-cmd --set-default-zone=drop
Reload rules	nft -f /etc/nftables.conf	iptables-restore < file	firewall-cmd --reload
Save rules	nft list ruleset > file	iptables-save > file	Permanent = saved automatically

COMMON PITFALLS

- ☒ Opening unnecessary ports.
- ☒ Forgetting to save rules.
- ☒ Setting default policy to ACCEPT.
- ☒ Not testing before applying rules.
- ☒ Ignoring IPv6 firewall rules.

REMEMBER

A firewall is your first line of defense.

Principle: Least privilege.

Allow what you need. Deny what you don't.

KEY TAKEAWAYS

- ☑ nftables is the future. Learn it.
- ☑ iptables is legacy but still important.
- ☑ firewalld simplifies firewall management.
- ☑ Packet filtering protects your Linux systems.
- ☑ Always test, monitor, and document your rules.

VERIQTA

18. NETWORK DIAGNOSTIC COMMANDS

"Great engineers don't guess — they investigate."
These commands reveal the truth."

PRODUCTION NOTE

- ☑ Verify layer by layer.
- ☑ Gather evidence before acting.
- ☑ Know what normal looks like.
- ☑ Use the right tool for the job.
- ☑ Document findings.

1. PING

Purpose

Check basic connectivity and packet loss.

Example

```
$ ping google.com
$ ping -c 4 8.8.8.8
```

What it shows

- Host reachable or not
- Latency (round-trip time)
- Packet loss
- Basic network health

2. TRACEROUTE

Purpose

Show the path packets take to the destination.

Example

```
$ traceroute google.com
$ traceroute -n 8.8.8.8
```

What it shows

- Each hop to destination
- Latency per hop
- Where packets stop
- Routing issues

3. TRACEPATH

Purpose

Discover MTU and route to destination (Linux).

Example

```
$ tracepath google.com
$ tracepath 8.8.8.8
```

What it shows

- Route path
- MTU along the path
- Where packet size causes drop

4. IP (IPROUTE2)

Purpose

Inspect and manage IP addresses, routes, links.

Example

```
$ ip addr show
$ ip route show
$ ip link show
$ ip neigh show
```

What it shows

- IP addresses
- Routing table
- Network interfaces
- Neighbor table

5. SS (SOCKET STATISTICS)

Purpose

Show active connections and listening ports.

Example

```
$ ss -tulpen
$ ss -tp state established
$ ss -ltn
```

What it shows

- Open connections
- Listening ports
- Process using sockets
- Connection states

6. NETSTAT (LEGACY)

Purpose

Show network connections, routing, interfaces (legacy).

Example

```
$ netstat -tulpen
$ netstat -rn
$ netstat -i
```

What it shows

- Active connections
- Routing table
- Interface statistics

Note: Use ss instead. netstat is deprecated.

7. ARP

Purpose

Display or manipulate ARP (IPv4 neighbor) table.

Example

```
$ arp -a
$ ip neigh show
```

What it shows

- IP → MAC mappings
- Reachability of neighbors
- Stale or incomplete entries
- Layer 2 issues

8. DIG (DNS LOOKUP)

Purpose

Powerful DNS query tool.

Example

```
$ dig google.com
$ dig @8.8.8.8 veriqta.com A
$ dig MX veriqta.com
```

What it shows

- DNS records (A, MX, TXT, etc.)
- Authoritative answers
- TTL and response details
- DNS server behavior

9. NSLOOKUP

Purpose

Query DNS servers (simple lookup tool).

Example

```
$ nslookup google.com
$ nslookup -type=mx veriqta.com
```

What it shows

- DNS resolution
- Mail servers
- Nameserver info
- Basic troubleshooting

10. HOST

Purpose

Simple DNS lookup utility.

Example

```
$ host google.com
$ host -t MX veriqta.com
$ host 8.8.8.8
```

What it shows

- DNS records
- IP addresses
- Mail servers
- Reverse lookups

NETWORK TROUBLESHOOTING WORKFLOW

1. Define the Problem — What is not working?
2. Check Connectivity — ping, traceroute, tracepath
3. Check Configuration — ip addr, ip route, ss, arp
4. Check Name Resolution — dig, nslookup, host
5. Check Firewall / Security — iptables/nftables, firewalld
6. Analyze & Fix — Apply fix, verify and monitor

QUICK REFERENCE TABLE

COMMAND	LAYER	USE CASE
ping	L3 / L4	Test connectivity & latency
traceroute	L3	Trace path & identify hop issues
tracepath	L3	Trace path & discover MTU
ip	L3 / L2	Show/modify IP, routes, links
ss	L4	Show sockets and connections
netstat	L3 / L4	Legacy connection & routing info
arp	L2	Show IP ↔ MAC mappings
dig	L7 (DNS)	Advanced DNS queries
nslookup	L7 (DNS)	Simple DNS lookups
host	L7 (DNS)	DNS lookup utility

PRO TIPS

- ☑ Always reproduce the issue.
- ☑ Test from multiple points.
- ☑ Check DNS before blaming the server.
- ☑ Capture evidence with timestamps.
- ☑ Layer by layer: L1 → L7.
- ☑ Document everything you find.

REMEMBER

Good diagnostics save hours.
Bad assumptions cause outages.
Use the right command at the right layer.

VERIQTA

19. PACKET CAPTURE

"When logs are not enough, capture the packets. Packets never lie."

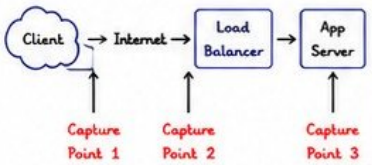
PRODUCTION NOTE

- [x] Capture only what you need.
- [x] Know where to capture.
- [x] Reduce noise with filters.
- [x] Protect sensitive data.
- [x] Remove captures after use.

1. PACKET CAPTURE OVERVIEW

- Capture network packets for analysis.
- Helps troubleshoot connectivity, latency, security, and protocol issues.
- Tools: tcpdump (CLI), Wireshark (GUI).
- Understand before analyzing.

WHERE TO CAPTURE?



Capture at multiple points to isolate the issue.

2. TCPDUMP ESSENTIALS

- Powerful CLI packet capture tool.
- Lightweight and available everywhere.
- Syntax: tcpdump [options] [filter]

```
$ tcpdump -i eth0 -n -vv
$ tcpdump -i eth0 host 10.0.0.10
$ tcpdump -i any port 80
$ tcpdump -i eth0 -w capture.pcap
$ tcpdump -r capture.pcap
$ tcpdump -c 100 -i eth0 port 443
```

COMMON OPTIONS

-i <iface>	Select interface
-n	Don't resolve hostnames
-vv	Verbose output
-w <file>	Write to file (pcap)
-r <file>	Read from file
-c <num>	Capture <num> packets

3. WIRESHARK BASICS

- GUI analyzer for captured packets.
- Powerful filtering and decoding.
- Open pcap file or capture live.

Basic Workflow

1. Start capture (or open capture file)
2. Apply display filters
3. Inspect packets in detail
4. Follow streams / conversations
5. Export or document findings

DISPLAY FILTER EXAMPLES

```
ip.addr == 10.0.0.10          # Filter by IP
tcp.port == 443               # Filter by port
http                          # HTTP traffic
dns                           # DNS traffic
tcp.flags.syn == 1 and tcp.flags.ack == 0
                              # SYN packets
```

4. CAPTURE FILTERS (tcpdump)

- Applied at capture time (reduces data volume).
- BPF (Berkeley Packet Filter) syntax.

EXAMPLES

FILTER	DESCRIPTION
host 10.0.0.10	Traffic to/from a host
net 10.0.0.0/24	Traffic to/from a network
port 80	Traffic on port 80
tcp	Only TCP packets
udp	Only UDP packets
port 22 and host 10.0.0.10	SSH traffic to host
not port 53	Exclude DNS traffic
tcp[tcpflags] & (tcp-syn) != 0	Only SYN packets

TIP

Use capture filters to capture less.
Use display filters to analyze smart.

5. PACKET ANALYSIS – WHAT TO LOOK FOR

AREA	WHAT TO CHECK
CONNECTIONS	Handshakes (SYN, SYN-ACK, ACK) Connection resets (RST) Unexpected connections
LATENCY	Retransmissions High response times Out-of-order packets
PROTOCOLS	HTTP status codes TLS handshake failures DNS query/response
SECURITY	Unusual ports or IPs Failed logins / brute force Data exfiltration patterns
ERRORS	ICMP errors (unreachable, TTL exceeded) Malformed packets Checksum errors

6. FOLLOWING STREAMS (WIRESHARK)

- Reconstruct full conversations.
- TCP Stream → Follow TCP Stream
- HTTP → Follow HTTP Stream
- DNS → Right click → Follow → DNS Stream

EXAMPLE: FOLLOW HTTP STREAM

```
GET /api/data HTTP/1.1
Host: app.veriqta.local
User-Agent: curl/8.5.0
Accept: */*

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 52

{"status":"ok","data":{"id":123,"value":"success"}}
```

7. PRODUCTION TROUBLESHOOTING EXAMPLES

SCENARIO	WHAT TO CAPTURE	WHAT TO CHECK
APP NOT LOADING	Client → LB → App	TCP handshake, RST, HTTP errors, timeouts
SLOW RESPONSE	Between tiers	Retransmissions, latency, queueing
DNS ISSUES	Client & Server	DNS queries, responses, NXDOMAIN, timeouts
TLS ERRORS	Client ↔ Server	TLS handshake, certs, alerts, protocol
CONNECTION RESET	Both sides	RST packets, who sent it, interval
PORT BLOCKED	From source	No SYN-ACK, ICMP errors, firewall drops

8. SAVE, COMPRESS & SHARE

```
# Save capture
$ tcpdump -i eth0 -w capture.pcap
# Compress capture
$ gzip capture.pcap
# Share securely
$ scp capture.pcap.gz admin@10.0.0.5:/tmp/
# Open in Wireshark
$ wireshark capture.pcap
```

NOTE

Always review captures for sensitive data (passwords, tokens, PII) before sharing.

9. QUICK REFERENCE CHEAT SHEET

TASK	TCPDUMP COMMAND	WIRESHARK FILTER
Capture all on eth0	tcpdump -i eth0	(no filter)
Capture to file	tcpdump -i eth0 -w out.pcap	(open file)
Capture host	tcpdump host 10.0.0.10	ip.addr == 10.0.0.10
Capture port 443	tcpdump port 443	tcp.port == 443
Capture HTTP	tcpdump port 80	http
Capture DNS	tcpdump port 53	dns
Capture TCP only	tcpdump tcp	tcp
Capture UDP only	tcpdump udp	udp

10. KEY TAKEAWAYS

- [x] Capture at the right point.
- [x] Use filters to reduce noise.
- [x] Understand the protocol behavior.
- [x] Analyze methodically.
- [x] Document everything you find.
- [x] Remove sensitive data and clean up.

> "Packets are the ground truth. Learn to read them."

VERIQTA

20. LINUX NETWORKING BEST PRACTICES

"Secure it. Monitor it. Log it. Fix it fast."
That's how production stays stable."

PRODUCTION NOTE

- ☑ Design for security.
- ☑ Monitor everything.
- ☑ Log meaningfully.
- ☑ Test regularly.
- ☑ Document your network.

1. NETWORK HARDENING

- Allow only what is required.
- Deny everything else by default.
- Use least privilege for network access.
- Keep systems and firmware updated.
- Disable unused services and ports.
- Use TLS for all sensitive traffic.
- Segment networks (VLANs/Subnets).
- Protect against spoofing and attacks.

HARDENING CHECKLIST

- ☑ Unnecessary ports closed
- ☑ Firewall rules reviewed
- ☑ Services minimized
- ☑ Strong authentication in place
- ☑ TLS enforced
- ☑ Network segmentation applied
- ☑ Intrusion detection enabled
- ☑ Regular security updates

2. NETWORK MONITORING

- Monitor availability, performance and errors.
- Watch bandwidth, latency and packet loss.
- Alert early, before users complain.
- Use both host and network level metrics.

WHAT TO MONITOR

- Availability (Up/Down)
- Latency (ms)
- Bandwidth (In/Out)
- Packet Loss
- Errors & Drops
- Connection States

USEFUL TOOLS

- Prometheus + Grafana
- Netdata
- Zabbix / Nagios
- nload, iftop, bmon
- ss, ip -s link, sar -n DEV

3. NETWORK LOGGING

- Logs are your evidence.
- Log enough to debug, not too much.
- Centralize logs for easy search.
- Time sync all systems (NTP).
- Protect logs from tampering.

WHAT TO LOG

- Firewall / nftables / iptables logs
- Connection attempts (accept/deny)
- DHCP events
- DNS queries and failures
- VPN / SSH access
- Interface up/down events
- Routing changes
- Authentication failures

LOG DESTINATION OPTIONS

- rsyslog → Remote syslog server
- journald → systemd journal
- Loki / Elasticsearch / OpenSearch
- Cloud logging (AWS CloudWatch, etc.)

4. TROUBLESHOOTING WORKFLOW

1 IDENTIFY	2 VERIFY BASIC CONNECTIVITY	3 CHECK NAME RESOLUTION	4 CHECK ROUTING & PATH	5 CHECK FIREWALL & FILTERS	6 CHECK SERVICE & APP	7 DEEP DIVE CAPTURE
What is the problem?	Can the host communicate?	Is DNS working correctly?	Is the route correct?	Is traffic allowed?	Is the service healthy?	Still not sure? Capture packets.
Collect symptoms from users, alerts, or logs.	ping, traceroute, ss, ip addr	dig, nslookup, host	ip route, traceroute, tracepath	nft list ruleset, iptables -L, firewall-cmd --list-all	ss, systemctl status, logs, error codes	tcpdump, Wireshark, Analyze packets.

☆ Always isolate the layer causing the issue. Fix one layer at a time.

5. GENERAL BEST PRACTICES

- Follow the principle of least privilege.
- Keep systems and tools up to date.
- Automate network configuration with IaC.
- Backup network configurations.
- Test failover and recovery regularly.
- Document network diagrams and IP plans.
- Use Infrastructure as Code for repeatability.
- Validate changes in a staging environment.

6. COMMON PITFALLS TO AVOID

- ✗ Opening too many ports "just in case".
- ✗ No logging or incomplete logging.
- ✗ Not monitoring latency and packet loss.
- ✗ Ignoring DNS failures.
- ✗ Wrong routing or missing default route.
- ✗ No documentation of network changes.
- ✗ Not testing firewall rules before applying.
- ✗ Overlooking time sync (NTP).

7. QUICK REFERENCE COMMANDS

TASK	COMMAND	DESCRIPTION
Show IP addresses	ip addr	List all IP addresses
Show routes	ip route	Display routing table
Test connectivity	ping -c 4 8.8.8.8	Ping a host
Trace route	traceroute 8.8.8.8	Trace path to host
Show sockets	ss -tulpen	Show listening ports
DNS lookup	dig example.com	DNS query
Capture packets	tcpdump -i eth0 -n	Capture on interface
Show ARP table	ip neigh show	ARP / neighbor table

8. NETWORK HEALTH BASELINE

A HEALTHY NETWORK SHOULD HAVE

- ☑ Low latency
- ☑ Low packet loss
- ☑ Stable connections
- ☑ Secure firewall rules
- ☑ Healthy DNS
- ☑ Synced time (NTP)
- ☑ Centralized logging
- ☑ Active monitoring
- ☑ Documented configurations

FINAL TAKEAWAYS

- ☑ A secure network is a reliable network.
- ☑ Monitor continuously and alert early.
- ☑ Logs tell the story when things go wrong.
- ☑ A systematic workflow saves time and reduces outages.
- ☑ Good engineers prevent issues; great engineers prepare to fix them fast.

VERIQTA

21. SWITCHING CONCEPTS

"Switching is how devices talk on the same network before the traffic reaches the router."

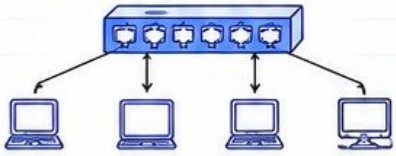
PRODUCTION NOTE

- ☑ Understand Layer 2 before Layer 3.
- ☑ Keep broadcast domains small.
- ☑ Use VLANs to segment.
- ☑ Monitor MAC tables and errors.
- ☑ Document your network.

1. WHAT IS A SWITCH?

- A switch connects devices in the same network.
- Operates at OSI Layer 2 (Data Link).
- Uses MAC addresses to forward frames.
- Each port is a separate collision domain.

SWITCH CONNECTING DEVICES



Devices can communicate efficiently without involving the router.

2. OSI MODEL RECAP

7	Application	(Protocols)
6	Presentation	
5	Session	
4	Transport	(TCP/UDP)
3	Network	(IP)
2	Data Link	(MAC)
1	Physical	(Bits)

Switches work at Layer 2 using MAC addresses.

3. MAC ADDRESS & FRAME FORWARDING

- Switch learns MAC addresses on each port.
- Builds a MAC address table (CAM table).
- Forwards frames only to the correct port.
- Unknown destination? Floods the frame.

FRAME FORWARDING PROCESS

Frame Arrives → Learn Source MAC → Check MAC Table → Forward or Flood

Port 1 | MAC Table Lookup | Port 2 (or Flood)

EXAMPLE MAC TABLE

MAC ADDRESS	PORT	TYPE	AGE
00:11:22:33:44:55	1	Dynamic	30s
00:11:22:33:44:66	2	Dynamic	15s
00:11:22:33:44:77	3	Dynamic	45s

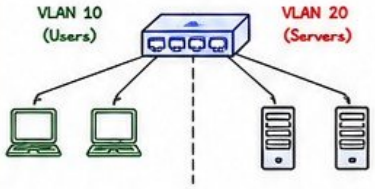
4. KEY SWITCHING TERMINOLOGY

TERM	DESCRIPTION
MAC Address	Unique hardware address (48-bit).
CAM Table	Content Addressable Memory table stores MAC to port mappings.
VLAN	Virtual LAN, segments Layer 2 domains.
Access Port	Connects end devices (single VLAN).
Trunk Port	Carries multiple VLANs (tagged).
STP (Spanning Tree)	Prevents Layer 2 loops.
Broadcast Domain	Area where broadcast traffic is received by all devices.
Unicast / Multicast / Broadcast	Types of traffic in a network.

5. VLANs (VIRTUAL LANs)

- VLANs logically segment the network.
- Devices in the same VLAN can communicate.
- VLANs reduce broadcast traffic.
- Helps security, performance and management.

VLAN EXAMPLE

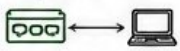


Different VLANs = Different broadcast domains

6. ACCESS PORT vs TRUNK PORT

ACCESS PORT

- For end devices (PCs, printers, IP Phones)
- Belongs to one VLAN
- Untagged traffic
- Example: Fa0/1



TRUNK PORT

- Connects switches/routers
- Carries multiple VLANs
- Tagged traffic (802.1Q)
- Example: Gi0/1

802.1Q TAGGING (TRUNK)

Dst MAC	Src MAC	802.1Q Tag	EtherType	Data	FCS

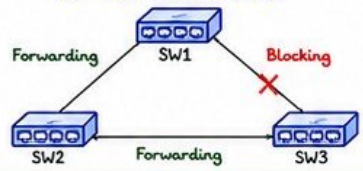
↓ 4 bytes

Tag contains: VLAN ID (12 bits) + Priority + Other fields

7. SPANNING TREE PROTOCOL (STP)

- Prevents Layer 2 loops.
- Blocks redundant links.
- Creates a loop-free topology.
- STP States: Blocking, Listening, Learning, Forwarding.

STP EXAMPLE TOPOLOGY



8. COMMON SWITCH FEATURES

Port Security	Limit MAC addresses on a port.
Storm Control	Protects against broadcast/multicast storms.
Port Mirroring (SPAN)	Copies traffic for monitoring.
QoS (Quality of Service)	Prioritize important traffic.
Energy Efficient Ethernet (EEE)	Saves power during low traffic.

9. SWITCHING BEST PRACTICES

- ☑ Use VLANs to segment networks.
- ☑ Disable unused ports.
- ☑ Enable Port Security on access ports.
- ☑ Use STP and monitor for topology changes.
- ☑ Keep firmware updated.
- ☑ Monitor MAC table, errors, and logs.
- ☑ Document your network design.
- ☑ Follow least privilege on management access.

COMMON ISSUES

- Broadcast storm → Entire network slow.
- MAC table full → Frames dropped.
- Loop in topology → Network down.
- VLAN mismatch → Devices cannot communicate.

QUICK CHECKLIST

- ☑ Is VLAN design correct?
- ☑ Are trunk ports tagging properly?
- ☑ Is STP blocking redundant paths?
- ☑ Are MAC tables stable?
- ☑ Are logs clean (no errors)?

KEY TAKEAWAY

Switching is the foundation of every network.
Master Layer 2, and Layer 3 becomes easy.
Design smart, segment well, and monitor always.

VERIQTA

22. VLANS

"VLANs segment at Layer 2, improving security, performance, and manageability."

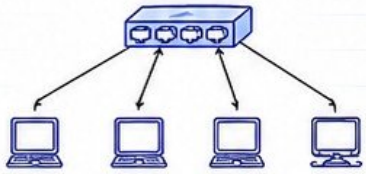
PRODUCTION NOTE

- ☑ Plan VLANs before deployment.
- ☑ Keep VLANs simple.
- ☑ Document VLAN ID, name & purpose.
- ☑ Secure trunk links.
- ☑ Monitor VLAN performance.

1. WHAT IS A VLAN?

- A VLAN (Virtual LAN) creates a logical network on top of a physical network.
- Devices in the same VLAN behave as if they are on the same switch.
- VLANs segment broadcast domains.
- Improves security, performance, and network management.

WITHOUT VLANs (ONE BROADCAST DOMAIN)

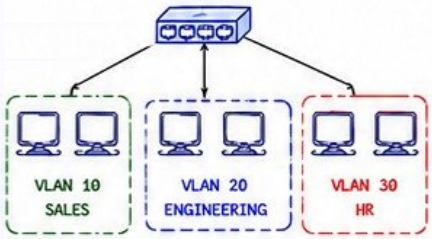


All devices share the same broadcast domain.

2. VLAN BENEFITS

- ☑ Reduces broadcast traffic.
- ☑ Improves security between departments.
- ☑ Better network organization.
- ☑ Easier moves, adds, and changes.
- ☑ Supports policy and access control.

WITH VLANs (MULTIPLE BROADCAST DOMAINS)



Each VLAN is a separate broadcast domain.

3. VLAN TYPES

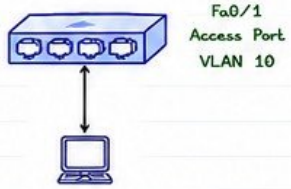
- Default VLAN: VLAN 1 (exists on all switches).
- Data VLAN: Used for user/device traffic.
- Management VLAN: Used for switch management.
- Native VLAN: Used on trunks for untagged frames.
- Voice VLAN: Used for IP phones (QoS).
- Unused VLANs: Keep to a minimum.

EXAMPLE VLAN PLAN

VLAN ID	NAME	PURPOSE	SUBNET
10	SALES	Sales Department	10.10.10.0/24
20	ENGINEERING	Engineering Dept	10.20.20.0/24
30	HR	Human Resources	10.30.30.0/24
99	MGMT	Network Management	10.99.99.0/24
999	NATIVE	Native VLAN	(Trunk Only)

4. ACCESS PORT (UNTAGGED)

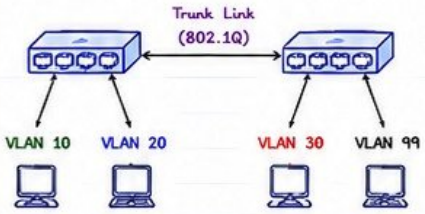
- Used for end devices (PCs, Printers, IP Phones).
- Carries traffic for one VLAN.
- Frames are untagged.



Traffic belongs to VLAN 10 (untagged).

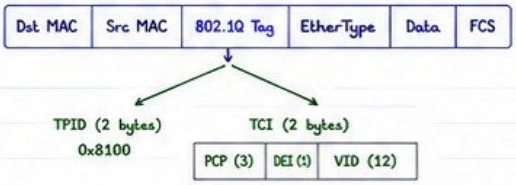
5. TRUNK PORT (802.1Q TAGGED)

- Used between switches/routers.
- Carries traffic for multiple VLANs.
- Frames are tagged with VLAN ID.



Trunk carries multiple VLANs using tags.

6. 802.1Q TAGGING FORMAT



PCP = Priority (QoS)
DEI = Drop Eligible Indicator
VID = VLAN ID (1-4094)

7. VLAN CONFIGURATION EXAMPLE (CISCO IOS)

```
! Create VLANs
Switch(config)# vlan 10                          ! Enter VLAN ID 10
Switch(config-vlan)# name SALES                  ! Name the VLAN
Switch(config)# vlan 20                          ! Enter VLAN ID 20
Switch(config-vlan)# name ENGINEERING            ! Name the VLAN
Switch(config)# vlan 30                          ! Enter VLAN ID 30
Switch(config-vlan)# name HR                     ! Name the VLAN
Switch(config)# vlan 99                          ! Management VLAN
Switch(config-vlan)# name MGMT
! Assign Access Port to VLAN 10
Switch(config)# interface fa0/1                  ! Access port config
Switch(config-if)# switchport mode access        ! Set mode to access
Switch(config-if)# switchport access vlan 10     ! Assign to VLAN 10
Switch(config-if)# spanning-tree portfast        ! PortFast for end devices
Switch(config-if)# spanning-tree bpduguard enable! Protect against rogue switches

! Configure Trunk Port
Switch(config)# interface gi0/1                  ! Trunk port config
Switch(config-if)# switchport mode trunk         ! Set mode to trunk
Switch(config-if)# switchport trunk allowed      ! Allow these VLANs
                    vlan 10,20,30,99
Switch(config-if)# switchport trunk native       ! Set Native VLAN
                    vlan 999
```

8. NATIVE VLAN

- Used for untagged frames on trunk links.
- Native VLAN must match on both ends.
- Best practice: Use a dedicated VLAN (e.g., 999).

BEST PRACTICE

- ☑ Never use VLAN 1 for native.
- ☑ Use a dedicated native VLAN.
- ☑ Allow only necessary VLANs on trunks.
- ☑ Disable unused VLANs.
- ☑ Document all VLANs.

COMMON MISTAKES

- ✗ Leaving VLAN 1 as native.
- ✗ Allowing all VLANs on trunks.
- ✗ Creating too many VLANs.
- ✗ Not documenting VLANs.
- ✗ Not pruning unused VLANs.

9. VLAN TROUBLESHOOTING

- Verify VLAN exists on all switches.
- Check access port VLAN assignment.
- Check trunk status and allowed VLANs.
- Verify native VLAN consistency.
- Use these commands:

```
$ show vlan brief
$ show interfaces trunk
$ show vlan id 10
$ show mac address-table vlan 10
$ show interfaces switchport
```

10. VLAN ID RANGE

- Valid VLAN IDs: 1 - 4094
- VLAN 1: Default VLAN (use carefully)
- 4095: Reserved

Design VLANs with purpose. Secure them with policy. Monitor them always.

VERIQTA

23. NETWORK SEGMENTATION

"Segment your network. Limit risk."
Security, performance, and control start with segmentation."

PRODUCTION NOTE

- ☑ Segment by purpose, not by default.
- ☑ Apply least privilege between segments.
- ☑ Control traffic with firewalls.
- ☑ Monitor and log between segments.
- ☑ Review segmentation regularly.

1. WHAT IS NETWORK SEGMENTATION?

- Divides a network into smaller isolated segments.
- Reduces attack surface.
- Improves security and compliance.
- Limits broadcast domains.
- Improves performance and manageability.

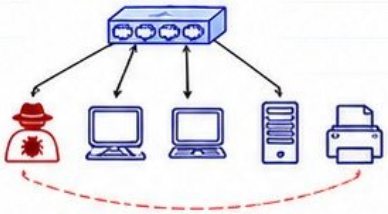
2. WHY SEGMENT?

- ☑ Contain threats and breaches.
- ☑ Enforce security policies.
- ☑ Meet compliance requirements.
- ☑ Protect sensitive data.
- ☑ Optimize performance.
- ☑ Simplify troubleshooting.

3. COMMON SEGMENTATION AREAS

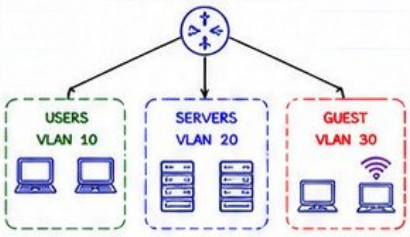
SEGMENT	PURPOSE	EXAMPLE
User Network	End-user devices	VLAN 10 10.10.10.0/24
Server Network	Application & data	VLAN 20 10.20.20.0/24
Management Network	Network devices	VLAN 30 10.30.30.0/24
Guest Network	Internet-only access	VLAN 40 10.40.40.0/24
Database Network	Database servers	VLAN 50 10.50.50.0/24
DMZ	Public-facing services	VLAN 60 10.60.60.0/24

NO SEGMENTATION (FLAT NETWORK)



If one device is compromised, the entire network is at risk.

WITH SEGMENTATION



Each segment is isolated.
Traffic is controlled between segments.

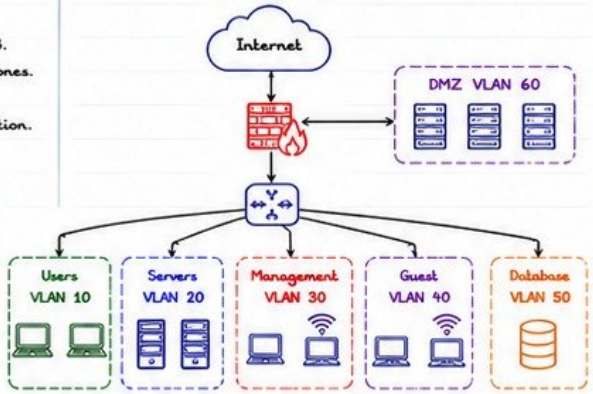
4. SEGMENTATION METHODS

- VLANs – Logical segmentation at Layer 2.
- Subnets – IP-level segmentation at Layer 3.
- Firewall Zones – Control traffic between zones.
- VRFs – Virtual routing and forwarding.
- Micro-segmentation – Workload-level isolation.

SEGMENTATION LAYERS

4	Application Micro-segmentation
3	Network Subnets, ACLs, Routing
2	Data Link VLANs, Switch Port Isolation
1	Physical Separate hardware, cabling

5. SEGMENTATION ARCHITECTURE EXAMPLE



Traffic between segments is controlled by firewalls, ACLs, and routing policies.

6. TRAFFIC CONTROL BETWEEN SEGMENTS

- Use firewalls or ACLs to control traffic.
- Allow only required ports and protocols.
- Apply least privilege.
- Log and monitor all inter-segment traffic.

EXAMPLE POLICY

Users (VLAN 10) → Servers (VLAN 20) Allow: TCP 80, 443	✓
Users (VLAN 10) → Database (VLAN 50) Deny	✗
Guest (VLAN 40) → Any Internal Deny	✗
Management (VLAN 30) → All Allow (Admin Only)	✓

TIP
Deny by default.
Allow only what is necessary.
Review rules regularly.

7. BENEFITS OF SEGMENTATION

- ☑ Limits the spread of attacks.
- ☑ Protects sensitive resources.
- ☑ Improves network performance.
- ☑ Simplifies compliance and auditing.
- ☑ Enables better monitoring and visibility.
- ☑ Supports Zero Trust Architecture.

8. SEGMENTATION BEST PRACTICES

- Design segments based on business function.
- Keep segments small and simple.
- Document VLANs, subnets, and policies.
- Use descriptive names and IP ranges.
- Test and validate all inter-segment rules.
- Review and update segmentation regularly.
- Automate where possible (IaC).

9. COMMON PITFALLS

- ✗ Too many VLANs without purpose.
- ✗ Allowing any-any traffic between segments.
- ✗ No documentation of segmentation.
- ✗ Ignoring management network security.
- ✗ Not monitoring inter-segment traffic.
- ✗ Flat networks in production.

QUICK CHECKLIST

- ☐ Segments defined?
- ☐ VLANs documented?
- ☐ IP ranges planned?
- ☐ Policies applied?
- ☐ Logging enabled?
- ☐ Regular reviews done?

10. KEY TAKEAWAY

Segment your network by function and risk.
Control traffic between segments.
A well-segmented network is a secure, fast, and manageable network.

USEFUL COMMANDS (LINUX)

```
# Show VLAN interfaces
ip -d link show
# Show routing table
ip route
# Show firewall rules (nftables)
sudo nft list ruleset
# Test connectivity between segments
ping 10.20.20.10
# Trace route
traceroute 10.50.50.10
```

VERIQTA

24. LOAD BALANCERS

"Distribute traffic. Improve availability.
Scale applications without sacrificing reliability."

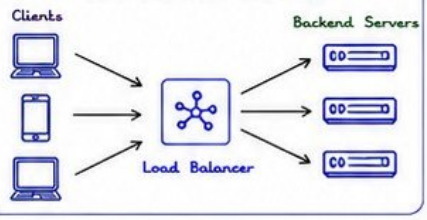
PRODUCTION NOTE

- ☑ Place LB in front of your apps.
- ☑ Health checks are critical.
- ☑ Use least privilege and TLS.
- ☑ Monitor connections and latency.
- ☑ Design for High Availability.

1. WHAT IS A LOAD BALANCER?

- A load balancer distributes incoming traffic across multiple servers.
- Improves availability, reliability and scalability.
- Prevents any single server from becoming a bottleneck.
- Can operate at Layer 4 or Layer 7.

LOAD BALANCER OVERVIEW



2. LAYER 4 vs LAYER 7

LAYER 4 (TRANSPORT)

- Operates at TCP/UDP level.
- Uses IP + Port to make decisions.
- Faster performance.
- Cannot inspect application data.
- Use cases: TCP services, databases, gaming, VPN, VoIP.

Examples: AWS NLB, HAProxy (TCP mode)

LAYER 7 (APPLICATION)

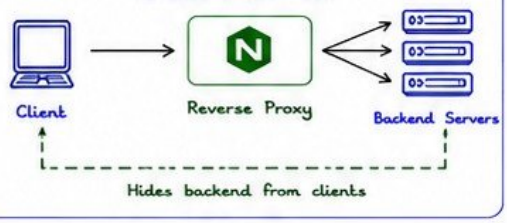
- Operates at HTTP/HTTPS level.
- Uses content (URL, Host, Header, Cookie) for routing.
- Supports SSL termination.
- Advanced routing and rewrites.
- Use cases: Web apps, APIs, Microservices.

Examples: AWS ALB, NGINX, HAProxy (HTTP mode)

3. REVERSE PROXIES

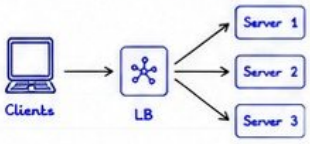
- Sit in front of backend servers and forward client requests.
- Add security, caching, compression, TLS termination, WAF.
- Common reverse proxies: NGINX, HAProxy, Envoy, Traefik.

REVERSE PROXY FLOW



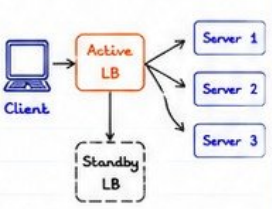
4. LOAD BALANCER ARCHITECTURES

- SINGLE INSTANCE (NOT RECOMMENDED)



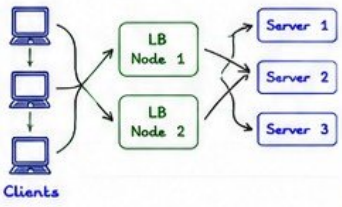
Single point of failure.

- ACTIVE / STANDBY



Failover on LB failure.

- ACTIVE / ACTIVE (HIGH AVAILABILITY)



High availability and scalability.

5. HAProxy OVERVIEW

HAProxy

- High performance TCP/HTTP load balancer.
- Supports Layer 4 and Layer 7.
- Health checks, SSL termination, routing, ACLs, rate limiting.
- Used in cloud, on-prem and containers.

Simple HAProxy Config Example (HTTP)

```
frontend http_in
    bind *:80
    default_backend app_servers

backend app_servers
    balance roundrobin
    option httpchk GET /health
    server app1 10.0.0.11:8080 check
    server app2 10.0.0.12:8080 check
    server app3 10.0.0.13:8080 check
```

6. NGINX OVERVIEW

NGINX

- High performance web server and reverse proxy.
- Excellent for HTTP/HTTPS (Layer 7).
- Supports caching, compression, rewrites,
- static content, WebSocket.
- Easy to configure and reload.

Simple NGINX Reverse Proxy Example

```
server {
  listen 80;
  server_name example.com;

  location / {
    proxy_pass http://app_backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For
                     $proxy_add_x_forwarded_for;
  }
}

upstream app_backend {
  server 10.0.0.11:8080;
  server 10.0.0.12:8080;
  server 10.0.0.13:8080;
}
```

7. LOAD BALANCER FEATURES

Health Checks	Ensure only healthy servers receive traffic.
SSL/TLS Termination	Offload SSL from backend servers.
Session Persistence (Sticky Sessions)	Route requests from the same client to the same server.
Path / Host Based Routing (L7)	Route based on URL path or hostname.
Rate Limiting	Protect backend from abuse.
Logging & Monitoring	Visibility into traffic, errors, latency.

8. LAYER 4 vs LAYER 7 COMPARISON

FEATURE	LAYER 4 (L4)	LAYER 7 (L7)
OSI Layer	Transport	Application
Protocol	TCP / UDP	HTTP / HTTPS
Routing Basis	IP + Port	Content (URL, Host, Header)
Performance	Higher (faster)	Lower (more processing)
SSL Termination	✗	✓
Use Application Data	✗	✓
Example Use Cases	Databases, VPN, Gaming, TCP services	Web Apps, APIs, Microservices

9. BEST PRACTICES

- ☑ Always use health checks.
- ☑ Use least privilege and secure your LBs.
- ☑ Enable TLS everywhere.
- ☑ Distribute LBs across Availability Zones.
- ☑ Monitor latency, errors, and connection counts.
- ☑ Implement rate limiting and WAF for L7.
- ☑ Keep configurations versioned and tested.

10. COMMON TROUBLESHOOTING

- 502 / 503 errors → Backend down or unhealthy.
- High latency → Check backend or network.
- No response → Security group / firewall.
- TLS errors → Certificate or protocol mismatch.
- Uneven traffic → Check health checks & persistence.
- Connection reset → Timeout or idle settings.

11. QUICK COMMANDS

Test LB (HTTP):	`curl -I http://example.com`
Test LB (HTTPS):	`curl -kI https://example.com`
Check open ports:	`ss -tulnp`
HAProxy stats (if enabled):	`curl http://lb:8404/stats`
NGINX status:	`curl http://lb/nginx_status`
Check backend health:	`curl http://backend:port/health`

VERIQTA

25. HIGH AVAILABILITY NETWORKING

"Design for failure. Build for redundancy.
One failure must not take down your network."

PRODUCTION NOTE

- ☑ Eliminate single points of failure.
- ☑ Use redundancy at every layer.
- ☑ Test failover regularly.
- ☑ Monitor HA components.
- ☑ Document recovery procedures.

1. WHAT IS HIGH AVAILABILITY NETWORKING?

- Ensures the network remains available even when a component fails.
- Built using redundancy, failover mechanisms, and automatic recovery.
- Goals: Minimize downtime, maintain connectivity, and ensure consistency.

KEY HA PRINCIPLES

- ☑ Redundancy – No single point of failure
- ☑ Detect – Monitor failures quickly
- ☑ Failover – Automatic transition
- ☑ Recover – Restore failed component
- ☑ Test – Verify HA works as expected

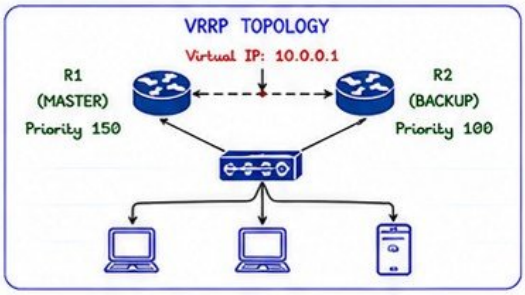
2. REDUNDANCY MODELS

MODEL	DESCRIPTION	EXAMPLE
Device Redundancy	Duplicate devices handle traffic	Routers, Switches, Firewalls
Link Redundancy	Multiple links between devices	LACP, ECMP
Path Redundancy	Multiple network paths to destination	Dual ISPs, Multiple Gateways
Service Redundancy	Duplicate services running	DNS, DHCP, VPN
Data Redundancy	Replicate or backup critical data	Databases, Config Backups

Redundancy is the foundation of HA.

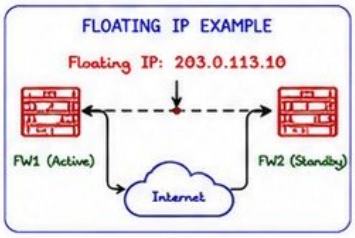
3. VRRP (VIRTUAL ROUTER REDUNDANCY PROTOCOL)

- Vendors: Standard (RFC 5798)
- Used to provide default gateway redundancy.
- Routers share a Virtual IP (VIP) and a MAC.
- One router is MASTER, others are BACKUP.
- If MASTER fails, BACKUP takes over VIP.

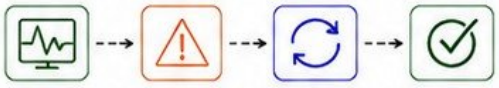


4. FLOATING IPs

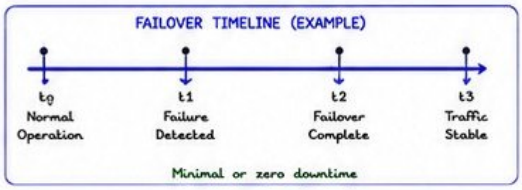
- A Floating IP is an IP address not bound to a single interface permanently.
- It "floats" between devices based on who is active.
- Common in firewalls, routers, and cloud environments.



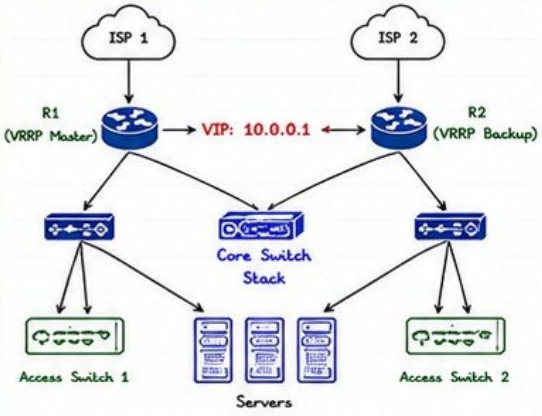
5. FAILOVER PROCESS



1. Monitor – Track link/device health continuously.
2. Detect – Identify failure or degradation.
3. Failover – Backup device takes over IP and routes.
4. Recover – Restore failed component and rejoin cluster.



6. HA NETWORK ARCHITECTURE EXAMPLE



7. VRRP CONFIGURATION EXAMPLE (LINUX)

```
# Install keepalived
sudo apt install keepalived

# /etc/keepalived/keepalived.conf
vrrp_instance VI_1 {
  state MASTER                # MASTER or BACKUP
  interface eth0              # Interface connected to LAN
  virtual_router_id 51        # Unique ID (same on all routers)
  priority 150                # Higher = preferred master
  advert_int 1                # Advertisement interval (seconds)
  authentication {
      auth_type PASS
      auth_pass StrongPass123
  }
  virtual_ipaddress {
      10.0.0.1/24 dev eth0    # Virtual IP (VIP)
  }
}
```

8. DESIGN BEST PRACTICES

- ☑ Use redundant devices and links.
- ☑ Place HA devices in separate racks/PDUs.
- ☑ Use VRRP/HSRP/GLBP for gateway HA.
- ☑ Use link aggregation (LACP) on switches.
- ☑ Continuously monitor health and latency.
- ☑ Test failover scenarios regularly.
- ☑ Automate configuration backups.
- ☑ Document HA design and procedures.

9. COMMON FAILURE SCENARIOS

SCENARIO	HA SOLUTION
Router Failure	VRRP failover to backup router
Link Failure	Use LACP, ECMP, or backup links
ISP Outage	Dual ISP with automatic failover
Switch Failure	Stacking / MLAG / Redundant uplinks
Firewall Failure	Active/Standby with floating IP
Power Failure	Dual power supplies + UPS/Generator

10. HA CHECKLIST

- ☐ All critical devices have a backup.
- ☐ VRRP/HSRP configured and tested.
- ☐ Floating IPs tested for failover.
- ☐ Link redundancy configured.
- ☐ Monitoring and alerting enabled.
- ☐ Backups of configs up to date.
- ☐ Failover runbook documented.
- ☐ Regular HA drills performed.

11. QUICK COMMANDS

```
# Check VRRP status
ip addr show | grep 10.0.0.1

# View keepalived status
systemctl status keepalived

# Show VRRP details
ip -d addr show dev eth0

# Check connectivity
ping 10.0.0.1

# Monitor VRRP advertisements
tcpdump -ni eth0 proto 112
```

12. KEY TAKEAWAY

High availability is not a feature,
it is a design mindset.

Redundancy + Monitoring + Tested Failover
=
A resilient, always-on network.

VERIQTA

26. NETWORK ADDRESS TRANSLATION

"NAT allows private networks to communicate with the world securely and efficiently."

PRODUCTION NOTE

- ☑ Understand SNAT, DNAT, and PAT.
- ☑ Use NAT deliberately, not by default.
- ☑ Limit exposure with DNAT rules.
- ☑ Monitor NAT table and connections.
- ☑ Document NAT architecture.

1. WHAT IS NAT?

- NAT (Network Address Translation) modifies IP addresses in packet headers.
- Enables private networks to access public networks.
- Hides internal IPs and conserves public IPs.
- Types: SNAT, DNAT, PAT (NAT Overload).

NAT IN ACTION

Private IPs are translated to public IPs when going out, and back when coming in.

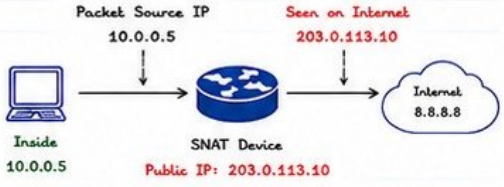
2. NAT TYPES OVERVIEW

TYPE	FULL NAME	DIRECTION	PURPOSE
SNAT	Source NAT	Inside → Outside	Outbound Internet access
DNAT	Destination NAT	Outside → Inside	Expose internal services
PAT	Port Address Translation (NAT Overload)	Inside → Outside	Many-to-one using ports

NAT changes IP addresses, but keeps connections intact using state tracking.

3. SNAT (SOURCE NAT)

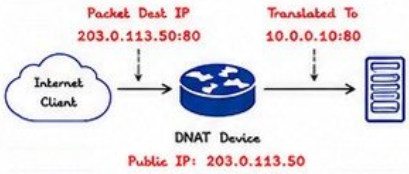
- Changes the source IP address of outgoing packets.
- Used for outbound access from private networks.
- Hides internal IPs behind a public IP.



Use Case: Users accessing websites, API calls, updates, cloud services.

4. DNAT (DESTINATION NAT)

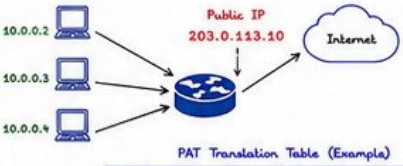
- Changes the destination IP address of incoming packets.
- Used to expose internal services to the internet.
- Maps public IP/port to private IP/port.



Use Case: Hosting websites, APIs, or applications for public access.

5. PAT (NAT OVERLOAD)

- Many private IPs share one public IP using different ports.
- Most common form of NAT.
- Conserves public IP addresses.



PAT Translation Table (Example)

Inside Local IP:Port	Inside Global IP:Port
10.0.0.2:51515	203.0.113.10:40001
10.0.0.3:51515	203.0.113.10:40002
10.0.0.4:51515	203.0.113.10:40003

Use Case: Internet access for many users/devices using a single public IP.

6. NAT TYPES COMPARISON

TYPE	DIRECTION	TRANSLATES	PURPOSE	EXAMPLE
SNAT	Inside → Outside	Source IP	Outbound access	10.0.0.5 → 203.0.113.10
DNAT	Outside → Inside	Destination IP	Expose services	203.0.113.50 → 10.0.0.10
PAT	Inside → Outside	Source IP + Port	Many to one	10.0.0.x → 203.0.113.10:4xxxx

NAT KEY NOTES

- → NAT is stateful – return traffic is automatically allowed.
- → Logs and monitor NAT tables for troubleshooting.
- → Avoid unnecessary NAT – it complicates troubleshooting.
- → Prefer private IPs internally, NAT only at boundaries.

7. LINUX IPTABLES EXAMPLES

SNAT (Masquerade)

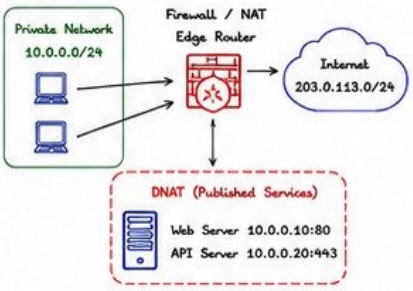
```
# Allow outbound traffic
iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

DNAT (Port Forwarding)

```
# Forward public 203.0.113.50:80 to 10.0.0.10:80
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 \
         -j DNAT --to-destination 10.0.0.10:80
iptables -A FORWARD -p tcp -d 10.0.0.10 --dport 80 -j ACCEPT
```

PAT is same as SNAT (MASQUERADE) using ports automatically.

8. PRODUCTION NAT ARCHITECTURE



9. NAT TROUBLESHOOTING CHECKLIST

- ☐ Check routing before NAT.
- ☐ Verify NAT rules order and match.
- ☐ Ensure return path is correct.
- ☐ Check connection tracking table.
- ☐ Verify firewall rules allow traffic.
- ☐ Monitor NAT table and port usage.
- ☐ Test from both inside and outside.

USEFUL COMMANDS (LINUX)

```
# View NAT table
iptables -t nat -L -n -v
# View connection tracking
conntrack -L | grep 203.0.113.10
# Watch conntrack live
conntrack -E
```

10. NAT BEST PRACTICES

- ☑ Use private IP ranges internally (RFC1918).
- ☑ Perform NAT at the network edge.
- ☑ Limit DNAT to necessary services only.
- ☑ Use specific ports for DNAT, not all.
- ☑ Enable logging for NAT events.
- ☑ Regularly review and clean NAT rules.
- ☑ Document NAT mappings and architecture.

NAT EXAMPLE SCENARIOS

SNAT Scenario: Users in 10.0.0.0/24 access the internet → Seen as 203.0.113.10

DNAT Scenario: Internet users access 203.0.113.50:443 → Reaches internal 10.0.0.20:443

PAT Scenario: 100 users browse internet using 203.0.113.10 with different source ports

KEY TAKEAWAY

NAT is essential for secure, efficient and scalable network communication.
Use the right type in the right place.

SNAT for outbound. DNAT for inbound. PAT for scale.

VERIQTA

27. VPN TECHNOLOGIES

"VPNs extend secure connectivity across untrusted networks."

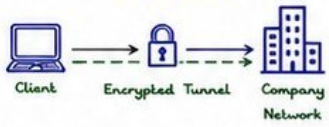
PRODUCTION NOTE

- ☑ Encrypt traffic end-to-end.
- ☑ Use strong authentication.
- ☑ Keep VPN software updated.
- ☑ Monitor VPN logs and usage.
- ☑ Document VPN designs.

1. WHAT IS A VPN?

- VPN (Virtual Private Network) creates an encrypted tunnel over the internet or untrusted networks.
- Ensures confidentiality, integrity, and authenticity.
- Enables secure remote access and site-to-site connectivity.

VPN IN ACTION



Traffic is encrypted while traveling over the internet.

2. VPN TECHNOLOGIES OVERVIEW

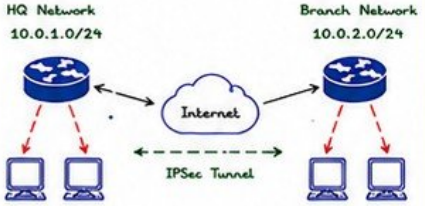
TECHNOLOGY	TYPE	LAYER	USE CASE	TRANSPORT
IPSec VPN	Site-to-Site / Remote Access	Layer 3	Enterprise networks, branch connectivity	UDP 500, 4500 (ESP)
SSL VPN	Remote Access	Layer 7	Users, web access, BYOD	TCP 443 (HTTPS)
WireGuard	Site-to-Site / Remote Access	Layer 3	High performance, modern VPN	UDP 51820

Choose the right VPN based on security, performance, ease of use, and infrastructure.

3. SITE-TO-SITE VPN OVERVIEW

- Connects entire networks.
- Common between data centers, branches, or cloud environments.
- Uses static routing or dynamic routing.

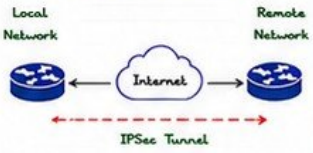
SITE-TO-SITE VPN TOPOLOGY



4. IPSEC VPN

- Secure at Layer 3 (Network Layer).
- Encapsulates and encrypts IP packets.
- Two modes: Transport & Tunnel.
- Strong encryption and authentication.

IPSEC ARCHITECTURE



KEY COMPONENTS

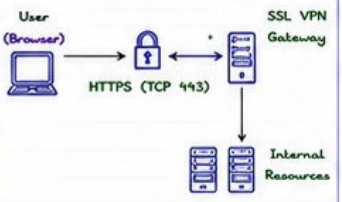
- ☑ IKE (Internet Key Exchange)
- ☑ Encrypts traffic using ESP (Encapsulating Security Payload)
- ☑ Authentication using PSK or Certificates
- ☑ Supports NAT Traversal

Common Ports: UDP 500 (IKE), UDP 4500 (NAT-T), ESP

5. SSL VPN

- Operates at Layer 7 (Application Layer).
- Uses HTTPS for secure communication.
- No special client required (browser-based).
- Ideal for remote users and web access.

SSL VPN ARCHITECTURE



KEY FEATURES

- ☑ Web portal access
- ☑ Granular access control
- ☑ MFA support
- ☑ Easy to deploy and user-friendly

Common Port: TCP 443 (HTTPS)

6. WIREGUARD VPN

- Modern, lightweight VPN protocol.
- High performance and low latency.
- Uses UDP and modern cryptography.
- Simple configuration and easy to manage.

WIREGUARD ARCHITECTURE

KEY FEATURES

- ☑ Very fast and efficient
- ☑ Minimal codebase (easy to audit)
- ☑ Uses advanced crypto (Curve25519, ChaCha20, Poly1305)
- ☑ Supports roaming and NAT traversal
- ☑ Ideal for modern environments

Common Port: UDP 51820

7. REMOTE ACCESS VPN

- Used by individual users to connect to corporate networks.
- Can be IPSec, SSL VPN, or WireGuard.
- Provides secure access to resources.

REMOTE ACCESS FLOW

8. VPN COMPARISON

FEATURE	IPSec VPN	SSL VPN	WireGuard
Layer	Layer 3	Layer 7	Layer 3
Use Case	Site-to-Site, Remote Access	Remote Access, Web Access	Site-to-Site, Remote Access
Transport	UDP (500/4500)	TCP 443	UDP 51820
Performance	High	Medium	Very High
Client Required	Yes	No (Browser)	Yes (Lightweight)
NAT Traversal	Yes	Yes	Yes
Setup Complexity	Medium	Low	Low
Best For	Enterprise Networks	Users, BYOD	Modern Environments

9. VPN SECURITY BEST PRACTICES

- ☑ Use strong encryption (AES-256).
- ☑ Use certificates or strong keys.
- ☑ Enable Perfect Forward Secrecy (PFS).
- ☑ Limit access with least privilege.
- ☑ Enable MFA for remote users.
- ☑ Monitor logs and detect anomalies.
- ☑ Keep VPN software up to date.
- ☑ Regularly test failover and recovery.

10. COMMON VPN TROUBLESHOOTING

- Cannot connect → Check firewall & open ports.
- Authentication failed → Verify credentials / certs.
- No route to destination → Check routing & NAT.
- Slow performance → Check latency & MTU.
- Tunnel drops → Check keepalive & timeouts.
- DNS issues → Use internal DNS or push DNS via VPN.

11. USEFUL COMMANDS (LINUX)

```
# Check IPSec status
ipsec status
# Show IPSec tunnels
ipsec statusall
# Test connectivity
ping -c 4 10.0.2.10
# Show WireGuard status
wg show
# Show WireGuard peers
wg show wg0
# Routes
ip route show
```

12. KEY TAKEAWAY

VPNs allow secure communication across untrusted networks.
Choose the right technology, secure it properly, and monitor it continuously.

VERIQTA

28. PROXY SERVERS

"Proxies sit between clients and servers to add security, control, performance, and visibility."

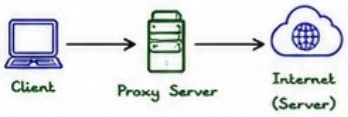
PRODUCTION NOTE

- ☑ Inspect and control traffic.
- ☑ Improve security and privacy.
- ☑ Cache content and save bandwidth.
- ☑ Balance load and increase availability.
- ☑ Log, monitor, and enforce policy.

1. WHAT IS A PROXY SERVER?

- A proxy server acts as an intermediary between clients and servers.
- It can filter, forward, modify, cache, and secure traffic.
- Proxies improve security, control access, performance, and anonymity.

PROXY IN ACTION



All traffic passes through the proxy.

2. TYPES OF PROXY SERVERS OVERVIEW

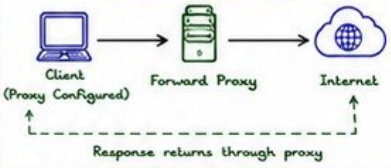
TYPE	DIRECTION	VISIBLE TO CLIENT	MAIN PURPOSE	EXAMPLE USE
Forward Proxy	Client → Internet	Yes	Control outbound traffic, anonymity	Web filtering, content control, privacy
Reverse Proxy	Internet → Server	No	Protect servers, load balancing, SSL offload	Web apps, APIs, Load balancing
Transparent Proxy	Client ↔ Internet (Intercepts)	No	Monitor and filter without client configuration	Web filtering, policy enforcement

Choose the right proxy type based on traffic direction and your security and performance goals.

3. FORWARD PROXY

- Client explicitly configures the proxy.
- Handles outbound requests.
- Useful for access control, caching, logging, and anonymity.

FORWARD PROXY FLOW



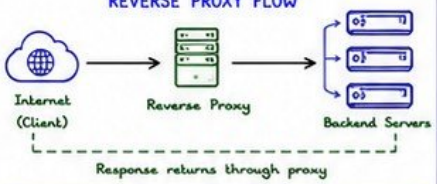
COMMON USE CASES

- ☑ Web filtering and content control
- ☑ User anonymity and privacy
- ☑ Bandwidth saving via caching
- ☑ DLP and policy enforcement
- ☑ Logging and auditing outbound traffic

4. REVERSE PROXY

- Sits in front of backend servers.
- Handles incoming client requests.
- Improves security, scalability, and availability.
- Can perform SSL termination and caching.

REVERSE PROXY FLOW



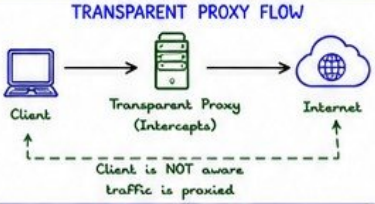
COMMON USE CASES

- ☑ Protect backend servers (hide internal IPs)
- ☑ SSL termination and offloading
- ☑ Load balancing across multiple servers
- ☑ Caching static content
- ☑ WAF and DDoS protection

5. TRANSPARENT PROXY

- Intepacts traffic without client configuration.
- Works at network level (e.g., firewall/router).
- Clients are unaware of the proxy.
- Used for monitoring, filtering, and policy control.

TRANSPARENT PROXY FLOW



COMMON USE CASES

- ☑ Web/content filtering at network level
- ☑ Policy enforcement without user action
- ☑ Malware scanning and threat protection
- ☑ Traffic logging and monitoring
- ☑ Compliance and governance control

6. PROXY FEATURE COMPARISON

FEATURE	FORWARD PROXY	REVERSE PROXY	TRANSPARENT PROXY
Traffic Direction	Client → Internet	Internet → Server	Client ↔ Internet
Client Configured	Yes	No	No
Visible to Client	Yes	No	No
Hides Client IP	Yes	Yes	Yes
Hides Server IP	No	Yes	No
Use for Caching	Yes	Yes	Yes
SSL Termination	No	Yes	No
Common Ports	3128, 8080	80, 443, 8443	Intercepts All

7. ENTERPRISE USE CASES

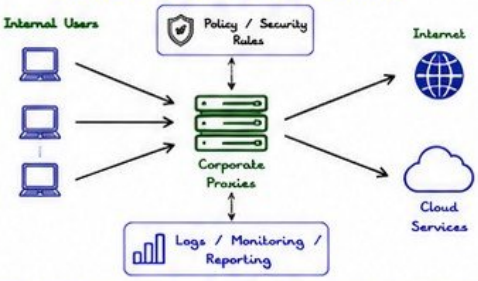
Internet Access Control
Restrict access to websites, categories, or applications.

Data Loss Prevention (DLP)
Inspect content and block sensitive data from leaving the network.

Bandwidth Optimization
Cache frequently used content and reduce external bandwidth.

Security & Threat Protection
Scan for malware, block malicious sites, and enforce security policies.

ENTERPRISE PROXY ARCHITECTURE EXAMPLE



8. PROXY BEST PRACTICES

- ☑ Use strong authentication for proxy access.
- ☑ Keep proxy software and rules updated.
- ☑ Implement HTTPS inspection carefully and legally.
- ☑ Define and enforce least-privilege policies.
- ☑ Enable logging, monitoring, and alerting.
- ☑ Size proxy servers for performance and scale.
- ☑ Use HA and redundancy for critical proxies.
- ☑ Regularly review and audit proxy rules.

KEY TAKEAWAY

Proxies are powerful tools for control, security, and performance—but only when designed, placed, and managed correctly.

9. COMMON PROXY TOOLS

- Squid (Open Source)
- HAProxy (Open Source)
- NGINX (Reverse Proxy)
- Zscaler (Cloud Proxy)
- Blue Coat (Enterprise)

10. QUICK COMMANDS (LINUX)

```
# Test proxy (Forward)
curl -x http://proxy:3128 http://example.com
# Check proxy status (Squid)
sudo systemctl status squid
# View Squid access logs
tail -F /var/log/squid/access.log
# Test reverse proxy (NGINX)
curl -I http://your-domain.com
# Check HAProxy status
sudo systemctl status haproxy
```

VERIQTA

29. ENTERPRISE NETWORK SECURITY

"Security is not a product, it is an architecture.
Layered defenses. Least privilege. Assume breach."

PRODUCTION NOTE

- ☑ Enforce least privilege access.
- ☑ Segment networks and isolate risk.
- ☑ Monitor, detect, and respond fast.
- ☑ Assume breach. Verify everything.
- ☑ Document and review regularly.

1. ACCESS CONTROL LISTS (ACLS)

- ACLs control traffic allowed to or from network interfaces.
- Evaluated top-down, first match wins.
- Can be Stateless or Stateful.
- Applied on routers, firewalls, or cloud security groups.

SAMPLE ACL (STANDARD)

```
access-list 10 permit 10.0.0.0 0.0.0.255
access-list 10 deny any
interface GigabitEthernet0/1
 ip access-group 10 in
```

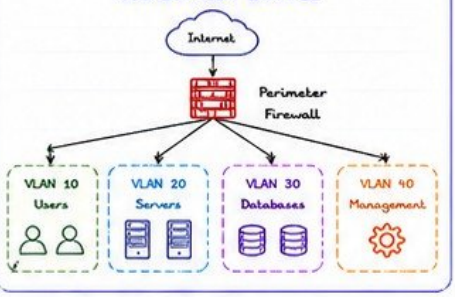
ACL TYPES

- Standard ACL – Filter by source IP only.
- Extended ACL – Filter by source, destination, protocol, and port.

2. NETWORK SEGMENTATION

- Divide network into segments to reduce risk.
- Limit lateral movement of threats.
- Use VLANs, subnets, firewalls, and policies.
- Segment by function, role, or sensitivity.

SEGMENTATION EXAMPLE

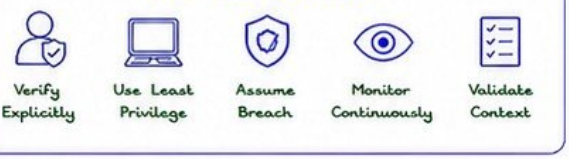


Segment by purpose.
Control access between segments.

3. ZERO TRUST SECURITY MODEL

- Never trust, always verify.
- Verify every user, device, and request.
- Least privilege access for everything.
- Continuous validation and monitoring.

ZERO TRUST PRINCIPLES

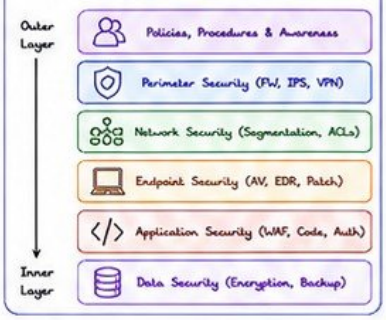


ZERO TRUST FLOW

4. DEFENSE IN DEPTH

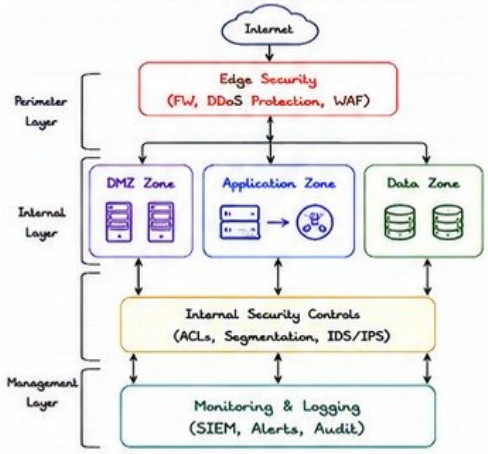
- Multiple layers of security controls.
- If one layer fails, others still protect.
- People, process, technology, and data.

DEFENSE IN DEPTH LAYERS



Layers work together to reduce overall risk.

5. ENTERPRISE SECURITY ARCHITECTURE



Secure architecture + strong controls = Resilient enterprise.

6. SECURITY CONTROL COMPARISON

CONTROL	PURPOSE	LAYER	EXAMPLE
ACLs	Filter traffic	Network	Allow only 10.0.0.0/24
Firewall	Allow/deny traffic	Network	Block port 23
Segmentation	Isolate networks	Network	VLANs, Subnets
IDS/IPS	Detect/Prevent threats	Network	Suricata, Snort
WAF	Protect web apps	Application	Cloudflare WAF
EDR/AV	Protect endpoints	Endpoint	CrowdStrike
Encryption	Protect data	Data	TLS, AES-256
MFA	Strong auth	Access	Duo, Google Auth

SECURITY MINDSET

- ☑ Security is everyone's responsibility.
- ☑ Build security into every layer.
- ☑ Monitor everything.
- ☑ Improve continuously.

7. EXAMPLE ACL – EXTENDED

```
access-list 101 permit tcp 10.0.10.0 0.0.0.255
                host 10.0.20.10 eq 443
access-list 101 permit tcp 10.0.10.0 0.0.0.255
                host 10.0.20.10 eq 22
access-list 101 deny ip any any log
interface GigabitEthernet0/0
 ip access-group 101 in
```

Always end with a deny any and log.
Review ACL hit counts regularly.

8. BEST PRACTICES

- ☑ Implement least privilege everywhere.
- ☑ Use segmentation to contain threats.
- ☑ Apply ACLs at the right place.
- ☑ Use encrypted protocols (TLS, IPSec).
- ☑ Enable MFA for all users.
- ☑ Monitor logs and set alerts.
- ☑ Patch systems and review configs.
- ☑ Test and validate security controls.

REMEMBER

You don't have to be perfect,
you just have to be harder to break.

9. COMMON THREATS & MITIGATIONS

THREAT	RISK	MITIGATION
Port Scanning	Reconnaissance	Firewall, IDS
Brute Force	Account Takeover	MFA, Lockout, WAF
Lateral Movement	Internal Spread	Segmentation, ACLs
Data Exfiltration	Data Loss	DLP, Monitoring
DDoS Attack	Service Unavailable	DDoS Protection
Phishing	Credential Theft	Awareness, Email Filter

10. QUICK COMMANDS (LINUX / NETWORK)

```
# Check IP and routes
ip addr
ip route

# Test connectivity
ping 8.8.8.8
traceroute google.com

# View open ports
ss -tulnp

# Check firewall (Linux)
sudo ufw status

# View iptables rules
sudo iptables -L -n -v

# Check VPN status (IPSec)
sudo ipsec statusall
```

VERIQTA

30. PRODUCTION NETWORK ARCHITECTURE

"Design networks that are secure, scalable, resilient, and easy to operate in production."

PRODUCTION NOTE

- ☑ Design for scale and failure.
- ☑ Isolate and protect critical assets.
- ☑ Use least privilege everywhere.
- ☑ Monitor, log and alert.
- ☑ Document architecture and flows.

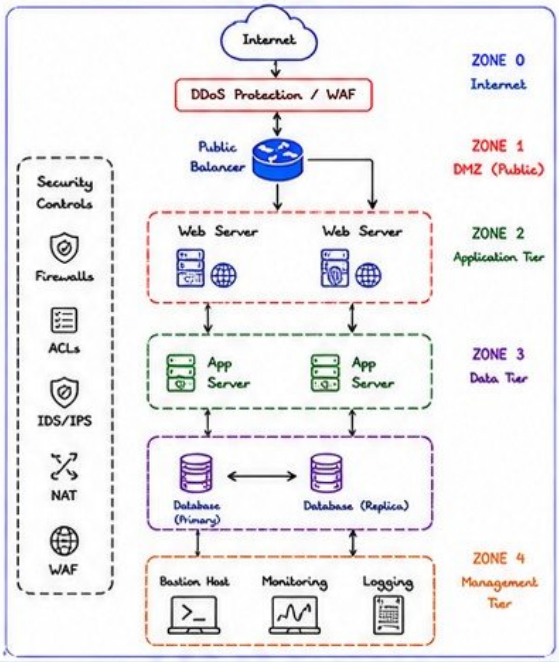
1. MULTI-TIER APPLICATION OVERVIEW

- Separates components by function.
- Improves security, scalability and manageability.
- Each tier sits in a specific network zone.

TYPICAL TIERS

Tier 0	Internet / Users
Tier 1	Web / Presentation
Tier 2	Application / Business Logic
Tier 3	Data / Database
Tier 4	Management / Monitoring

2. PRODUCTION ARCHITECTURE DIAGRAM



3. SECURITY ZONES DESCRIPTION

ZONE	NAME	PURPOSE	EXAMPLE COMPONENTS
0	Internet	Untrusted external network	Users, Attackers, Internet
1	DMZ (Public)	Public facing services	Load Balancer, Web Servers, WAF
2	Application	Application and business logic	App Servers, APIs, Microservices
3	Data	Sensitive data and storage	Databases, Caches, File Storage
4	Management	Operations and monitoring access	Bastion Host, Monitoring, Logs

KEY DESIGN PRINCIPLES

- ☑ Segment by function and trust level.
- ☑ Never expose internal services to the internet.
- ☑ Allow only required traffic between zones.
- ☑ Use firewalls and ACLs to enforce policies.
- ☑ Monitor and log every layer.

4. DMZ (DEMILITARIZED ZONE)

- Acts as a buffer between internet and internal network.
- Hosts public facing services.
- If DMZ is compromised, internal network remains protected.

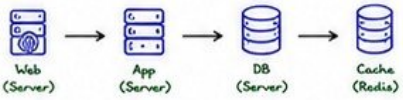
DMZ COMPONENTS

- ☑ Load Balancers
- ☑ Web Servers
- ☑ WAF / Reverse Proxy
- ☑ DNS / Mail Gateways
- ☑ Public APIs

5. INTERNAL SERVICES

- Application servers process business logic.
- Database tier stores critical data.
- Internal communication is restricted to required ports and protocols.

INTERNAL FLOW EXAMPLE



Allowed ports:
Web → App (443) | App → DB (3306) | App → Cache (6379)

6. SECURITY CONTROLS PER ZONE

ZONE	SEGMENTATION	FIREWALL RULES	ACCESS
0 Internet	N/A	Allow only to DMZ	Public
1 DMZ	Public Subnets	Allow 80/443 from Internet Deny all else	Public Services
2 App Tier	Private Subnets	Allow only from DMZ on required ports	Internal
3 Data Tier	Private Subnets	Allow only from App Tier on DB ports	Highly Restricted
4 Mgmt Tier	Isolated Subnet	Allow only from Admin IPs via Bastion	Admin Only

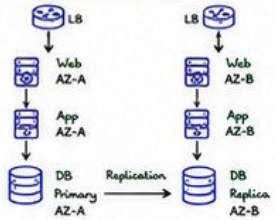
7. TRAFFIC FLOW OVERVIEW

1. User connects to application (HTTPS)
2. Traffic goes to DDoS /WAF
3. Request reaches Public Load Balancer
4. Traffic forwarded to Web Servers in DMZ
5. Web Servers call App Servers in App Tier
6. App Servers access DB in Data Tier

8. HIGH AVAILABILITY CONSIDERATIONS

- ☑ Deploy resources across multiple AZs.
- ☑ Use redundant Load Balancers.
- ☑ Database replication and automatic failover.
- ☑ Use Health Checks for all services.
- ☑ Design for failure, not just traffic.

EXAMPLE HA SETUP



9. MONITORING & OBSERVABILITY

- ☑ Monitor every zone and component.
- ☑ Collect logs from firewalls, servers and applications.
- ☑ Set alerts for failures and anomalies.
- ☑ Use centralized dashboards.

WHAT TO MONITOR

- Network Latency / Packet Loss
- Firewall Logs / Denied Traffic
- Server & Application Health
- Database Performance
- Security Events (IDS/IPS)

10. BEST PRACTICES

- ☑ Follow least privilege at every layer.
- ☑ Keep systems and firmware updated.
- ☑ Use encrypted protocols (TLS).
- ☑ Regular vulnerability scanning.
- ☑ Backup critical data and configs.
- ☑ Document architecture and flows.
- ☑ Test failover and recovery regularly.

KEY TAKEAWAY

A strong network architecture is layered, segmented, and continuously protected. Design for security, reliability and operational simplicity.

VERIQTA

31. CLOUD NETWORKING FUNDAMENTALS

"Cloud networking is the foundation of every scalable, secure, and resilient cloud architecture."

PRODUCTION NOTE

- ☑ Design for security and scalability.
- ☑ Use least privilege and segmentation.
- ☑ Monitor, log and alert everything.
- ☑ Document network designs.

1. WHAT IS CLOUD NETWORKING?

- Cloud networking provides connectivity between resources in the cloud.
- It enables secure communication between users, applications, and services.
- It is the foundation for security, scalability, and high availability.

CLOUD NETWORKING IN ACTION

Fast, secure, and reliable connectivity.

2. CLOUD NETWORKING MODELS

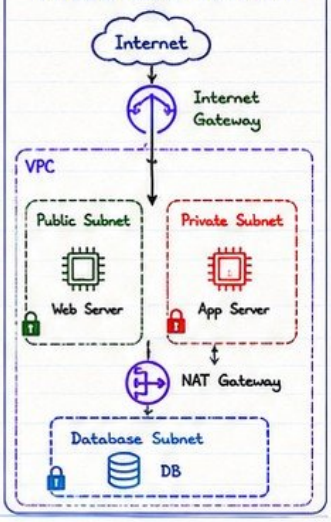
MODEL	DESCRIPTION	EXAMPLE
Public Cloud	Networking provided by cloud provider.	AWS, Azure, GCP
Private Cloud	Dedicated cloud infrastructure for one organization.	VMware, OpenStack
Hybrid Cloud	Combination of public and private cloud networks.	On-prem + AWS
Multi-Cloud	Using multiple cloud providers for different workloads.	AWS + Azure

Choose the right model based on business needs, compliance, and cost.

3. CLOUD NETWORKING COMPONENTS

- Virtual Private Cloud (VPC)
- Subnets (Public, Private)
- Route Tables
- Internet Gateway
- NAT Gateway
- Network ACLs
- Security Groups
- Load Balancers
- VPN / Direct Connect
- DNS Services

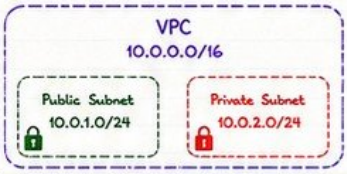
TYPICAL CLOUD NETWORK



4. VIRTUAL PRIVATE CLOUD (VPC)

- Isolated virtual network in the cloud.
- You define IP ranges, subnets, and routing.
- Provides control, security, and scalability.

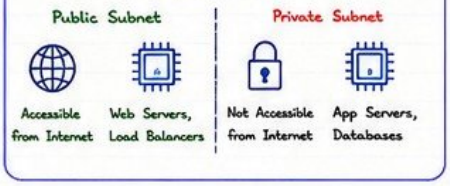
VPC STRUCTURE



5. SUBNETS

- Divide a VPC into smaller IP ranges.
- Public subnets are internet-facing.
- Private subnets are isolated.

SUBNET TYPES



6. ROUTING IN CLOUD

- Route tables control traffic flow.
- Default route to Internet Gateway for public subnets.
- Default route to NAT Gateway for private subnets.

ROUTE TABLE EXAMPLE

DESTINATION	TARGET	PURPOSE
10.0.0.0/16	local	Internal communication
0.0.0.0/0	Internet Gateway	Public subnet internet access
0.0.0.0/0	NAT Gateway	Private subnet internet access

7. SECURITY CONTROLS

- Security Groups act as virtual firewalls at instance level.
- Network ACLs control traffic at subnet level.

SECURITY GROUP (STATEFUL)

- ☑ Allow rules
- ☑ Deny rules
- ☑ Stateful (return traffic allowed)
- ☑ Applied to instances

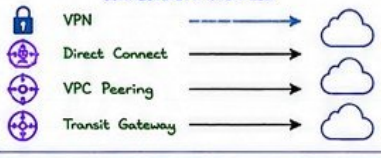
NETWORK ACL (STATELESS)

- ☑ Allow rules
- ☑ Deny rules
- ☑ Stateless (return traffic must be allowed)
- Applied to subnets

8. CLOUD CONNECTIVITY OPTIONS

- VPN – Encrypted connection over Internet.
- Direct Connect – Private, dedicated link.
- Peering – Connect VPCs privately.
- Transit Gateway – Hub-and-spoke connectivity.

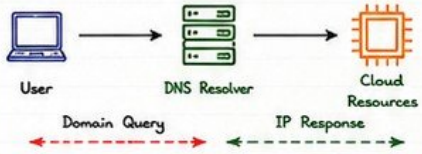
CONNECTIVITY OVERVIEW



9. DNS IN CLOUD

- DNS translates names to IP addresses.
- Use managed DNS for high availability.
- Private DNS for internal resources.

DNS FLOW



10. CLOUD NETWORKING BEST PRACTICES

- ☑ Design VPCs with clear CIDR planning.
- ☑ Use least privilege for security groups and NACLs.
- ☑ Segment workloads using subnets.
- ☑ Use NAT Gateways for private internet access.
- ☑ Monitor network traffic and flow logs.
- ☑ Encrypt traffic in transit (TLS, VPN).
- ☑ Use Route 53 for DNS management.
- ☑ Use tagging for network resources.
- ☑ Review and audit network configurations.
- ☑ Plan for redundancy and high availability.

KEY TAKEAWAY

Cloud networking is the backbone of the cloud. Design it right, secure it well, and your application will scale with confidence.

COMMON CLOUD PORTS

HTTP	80	MySQL	3306
HTTPS	443	PostgreSQL	5432
SSH	22	SMTP	25
DNS	53	IMAPS	993
RDP	3389	SMTPS	465

VERIQTA

32. AWS VPC ARCHITECTURE

"VPC is your logically isolated network in AWS where you control your IP address space, subnets, routing, and security."

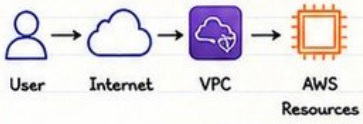
PRODUCTION NOTE

- ☑ Design for security and isolation.
- ☑ Use least privilege and segmentation.
- ☑ Monitor, log and alert everything.
- ☑ Document architecture and flows.

1. WHAT IS AWS VPC?

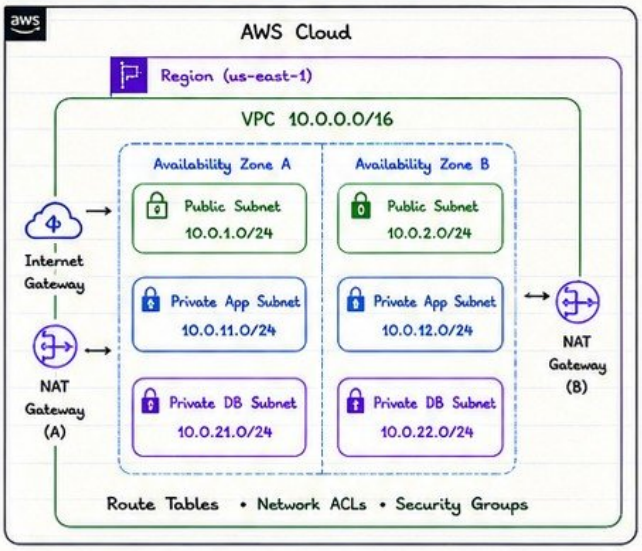
- Amazon VPC lets you launch AWS resources in a logically isolated virtual network.
- You define your IP address range, subnets, routing, gateways, and security.
- You have complete control over your network environment.

VPC IN ACTION



Your own virtual network in the AWS Cloud.

2. AWS VPC ARCHITECTURE OVERVIEW



3. VPC COMPONENTS

- **VPC:** Logically isolated network.
- **Subnets:** Divide the VPC into smaller networks.
- **Internet Gateway:** Enables internet access.
- **NAT Gateway:** Allows private subnets to access the internet.
- **Route Tables:** Control traffic routing.
- **Network ACLs:** Stateless traffic filtering at subnet level.
- **Security Groups:** Stateful traffic filtering at instance level.
- **VPC Endpoints:** Private access to AWS services.
- **Peering / TGW:** Connect VPCs and on-premises networks.

4. IP ADDRESSING IN VPC

- Choose a CIDR block for your VPC.
- Default CIDR: 10.0.0.0/16
- You can have multiple CIDR blocks.
- CIDR sizes: /16 to /28

CIDR EXAMPLE

VPC CIDR:	10.0.0.0/16	
Public Subnets:	10.0.1.0/24	10.0.2.0/24
Private App Subnets:	10.0.11.0/24	10.0.12.0/24
Private DB Subnets:	10.0.21.0/24	10.0.22.0/24

5. ROUTE TABLES

- Each subnet is associated with a route table.
- Routes determine where traffic is sent.
- Main route table is created by default.

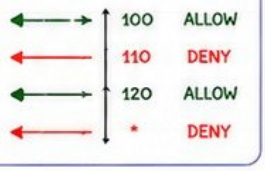
ROUTE TABLE EXAMPLE

DESTINATION	TARGET	PURPOSE
10.0.0.0/16	local	Internal communication
0.0.0.0/0	igw-12345678	Internet access (Public Subnet)
0.0.0.0/0	nat-12345678	Internet access (Private Subnet)

6. NETWORK ACLs vs SECURITY GROUPS

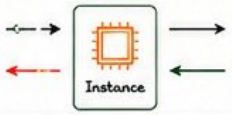
NETWORK ACL (STATELESS)

- Operates at subnet level.
- Allow or Deny rules.
- Evaluated in order (numbered).
- Affects all resources in subnet.



SECURITY GROUP (STATEFUL)

- Operates at instance level.
- Allow rules only.
- Return traffic is auto-allowed.
- Affects only attached instances.



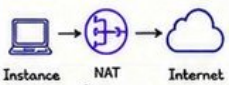
7. INTERNET ACCESS

- Public subnets use Internet Gateway (IGW).
- Private subnets use NAT Gateway.

PUBLIC SUBNET FLOW

Direct internet access

PRIVATE SUBNET FLOW



Outbound internet access only

8. VPC ENDPOINTS

- Private access to AWS services without using internet.
- Improves security and reduces costs.

VPC ENDPOINT TYPES

Gateway Endpoint (S3, DynamoDB)

Interface Endpoint (PrivateLink)

9. VPC PEERING & CONNECTIVITY

- Peering connects VPCs privately.
- Use Transit Gateway for hub-and-spoke.
- Use Direct Connect or VPN for on-prem.

CONNECTIVITY OPTIONS

10. VPC DESIGN BEST PRACTICES

- ☑ Use multiple AZs for high availability.
- ☑ Public subnets for ALB, NAT, Bastion hosts.
- ☑ Private subnets for applications and databases.
- ☑ Use least privilege with Security Groups and NACLs.
- ☑ Enable VPC Flow Logs.
- ☑ Use VPC Endpoints for AWS services.
- ☑ Segment networks by environment (Dev, Test, Prod).
- ☑ Follow CIDR planning to avoid overlap.
- ☑ Monitor and alert on network activity.
- ☑ Document architecture and IP plan.

KEY TAKEAWAY

A well-designed VPC is the foundation of secure, scalable, and highly available applications in AWS.

11. COMMON AWS VPC PORTS

SERVICE	PROTOCOL	PORT	PURPOSE
HTTP	TCP	80	Web traffic
HTTPS	TCP	443	Secure web traffic
SSH	TCP	22	Secure remote access
RDP	TCP	3389	Remote desktop
DNS	UDP	53	Name resolution

VERIQTA

33. SUBNETS IN AWS

"Subnets let you divide your VPC into smaller networks to organize, secure, and scale your resources."

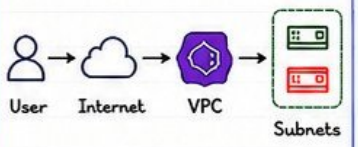
PRODUCTION NOTE

- ☑ Design for security and isolation.
- ☑ Use least privilege and segmentation.
- ☑ Monitor, log and alert everything.
- ☑ Document subnets and IP usage.

1. WHAT IS A SUBNET?

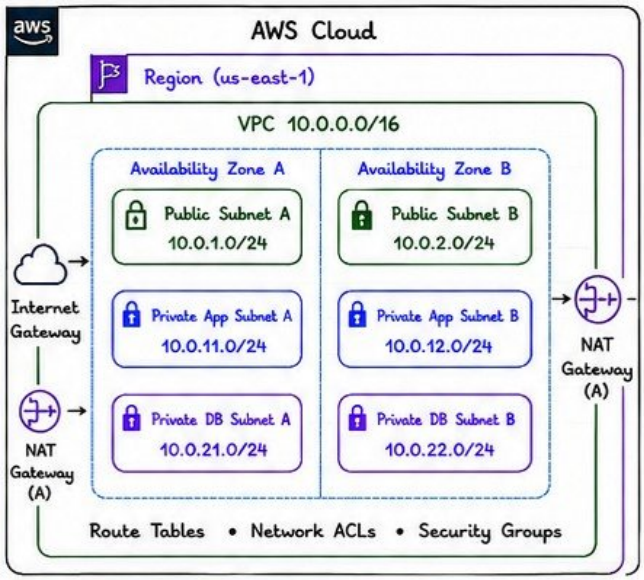
- A subnet is a range of IP addresses within a VPC.
- You can launch AWS resources in a subnet.
- Subnets must be in one Availability Zone.
- Subnets can be public or private.

SUBNET IN ACTION



Organize resources with logical isolation.

2. SUBNETS OVERVIEW



3. TYPES OF SUBNETS

PUBLIC SUBNET

- Has a route to an Internet Gateway.
- Resources can be accessed from the internet.
- Used for: ALB, Bastion, NAT GW, public web servers.

PRIVATE SUBNET

- No direct route to the Internet.
- Resources are not directly accessible from the internet.
- Used for: App servers, databases, internal services.

DATABASE SUBNET

- Private subnet used specifically for databases.
- Extra isolation and security.
- Used for: RDS, Aurora, etc.

4. SUBNET RULES

- Subnets must be within one AZ.
- Subnets cannot overlap.
- Each subnet must have a unique CIDR block.
- You can have multiple subnets per AZ.

CIDR EXAMPLE

VPC CIDR:	10.0.0.0/16	
Public Subnet:	10.0.1.0/24	10.0.2.0/24
Private App:	10.0.11.0/24	10.0.12.0/24
Private DB:	10.0.21.0/24	10.0.22.0/24

5. SUBNET CIDR PLANNING

- Plan IP ranges based on capacity.
- Leave room for growth.
- Follow AWS recommended CIDR best practices.

SUBNET CIDR PLANNING EXAMPLE

TYPE	AZ	CIDR BLOCK	USAGE
Public	us-east-1a	10.0.1.0/24	Web, ALB, NAT GW
Public	us-east-1b	10.0.2.0/24	Web, ALB, NAT GW
Private App	us-east-1a	10.0.11.0/24	App Servers
Private App	us-east-1b	10.0.12.0/24	App Servers
Private DB	us-east-1a	10.0.21.0/24	Databases
Private DB	us-east-1b	10.0.22.0/24	Databases

6. DEFAULT SUBNETS

- AWS creates default subnets in every AZ when you create a VPC.
- You can delete or modify default subnets.
- Best practice: Create your own custom subnets.

DEFAULT SUBNET EXAMPLE

us-east-1a	us-east-1b
172.31.16.0/20	172.31.32.0/20
Default Subnet	Default Subnet

Note: CIDR ranges vary by region.

7. ROUTING IN SUBNETS

- Each subnet is associated with a route table.
- Route tables control traffic flow.
- Public subnet route: 0.0.0.0/0 → IGW
- Private subnet route: 0.0.0.0/0 → NAT GW

ROUTE TABLE EXAMPLE

DESTINATION	TARGET	PURPOSE
10.0.0.0/16	local	Internal (VPC)
0.0.0.0/0	igw-12345678	Internet access (Public Subnet)
0.0.0.0/0	nat-12345678	Internet access (Private Subnet)

8. NETWORK ACLs vs SECURITY GROUPS

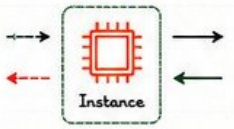
NETWORK ACL (STATELESS)

- Operates at subnet level.
- Allow or Deny rules.
- Evaluated in order (numbered).
- Affects all resources in subnet.

←	100	ALLOW
←	110	ALLOW
←	120	DENY
←	*	DENY

SECURITY GROUP (STATEFEUL)

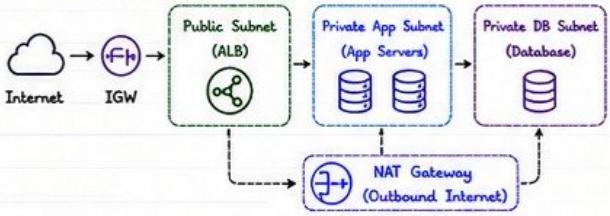
- Operates at instance level.
- Allow rules only.
- Return traffic is allowed.
- Affects only attached instances.



9. SUBNET BEST PRACTICES

- ☑ Use multiple AZs for high availability.
- ☑ Use public subnet for internet-facing resources only.
- ☑ Use private subnet for internal workloads.
- ☑ Place databases in isolated subnets.
- ☑ Use NAT Gateway for outgoing internet access.
- ☑ Monitor subnet IP usage.
- ☑ Use meaningful CIDR blocks.
- ☑ Document subnet design and purpose.

10. SUBNET DESIGN PATTERN



KEY TAKEAWAY

Subnets help you organize your VPC, control network traffic, and secure your workloads. Good subnet design is the foundation of a secure and scalable AWS architecture.

COMMON AWS SUBNET PORTS

SERVICE	PROTOCOL	PORT	PURPOSE
HTTP	TCP	80	Web traffic
HTTPS	TCP	443	Secure web traffic
SSH	TCP	22	Secure remote access
RDP	TCP	3389	Remote desktop
DNS	UDP	53	Name resolution

VERIQTA

34. ROUTE TABLES

"Route tables control where network traffic is sent within your VPC."

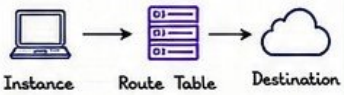
PRODUCTION NOTE

- ☑ Design for simplicity and stability.
- ☑ Use least privilege and segmentation.
- ☑ Monitor, log and alert route changes.
- ☑ Document routbles and IP purpose.

1. WHAT IS A ROUTE TABLE?

- A route table contains a set of rules, called routes.
- Each route defines where traffic is sent for a destination.
- Subnets are associated with a route table.
- Every subnet must have a route table.

ROUTE IN ACTION



Traffic follows the best matching route to reach the destination.

2. ROUTE TABLE COMPONENTS

COMPONENT	DESCRIPTION
Route Table	Container for a set of routes.
Destination	IP address or CIDR block.
Target	Where traffic is sent (IGW, NAT GW, TGW, Local, etc.)
Association	Connects a subnet to a route table.
Main Route Table	Default route table created with every VPC.

3. MAIN ROUTE TABLE

- Created automatically with each VPC.
- Cannot be deleted.
- New subnets are automatically associated with it.
- You can edit routes, but it's a best practice to create custom route tables.

DEFAULT MAIN ROUTE TABLE

- ☑ Associated with all new subnets by default.
- ☑ Contains default "local" route.
- ☑ You can add routes, but use with care.

Default local route:

DESTINATION	TARGET	PURPOSE
10.0.0.0/16	local	Internal VPC communication

4. ROUTE TABLE EXAMPLE

EXAMPLE: PUBLIC SUBNET ROUTE TABLE

DESTINATION	TARGET	PURPOSE
10.0.0.0/16	local	VPC internal communication
0.0.0.0/0	igw-12345678	Internet access
S3 Prefix List	pl-12345678	Access to S3 via prefix list
Prefix List	pl-87654321	Access to DynamoDB via prefix list

Note: Prefix lists help keep routes updated automatically as AWS services scale.

5. ROUTE TARGET TYPES

TARGET	DESCRIPTION
Internet Gateway (IGW)	For internet access from public subnets.
NAT Gateway	For private subnet outbound internet access.
Transit Gateway (TGW)	For connecting multiple VPCs and on-premises networks.
VPC Peering	For routing between peered VPCs.
Virtual Private Gateway	For VPN connections to on-premises.
local	For local VPC communication.

6. LONGEST PREFIX MATCH

- AWS uses longest prefix match to determine the best route.
- More specific routes take priority.
- Example:

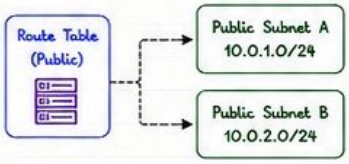
DESTINATION	TARGET	
10.0.1.0/24	local	(More specific)
10.0.0.0/16	igw-12345678	(Less specific)

Traffic to 10.0.1.5 will use 10.0.1.0/24 route, not 10.0.0.0/16.

7. SUBNET ASSOCIATION

- Each subnet must be associated with one route table.
- A route table can be associated with multiple subnets.
- Use custom route tables for better control and isolation.

ASSOCIATION EXAMPLE



8. CREATE CUSTOM ROUTE TABLE

- Go to VPC Console → Route Tables → Create route table.
- Add routes as needed.
- Associate the route table with specific subnets.
- Use for different subnet types (public, private, database).

CUSTOM ROUTE TABLE BENEFITS

- ☑ Better control over traffic flow
- ☑ Isolation between subnet types
- ☑ Easier troubleshooting
- ☑ Support for advanced architectures

9. TROUBLESHOOTING ROUTE ISSUES

- Check if route exists in the table.
- Verify correct route target.
- Confirm subnet association.
- Use Reachability Analyzer.
- Check for conflicting routes.

COMMON ISSUES

- ✗ Missing route to Internet Gateway
- ✗ Wrong route target (e.g., NAT instead of IGW)
- ✗ Subnet not associated with correct route table
- ✗ Overlapping CIDR blocks
- ✗ Incorrect longest prefix match

10. ROUTE TABLE BEST PRACTICES

- ☑ Use custom route tables for different subnet types.
- ☑ Keep route tables simple and easy to understand.
- ☑ Use prefix lists for AWS services.
- ☑ Monitor and audit route changes.
- ☑ Document purpose and ownership.
- ☑ Review route tables regularly.

KEY TAKEAWAY

Route tables are the GPS of your VPC. If the routes are wrong, traffic gets lost. Design, monitor, and document them like a pro.

COMMON ROUTE TARGETS

TARGET	USED FOR
igw-xxxxxxxx	Internet access for public subnets
nat-xxxxxxxx	Outbound internet for private subnets
tgw-xxxxxxxx	Multi-VPC and on-prem connectivity
vgw-xxxxxxxx	VPN to on-premises
pcx-xxxxxxxx	VPC Peering connectivity
local	VPC internal communication

VERIQTA

35. INTERNET GATEWAY
NAT GATEWAY • EGRESS-ONLY GATEWAY

"Gateways connect your VPC to the outside world securely and intelligently."

PRODUCTION NOTE

- ☑ Use the right gateway for the right job.
- ☑ Keep private resources private.
- ☑ Monitor and secure your egress.

1. INTERNET GATEWAY (IGW)

- Allows communication between your VPC and the internet.
- Attached to the VPC.
- Used by public subnets.
- Enables inbound and outbound internet access.

2. NAT GATEWAY

- Allows instances in private subnets to access the internet.
- Allows outbound traffic only.
- Provides internet access for updates, patches, and external APIs.
- Requires an Elastic IP.
- Deployed in a public subnet.

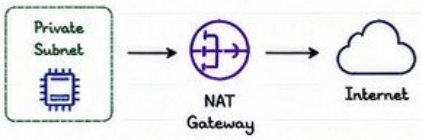
3. EGRESS-ONLY INTERNET GATEWAY

- Allows IPv6 outbound internet access for resources in private subnets.
- No inbound ronnections from the internet.
- Used with IPv6-only subnets.
- No Elastic IP required.

IGW IN ACTION

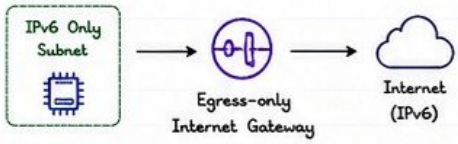
Internet ⟷ Internet Gateway ⟷ Public Subnet Resources
(Two-way communication)

NAT GATEWAY IN ACTION



Private Subnet → NAT Gateway → Internet
(Outbound only)

EGRESS-ONLY GATEWAY IN ACTION

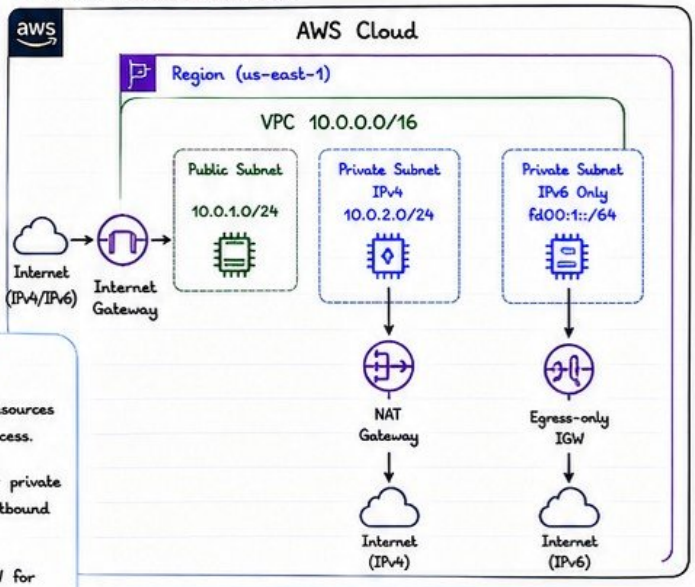


IPv6 Only Subnet → Egress-only IGW → Internet (IPv6)
(Outbound IPv6 only)

4. COMPARISON

FEATURE	INTERNET GATEWAY	NAT GATEWAY	EGRESS-ONLY IGW
Traffic Direction	Inbound & Outbound	Outbound Only	Outbound IPv6 Only
IP Version	IPv4 & IPv6	IPv4 Only	IPv6 Only
Attached To	VPC	Public Subnet	VPC
Required IP	No	Elastic IP (EIP)	No
Use Case	Public resources	Private subnet outbound	IPv6-only outbound
Allows Inbound	Yes	No	No

5. ARCHITECTURE OVERVIEW



6. ROUTING CONFIGURATION

PUBLIC SUBNET ROUTE TABLE	
DESTINATION	TARGET
10.0.0.0/16	local
0.0.0.0/0	Internet Gateway
::/0	Internet Gateway

→ Used by public subnets to access the internet.

PRIVATE SUBNET (IPv4) ROUTE TABLE	
DESTINATION	TARGET
10.0.0.0/16	local
0.0.0.0/0	NAT Gateway

→ Used by private IPv4 subnets for outbound internet access.

PRIVATE SUBNET (IPv6) ROUTE TABLE	
DESTINATION	TARGET
fd00:1::/56	local
::/0	Egress-only IGW

→ Used by IPv6-only subnets for outbound IPv6 internet access.

7. BEST PRACTICES

- ☑ Use IGW for public resources that need internet access.
- ☑ Use NAT Gateway for private subnets that need outbound IPv4 internet access.
- ☑ Use Egress-only IGW for IPv6-only workloads.
- ☑ Place NAT Gateway in a public subnet.
- ☑ Minimize NAT Gateways (one per AZ is enough).
- ☑ Monitor NAT Gateway metrics and data usage.

8. COMMON USE CASES

Internet Gateway:
Public websites, ALB, NAT, VPN termination.

NAT Gateway:
Private EC2 instances, app updates, OS patches, external APIs.

Egress-only IGW:
IPv6-only workloads, container platforms, modern microservices.

KEY TAKEAWAY

Internet Gateway opens the door.
NAT Gateway lets private resources talk out.
Egress-only Gateway gives IPv6 resources secure outbound access.
Use the right gateway to build secure, scalable, and cost-effective architectures.

10. GATEWAY SELECTION GUIDE

NEED	USE THIS GATEWAY	WHY
Public resources need internet	Internet Gateway	Allows inbound and outbound access.
Private IPv4 resources need internet	NAT Gateway	Provides secure outbound internet access.
Private IPv6 resources need internet	Egress-only IGW	Enables outbound IPv6 without inbound.
VPN or Direct Connect attachment	Internet Gateway	Supports routing to/from on-prem.
Simple outbound for IPv6-only	Egress-only IGW	Lightweight and cost-effective.

VERIQTA

36. SECURITY GROUPS vs NETWORK ACLs

"Both control traffic. They work differently, Use both the right way."

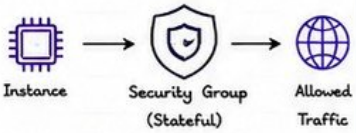
PRODUCTION NOTE

- ☑ Security Groups = stateful
- ☑ Network ACLs = stateless
- ☑ Evaluate SG first, then NACL
- ☑ Use both for strong defense

1. SECURITY GROUPS (SG)

- Stateful firewall for ENIs.
- Attached to EC2, RDS, ALB, etc.
- Allow rules only.
- Return traffic automatically allowed.
- Evaluated at the instance level.

SECURITY GROUP IN ACTION

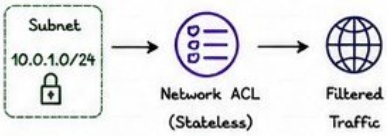


Allow rules you define.
Return traffic allowed automatically.

2. NETWORK ACLs (NACL)

- Stateless firewall for subnets.
- Associated with subnet.
- Allow and Deny rules.
- Return traffic must be allowed explicitly.
- Evaluated at the subnet level.

NETWORK ACL IN ACTION

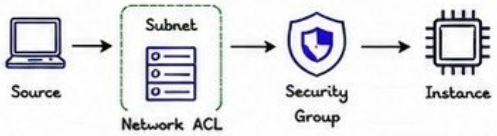


Allow or Deny rules you define.
Return traffic must be allowed.

3. HOW THEY WORK TOGETHER

- Traffic goes through both.
- Security Group evaluated first.
- Then Network ACL.
- If either one denies, traffic is blocked.
- Use SGs for instance-level control.
- Use NACLs for subnet-level control.

TRAFFIC FLOW



Source → NACL → Security Group → Instance

Deny at any point = Blocked

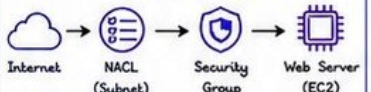
4. SECURITY GROUPS vs NETWORK ACLs COMPARISON

FEATURE	SECURITY GROUPS (SG)	NETWORK ACLs (NACL)	KEY DIFFERENCE
Type	Stateful	Stateless	SG remembers connection, NACL does not.
Applied To	ENI (Instance level)	Subnet (Subnet level)	SG = instance, NACL = subnet.
Rules	Allow only	Allow and Deny	NACL can explicitly deny.
Return Traffic	Automatically allowed	Must be allowed explicitly	Biggest operational difference.
Rule Evaluation	Evaluated first	Evaluated after SG	Order matters.
Default Behavior	Deny all	Allow all	SG blocks by default, NACL allows by default.
Association	Manually attach to resource	Associated with subnet	SG per resource, NACL per subnet.
Example Use	Allow HTTP/HTTPS to web server	Block specific IPs at subnet level	SG for app, NACL for network.

5. EXAMPLE SCENARIO

- Web server in subnet 10.0.1.0/24
- Allow HTTP (80) from internet
- Allow SSH (22) from office IP
- Block all other traffic

SCENARIO DIAGRAM



NACL filters subnet traffic.
SG allows only what instance needs.

6. EXAMPLE RULES

NETWORK ACL (SUBNET 10.0.1.0/24)

Rule #	Type	Protocol	Port	Source/Destination	Action
100	Allow	TCP	80	0.0.0.0/0	ALLOW
110	Allow	TCP	22	203.0.113.10/32	ALLOW
200	Deny	All	All	0.0.0.0/0	DENY
*	Allow	All	All	0.0.0.0/0	ALLOW

Remember: Return traffic must be allowed.

SECURITY GROUP (WEB SERVER SG)

Type	Protocol	Port	Source	Action
HTTP	TCP	80	0.0.0.0/0	ALLOW
SSH	TCP	22	203.0.113.10/32	ALLOW

Return traffic automatically allowed.

7. BEST PRACTICES

- ☑ Use Security Groups for instance-level security.
- ☑ Use Network ACLs for subnet-level protection.
- ☑ Follow least privilege principle.
- ☑ Keep rules simple and documented.
- ☑ Test rules before blocking traffic.
- ☑ Monitor and review rules regularly.

DEFENSE IN DEPTH

NACL protects the subnet.
SG protects the instance.
Together = Strong Security.

KEY TAKEAWAY

Use Security Groups to control what reaches your instances. Use Network ACLs to control what enters and leaves your subnets. Together, they keep your AWS environment secure.

8. COMMON USE CASES

- **Security Groups:** Control app access, database access, management access.
- **Network ACLs:** Block bad IP ranges, restrict subnet access, enforce compliance.
- **Both Together:** Build layered security, reduce attack surface.

VERIQTA

37. VPC PEERING • TRANSIT GATEWAY • PRIVATE CONNECTIVITY

"Connect your VPCs, on-premises, and private services securely and efficiently."

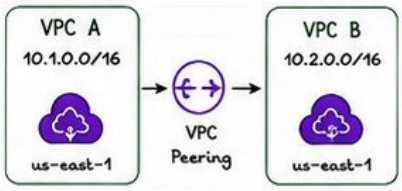
PRODUCTION NOTE

- ☑ Design for scalability and growth.
- ☑ Use the right connectivity option.
- ☑ Secure private traffic always.
- ☑ Monitor and optimize connectivity.

1. VPC PEERING

- Connects two VPCs privately using AWS backbone.
- No transitive routing.
- No overlapping CIDRs.
- Supports IPv4 and IPv6.
- Use for simple, direct VPC-to-VPC communication.

VPC PEERING IN ACTION

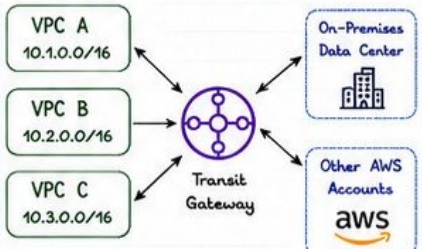


Route tables updated on both VPCs.
Traffic flows privately over AWS backbone.

2. TRANSIT GATEWAY (TGW)

- Central hub for connecting multiple VPCs and on-premises networks.
- Simplifies network management.
- Supports transitive routing.
- Scales to thousands of attachments.
- Use for complex, scalable architectures.

TRANSIT GATEWAY IN ACTION



Centralized routing, segmentation, and control.

3. PRIVATE CONNECTIVITY OPTIONS

- Keep traffic private and secure.
- No exposure to the public internet.
- Multiple options based on use case.
- Lower latency and higher security.
- Supports hybrid and multi-cloud.

PRIVATE CONNECTIVITY OPTIONS

AWS Direct Connect
Dedicated private connection to AWS.

AWS VPN
Encrypted connection over the internet.

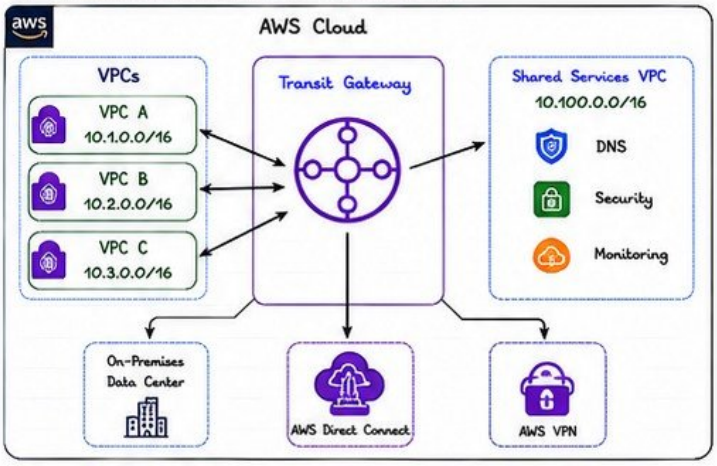
Transit Gateway
Hub for VPCs and on-premises connectivity.

VPC Peering
Direct private connection between two VPCs.

4. CONNECTIVITY OPTIONS COMPARISON

FEATURE	VPC PEERING	TRANSIT GATEWAY	DIRECT CONNECT	VPN
Use Case	VPC to VPC	Hub & Spoke (VPCs + On-Prem)	On-Prem to AWS Dedicated	On-Prem to AWS Internet-based
Scale	Limited	High (Thousands of attachments)	High	Moderate
Transitive Routing	No	Yes	Yes	Yes
Public Internet	No	No	No	Yes (encrypted)
Bandwidth	Up to 10 Gbps	Up to 10 Gbps per attachment	Up to 100 Gbps+	Up to 1.25 Gbps
Setup Complexity	Low	Medium	High	Medium
Cost	Low	Medium	High	Low

5. ENTERPRISE CONNECTIVITY ARCHITECTURE



6. ROUTING EXAMPLE

VPC PEERING ROUTES

In VPC A Route Table

Destination	Target
10.2.0.0/16	pcx-123abc
0.0.0.0/0	igw-123456

In VPC B Route Table

Destination	Target
10.1.0.0/16	pcx-123abc
0.0.0.0/0	igw-789012

VPC Peering

TRANSIT GATEWAY ROUTES

TGW Route Table

Destination	Target
10.1.0.0/16	tgw-attach-vpc-a
10.2.0.0/16	tgw-attach-vpc-b
192.168.0.0/16	tgw-attach-dx

TGW propagates and routes traffic between all attachments.

7. BEST PRACTICES

- ☑ Use VPC Peering for simple, few VPC connections.
- ☑ Use Transit Gateway for scalable hub-and-spoke.
- ☑ Use Direct Connect for stable, high bandwidth needs.
- ☑ Use VPN for backup or low bandwidth needs.
- ☑ Avoid overlapping CIDRs.
- ☑ Segment networks using route tables.
- ☑ Monitor connections with CloudWatch.
- ☑ Enable Flow Logs for visibility.

PRINCIPLE

Choose the right connectivity option for your use case. Simplify when possible, but design for scale when needed.

8. COMMON USE CASES

- **VPC Peering:** Connect dev and test VPCs. Simple, cost-effective.
- **Transit Gateway:** Connect multiple VPCs and on-premises networks.
- **Direct Connect:** High bandwidth, low latency private connection.
- **VPN:** Secure encrypted connection over the internet.

KEY TAKEAWAY

Build secure, scalable, and high-performance networks by using VPC Peering for simplicity, Transit Gateway for scale, and private connectivity for hybrid architectures.

9. CONNECTIVITY DESIGN CHECKLIST

- ☑ Do VPC CIDR ranges not overlap?
- ☑ Are route tables updated correctly?
- ☑ Is the right connectivity option chosen?
- ☑ Is security enforced (SG, NACL, Firewall)?
- ☑ Is monitoring and logging enabled?
- ☑ Is the design scalable for future growth?
- ☑ Is there a backup path if needed?

VERIQTA

38. HYBRID NETWORKING • DIRECT CONNECT • VPN • MULTI-REGION NETWORKING

"Extend your network beyond AWS with secure, reliable, and high-performance connectivity."

PRODUCTION NOTE

- ☑ Design for reliability and resiliency.
- ☑ Use private connectivity first.
- ☑ Encrypt traffic in transit.
- ☑ Monitor connectivity continuously.
- ☑ Test failover and recovery paths. ⚠

1. HYBRID NETWORKING OVERVIEW

- Connect on-premises networks to AWS.
- Use Direct Connect or VPN.
- Keep traffic private, secure, and fast.
- Enable hybrid cloud architectures.
- Support applications, data, and users.

2. AWS DIRECT CONNECT

- Dedicated network connection to AWS.
- High bandwidth, low latency.
- More consistent performance.
- Private connectivity, not over the internet.
- Use for production workloads.

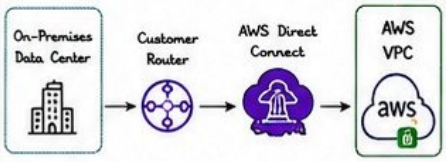
3. VPN (SITE-TO-SITE)

- Encrypted connection over the internet.
- Use VPN Gateway in AWS.
- Good for backup or smaller bandwidth.
- Cost-effective and easy to set up.
- Supports high availability (HA VPN).

HYBRID NETWORKING IN ACTION

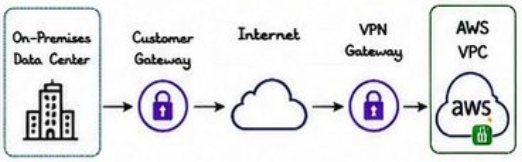
Private and secure connectivity between your data center and AWS.

DIRECT CONNECT ARCHITECTURE



Traffic stays on the AWS backbone.
More secure, more reliable, more predictable.

VPN ARCHITECTURE

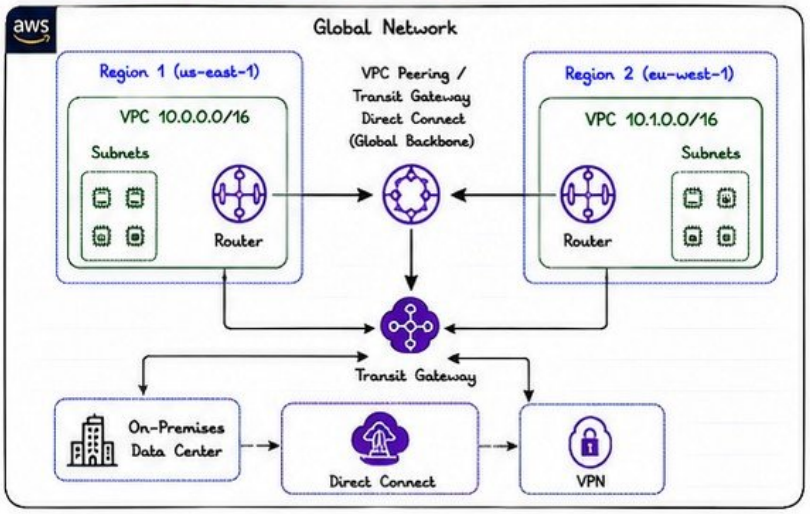


Traffic is encrypted over the internet.
Lower bandwidth and more variable latency.

4. CONNECTIVITY OPTIONS COMPARISON

FEATURE	DIRECT CONNECT	VPN (SITE-TO-SITE)
Connection Type	Dedicated Private Link	Encrypted over Internet
Performance	High, Consistent	Variable
Bandwidth	Up to 100 Gbps+	Up to 1.25 Gbps (per tunnel)
Latency	Low and Predictable	Higher and Variable
Reliability	High	Medium
Security	Private, Not over Internet	Encrypted (IPsec)
Use Case	Production Workloads	Backup, Dev/Test, Low Bandwidth
Cost	Higher	Lower

5. MULTI-REGION NETWORKING ARCHITECTURE



6. CONNECTIVITY WORKFLOW

DIRECT CONNECT FLOW

On-Premises Router
↓
Direct Connect Location
↓
AWS Direct Connect
↓
AWS VPC

VPN FLOW

On-Premises Router
↓
Internet
↓
AWS VPN Gateway
↓
AWS VPC

TRAFFIC FLOW

- ☑ Hybrid workloads access AWS.
- ☑ Data flows securely and privately.
- ☑ Use the right connectivity for the right workload.

7. BEST PRACTICES

- ☑ Use Direct Connect for mission-critical workloads.
- ☑ Use VPN for backup and non-critical workloads.
- ☑ Enable HA VPN for redundancy.
- ☑ Monitor connectivity with CloudWatch and alarms.
- ☑ Use Transit Gateway for scalable multi-VPC and multi-region.
- ☑ Encrypt traffic everywhere.

KEY TAKEAWAY

Hybrid networking connects your world to the cloud.
Use Direct Connect for performance, VPN for cost-effectiveness, and design for high availability.

8. COMMON USE CASES

- Extend data center to AWS.
- Disaster recovery.
- Migrate workloads securely.
- Access AWS services privately.
- Multi-region applications.

9. CONNECTIVITY CHECKLIST

- ☑ Choose Direct Connect or VPN?
- ☑ Is bandwidth requirement met?
- ☑ Is HA/Redundancy configured?
- ☑ Are routes configured correctly?
- ☑ Is security (SG, NACL, Firewall) in place?
- ☑ Is monitoring and alerting enabled?
- ☑ Is failover tested?
- ☑ Is documentation updated?
- ☑ Is backup path available?

VERIQTA

39. DNS IN AWS • ROUTE 53 PRIVATE HOSTED ZONES • RESOLVER

"DNS is the phonebook of your cloud.
Route 53 makes it fast, reliable, and private."

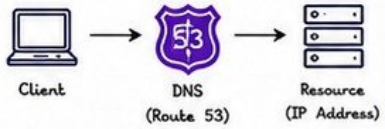
PRODUCTION NOTE

- ☑ Good DNS design = reliable apps.
- ☑ Use private DNS for internal apps.
- ☑ Keep TTLs low for dynamic systems.
- ☑ Monitor DNS health and latency.
- ☑ Document zones, records, and flows.

1. DNS IN AWS OVERVIEW

- DNS translates names to IP addresses.
- Route 53 is AWS's scalable DNS service.
- Used for external and internal domains.
- Highly available, low latency, and secure.
- Supports routing, health checks, and DNS failover.

DNS RESOLUTION FLOW



Client asks DNS → DNS resolves → Returns IP → Client connects.

2. AMAZON ROUTE 53 (OVERVIEW)

- Scalable DNS with 100% SLA.
- Supports public and private hosted zones.
- Traffic routing policies for availability.
- Health checks for endpoints and resources.
- Integrates with AWS services.

ROUTE 53 CAPABILITIES

Domain Registration	DNS (Hosted Zones)
Traffic Routing	Health Checks
DNS Firewall (Shield)	Monitoring (CloudWatch)

3. DNS TYPES IN ROUTE 53

- Public Hosted Zones: For internet domains.
- Private Hosted Zones: For internal domains.
- Resolver: For DNS queries between VPCs and on-premises.

DNS TYPES

Public Hosted Zone (example.com)	Resolves records for internet users.
Private Hosted Zone (internal.local)	Resolves records within VPC / private network.
Resolver (Inbound/Outbound)	Resolves DNS between VPCs and on-premises.

4. PUBLIC HOSTED ZONE EXAMPLE

- Used for internet-facing domains.
- Managed by Route 53 name servers.
- Example domain: veriqta.com

PUBLIC HOSTED ZONE RECORDS

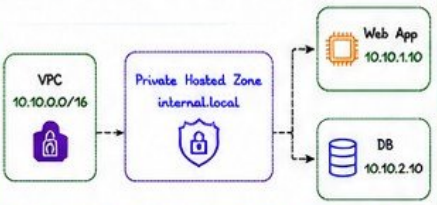
RECORD	TYPE	VALUE	TTL
veriqta.com	A	203.0.113.10	300
www.veriqta.com	CNAME	veriqta.com	300
api.veriqta.com	A	203.0.113.20	300
mail.veriqta.com	MX	10 mail.veriqta.com	300

Records are public and resolvable over the internet.

5. PRIVATE HOSTED ZONE

- Used for internal domain resolution.
- Associates with one or more VPCs.
- Not resolvable from the internet.
- Example domain: internal.local

PRIVATE HOSTED ZONE EXAMPLE

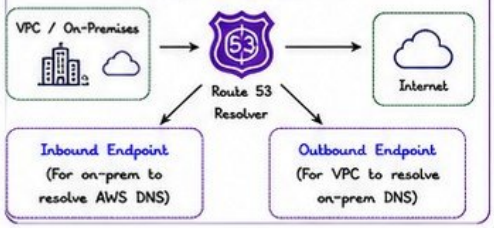


VPC resources resolve internal names to private IPs.

6. ROUTE 53 RESOLVER

- Resolves DNS between VPCs and on-premises.
- Two components: Inbound Endpoints & Outbound Endpoints.
- Uses DNS over TLS for encryption.
- Supports conditional forwarding.

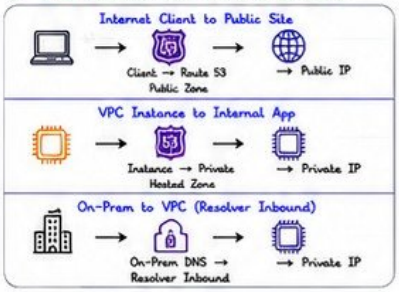
RESOLVER ARCHITECTURE



7. DNS SOLUTIONS COMPARISON

FEATURE	PUBLIC HOSTED ZONE	PRIVATE HOSTED ZONE	ROUTE 53 RESOLVER	USE CASE
Visibility	Internet	VPC / Private Network	VPC ↔ On-Premises	Where DNS resolution is needed
Resolvable From	Anywhere	Associated VPCs only	VPCs and On-Premises	Internal apps, hybrid apps
Domain Example	example.com	internal.local	Custom / Any Domain	Internet, intranet, hybrid
Internet Reachable	✅ Yes	❌ No	❌ No (uses endpoints)	Security Control
Configuration	Public NS records	Associate with VPC	Create endpoints & rules	Setup Complexity
Cost	Low	Low	Medium	Resolver endpoints incur cost

8. DNS RESOLUTION PATH EXAMPLES



9. BEST PRACTICES

- ☑ Use meaningful DNS naming conventions.
- ☑ Use Private Hosted Zones for internal apps.
- ☑ Keep TTLs low for dynamic environments.
- ☑ Use health checks with routing policies.
- ☑ Use Resolver for hybrid and multi-VPC DNS.
- ☑ Enable DNS query logging for visibility.
- ☑ Use DNS over TLS for privacy and security.
- ☑ Document zones, records, and forwarding rules.

KEY TAKEAWAY

Route 53 gives you powerful DNS for every use case — public, private, or hybrid. Design DNS right, and everything else just works.

10. COMMON USE CASES

- Host public websites and APIs.
- Run internal apps in private networks.
- Connect on-premises data centers.
- Support multi-region applications.
- Enable disaster recovery and failover.

VERIQTA

40. CLOUD NETWORKING BEST PRACTICES

"Design secure, scalable, and cost-effective networks that grow with your business."

PRODUCTION NOTE

- ☑ Plan for scale and growth.
- ☑ Secure by design.
- ☑ Automate and document.
- ☑ Monitor, alert, and optimize.
- ☑ Test disaster recovery.

1. MULTI-ACCOUNT NETWORKING

- Use multiple AWS accounts to isolate workloads.
- Centralize networking and security controls.
- Share services across accounts securely.
- Improve security, billing segregation, and scale.

2. SHARED SERVICES MODEL

- Centralize shared services in dedicated accounts.
- Examples: DNS, Security, Logging, Monitoring.
- Reduces duplication and simplifies management.
- Enforce access using IAM and resource policies.

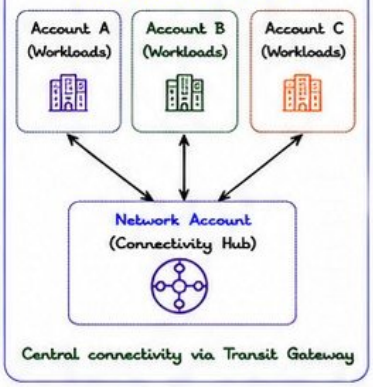
3. LANDING ZONE DESIGN

- Provide a secure baseline for new AWS environments.
- Pre-configured accounts, network, and guardrails.
- Built using AWS Control Tower or Organizations.
- Enforces compliance and accelerates delivery.

4. ENTERPRISE NETWORK DESIGN

- Design for scale, security, and high availability.
- Use hub-and-spoke or mesh models.
- Enforce segmentation and least privilege.
- Automate with IaC and monitor everything.

MULTI-ACCOUNT MODEL



Central connectivity via Transit Gateway

SHARED SERVICES ARCHITECTURE

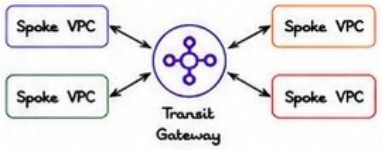
- **Security Account** — GuardDuty, Security Hub, WAF, Firewall Manager
- **Logging Account** — CloudWatch Logs, S3 Log Archive, Athena
- **DNS Account** — Route 53 Resolver, Central DNS
- **Monitoring Account** — CloudWatch, Grafana, Alerting, Dashboards

LANDING ZONE COMPONENTS

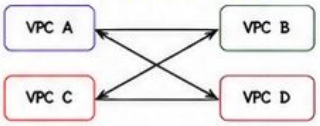
- **AWS Organizations** — Multi-account management
- **Control Tower** — Automated account setup
- **Identity Center** — Centralized access (SSO)
- **Guardrails** — Preventive + detective controls
- **Baseline Network** — VPC, Subnets, TGW, DNS

ENTERPRISE NETWORK TOPOLOGIES

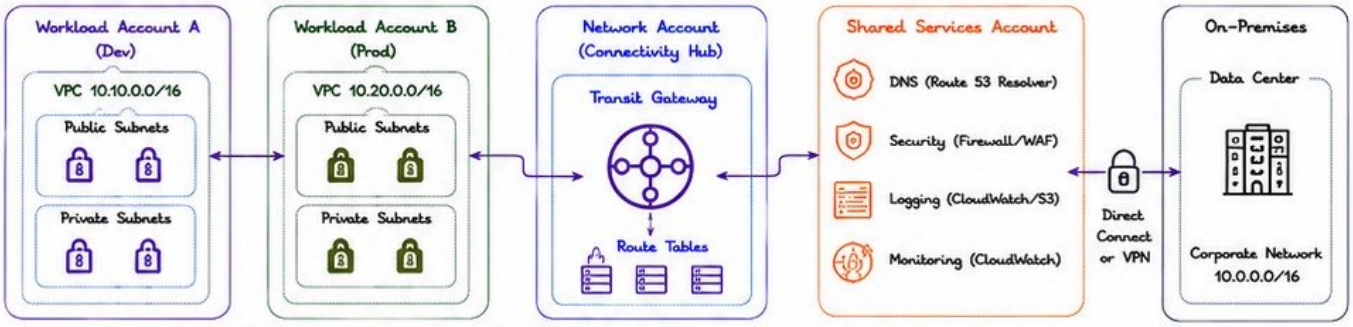
HUB AND SPOKE



MESH (FULLY CONNECTED)



5. ENTERPRISE NETWORK ARCHITECTURE (EXAMPLE)



All accounts connected via Transit Gateway for secure and centralized communication.

6. DESIGN PRINCIPLES

- ☑ **Security First:** Least privilege access, segmentation, encryption.
- ☑ **Reliability:** Multi-AZ, redundant paths, automated failover.
- ☑ **Scalability:** Design for growth, avoid limits and bottlenecks.
- ☑ **Operational Excellence:** Automate, document, monitor.
- ☑ **Cost Optimization:** Right-size, use TGW route filtering, reduce NAT usage.

7. BEST PRACTICES CHECKLIST

- ☑ Use multiple accounts for isolation.
- ☑ Centralize connectivity with Transit Gateway.
- ☑ Use private subnets for workloads.
- ☑ Use NAT Gateway or egress-only IGW as needed.
- ☑ Enable DNS resolution for all VPCs.
- ☑ Use PrivateLink for private service access.
- ☑ Enforce tagging and naming standards.
- ☑ Use IaC (Terraform/Cloudformation).
- ☑ Enable CloudWatch and VPC Flow Logs.
- ☑ Review and test network controls regularly.

KEY TAKEAWAY

Great cloud networks don't just connect resources — they protect your business, enable scale, and deliver reliability every day.

Plan well, automate, and continuously improve.

8. COMMON USE CASES

- Multi-account enterprises with strict separation.
- Hybrid environments (on-prem + AWS).
- Shared security, logging, and monitoring services.
- Global applications with multi-region networking.

9. PATTERN COMPARISON

PATTERN	BEST FOR	PROS	CONS
Hub and Spoke	Most enterprises	Central control, easy to manage, scalable, cost-effective	Adds a hop, TGW limits
Full Mesh	High performance needs	Low latency, high availability	Complex, high cost
Multi-Region Hub	Global applications	Centralized services, consistent policies	More complex routing
Decentralized	Small/simple environments	Simple, low cost	Harder to scale and govern

VERIQTA

41. KUBERNETES NETWORKING ARCHITECTURE

"Kubernetes networking connects containers, services, clusters, and the outside world securely and reliably."

PRODUCTION NOTE

- [x] Design for isolation and security.
- [x] Use network policies always.
- [x] Encrypt traffic in transit.
- [x] Observe, monitor, and troubleshoot.
- [x] Plan for scale and multi-cluster.

1. KUBERNETES NETWORKING OVERVIEW

- Every Pod gets its own IP address.
- Pods can communicate with each other without NAT.
- Services provide stable virtual IPs.
- Ingress exposes applications to users.
- CNI plugin handles Pod networking.
- Network Policies control traffic.
- Works across nodes, AZs, and regions.

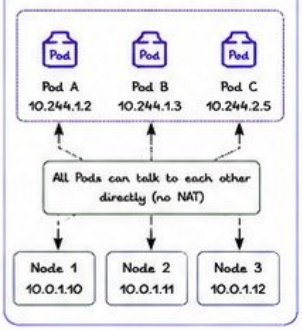
KEY NETWORKING BUILDING BLOCKS

- Pod Network (Flat or Overlay)
- Services (ClusterIP, NodePort, LoadBalancer)
- Ingress Controller
- Network Policies
- CNI Plugin (Calico, Cilium, Weave, etc.)
- DNS (CoreDNS)
- External Connectivity (Internet, On-prem)

2. POD NETWORKING

- Each Pod gets a unique IP.
- Pods communicate directly.
- No NAT between Pods.
- Works across nodes.
- Implemented by CNI plugin.

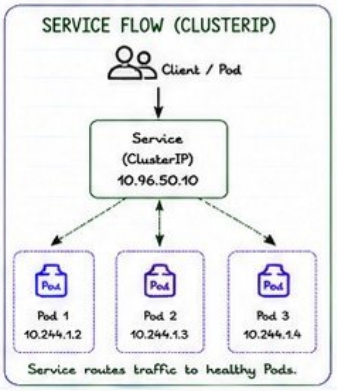
POD NETWORK EXAMPLE



3. SERVICES

- Provide stable virtual IP and DNS.
- Decouple clients from Pods.
- Types: ClusterIP, NodePort, LoadBalancer, ExternalName.
- Kube-proxy routes traffic to Pods.

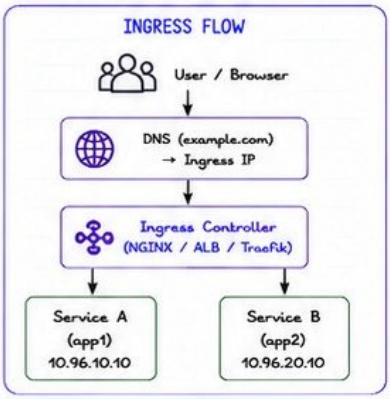
SERVICE FLOW (CLUSTERIP)



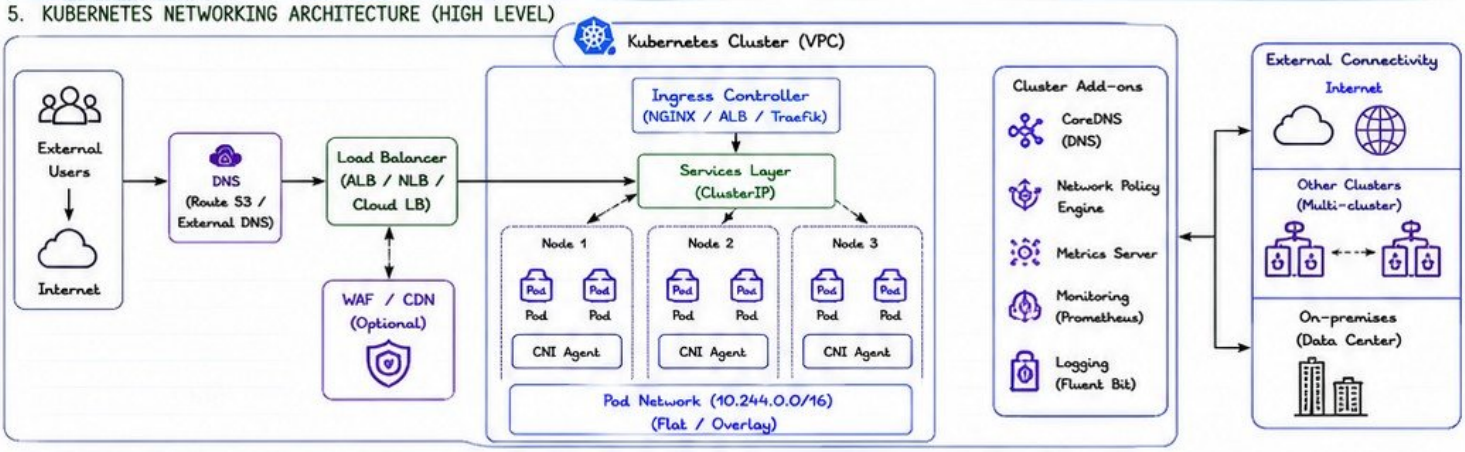
4. INGRESS

- HTTP/HTTPS access to services.
- Layer 7 routing (host/path based).
- TLS termination and WAF integration.
- Managed by Ingress Controller.
- Example: NGINX, ALB, Traefik.

INGRESS FLOW



5. KUBERNETES NETWORKING ARCHITECTURE (HIGH LEVEL)

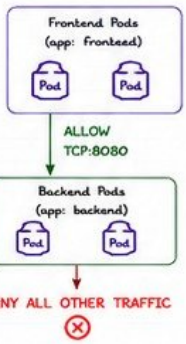


6. CNI PLUGINS (EXAMPLES)

PLUGIN	TYPE	FEATURES
Calico	Overlay/Native	Network Policies, BGP, scalable
Cilium	eBPF	High performance, L7 policies
Weave Net	Overlay	Simple, easy to setup
Amazon VPC CNI	VPC Native	AWS VPC integration
Flannel	Overlay	Simple, lightweight

7. NETWORK POLICY EXAMPLE

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 8080
```



8. BEST PRACTICES

- [x] Use a CNI plugin that supports Network Policies.
- [x] Default deny all ingress and egress.
- [x] Use Services instead of Pod IPs.
- [x] Use Ingress for all external access.
- [x] Encrypt traffic (TLS) everywhere.
- [x] Monitor network flow and latency.
- [x] Use DNS for service discovery (CoreDNS).
- [x] Design for multi-AZ and multi-cluster.

KEY TAKEAWAY

Kubernetes networking provides secure, scalable and reliable connectivity for your applications. Understand the building blocks, enforce policies, and observe everything.

9. COMMON TROUBLESHOOTING COMMANDS

```
# Check Pod IP
kubectl get pods -o wide
# Check Services
kubectl get svc
# Check Endpoints
kubectl get endpoints <service-name> -o wide
# Describe Network Policy
kubectl describe networkpolicy
# Test connectivity
kubectl run -it --rm netshoot --image=nicolaka/netshoot
# Capture traffic (on node)
sudo tcpdump -i any port 80 -n -vv
```

10. INTERVIEW TIPS

- Explain how Pod-to-Pod communication works.
- Differentiate ClusterIP, NodePort, LoadBalancer.
- Explain how Ingress Controller works.
- What is Network Policy and why it's important?
- How would you debug a connectivity issue?

VERIQTA

42. POD NETWORKING

"In Kubernetes, every Pod gets its own IP address and can communicate with any Pod in the cluster."

PRODUCTION NOTE

- ☑ Flat network for all Pods.
- ☑ Every Pod is reachable.
- ☑ No NAT between Pods.
- ☑ Secure with Network Policies.
- ☑ Design for scale and performance.

1. POD NETWORKING OVERVIEW

- Every Pod gets a unique IP address.
- All Pods can communicate with each other directly.
- No NAT between Pods.
- Uses CNI plugin for networking.
- Supports Services, DNS, and Network Policies.

POD NETWORKING PRINCIPLES

- Unique IP per Pod
- Direct Pod-to-Pod Communication
- Flat Network (No NAT)
- Policy-based Security
- Scalable and Highly Available

2. POD IP ADDRESSING

- IPs are assigned from the Pod CIDR (range per node).
- Each node has a Pod CIDR block.
- IPs are assigned by the CNI plugin.
- Pods retain IP for their lifetime.

POD CIDR EXAMPLE

Cluster Pod CIDR: 10.244.0.0/16

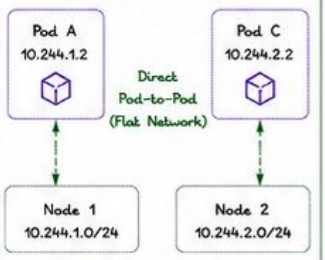
Node 1 (10.244.1.0/24)	Node 2 (10.244.2.0/24)
Pod A 10.244.1.2	Pod C 10.244.2.2
Pod B 10.244.1.3	Pod D 10.244.2.3

All Pods can talk to each other across nodes.

3. POD-TO-POD COMMUNICATION

- Any Pod can reach any other Pod without NAT.
- Communication happens at Layer 3.
- Works across nodes and AZs.
- Example: Pod A → Pod C.

POD-TO-POD FLOW



No NAT, no proxies.
Pure IP routing.

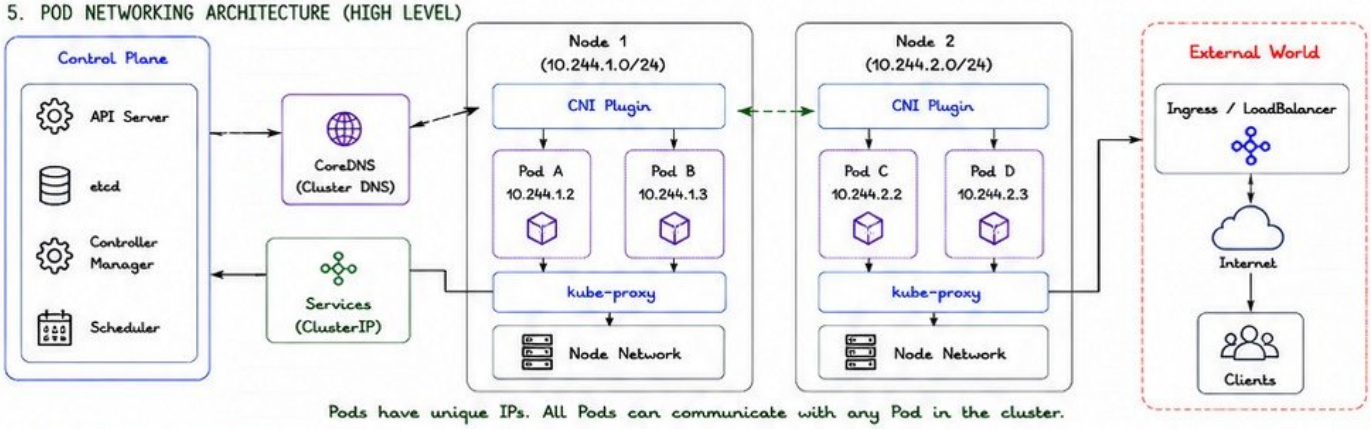
4. POD NETWORKING STACK

- CNI plugin creates the network.
- Assigns IP, routes, and interfaces.
- Works with kube-proxy, CoreDNS, Services, and Ingress.
- Supports overlay or native routing.

POD NETWORKING STACK

- Application (in Pod)
- Kubernetes (Pods, Services, DNS)
- CNI Plugin (Calico, Cilium, Flannel)
- Host Network (Node)
- Physical Network

5. POD NETWORKING ARCHITECTURE (HIGH LEVEL)

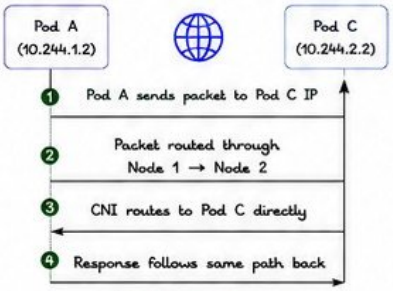


Pods have unique IPs. All Pods can communicate with any Pod in the cluster.

6. CNI PLUGINS COMPARISON

PLUGIN	TYPE	FEATURES	BEST FOR
Calico	Route-based	Network Policies, BGP, High performance	Production Enterprise
Cilium	eBPF	High performance, L7 policies, Observability	Advanced Security Observability
Flannel	Overlay	Simple, easy to deploy	Small Clusters Learning
Weave Net	Overlay	Simple, built-in encryption	Small to Medium Clusters
Amazon VPC CNI	VPC Native	AWS VPC integration, ENI per Pod	AWS EKS Production

7. POD NETWORKING FLOW (EXAMPLE)



No NAT. Pure IP routing.

8. BEST PRACTICES

- ☑ Use CNI plugin that supports Network Policies.
- ☑ Use Pod CIDR big enough for scale.
- ☑ Monitor IP exhaustion on nodes.
- ☑ Enable Network Policies for security.
- ☑ Use CoreDNS for stable name resolution.
- ☑ Encrypt traffic with TLS for services.
- ☑ Use Observability (Cilium, Calico, Prometheus).

KEY TAKEAWAY

Kubernetes pod networking is built on a flat, routable network. Every Pod is reachable. No NAT. No proxies. Secure, scalable, and highly available when designed right.

9. TROUBLESHOOTING COMMANDS

```
# Check Pod IP
kubectl get pod -o wide
# Check Pod CIDR on node
kubectl get node -o jsonpath='{.items[*].spec.podCIDR}'
# Test connectivity between Pods
kubectl run -it --rm debug --image=busybox -- sh
# Inside pod: ping <pod-ip>
 ping 10.244.2.2
# Check CNI status
kubectl -n kube-system get pods | grep -i cni
```

10. COMMON ISSUES

- ⊗ Pod IPs not reachable → Check routing / CNI.
- ⊗ DNS not resolving → Check CoreDNS.
- ⊗ Pod IP conflicts → Check CIDR overlap.
- ⊗ Network Policies blocking → Review rules.
- ⊗ High latency → Check MTU / Network paths.

11. INTERVIEW TIPS

- Explain flat networking and no NAT between Pods.
- Describe how CNI assigns IPs to Pods.
- How does Pod-to-Pod communication work?
- What happens when two Pods are on different nodes?
- How do Network Policies secure Pod communication?

VERIQTA

43. CLUSTER NETWORKING

CNI • OVERLAY NETWORKS • UNDERLAY NETWORKS

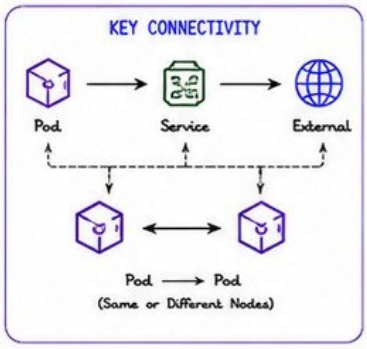
"Cluster networking connects all Pods, Nodes, and Services with speed, security, and reliability."

PRODUCTION NOTE

- ☑ Choose the right CNI for your needs.
- ☑ Understand overlay vs underlay.
- ☑ Design for performance and scale.
- ☑ Secure east-west and north-south traffic.
- ☑ Monitor network health continuously.

1. CLUSTER NETWORKING OVERVIEW

- Cluster networking connects all Pods across Nodes.
- Provided by CNI (Container Network Interface).
- Enables Pod-to-Pod, Pod-to-Service, and external communication.
- Must be scalable, secure, and highly available.



2. CNI (CONTAINER NETWORK INTERFACE)

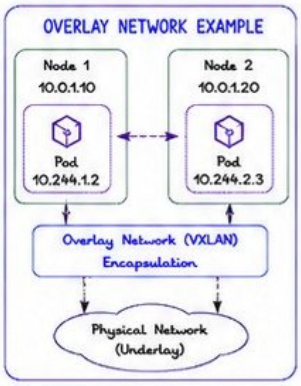
- CNI is the standard for configuring networking in Kubernetes.
- Installed on each node.
- Assigns IP, manages routing, and enforces policies.
- Plug-and-play network plugins.

POPULAR CNI PLUGINS

CNI	TYPE	FEATURES
Calico	Overlay/Route Based	High performance, Network Policies, BGP
Cilium	eBPF (Overlay)	High performance, L7 policies, Observability
Flannel	Overlay	Simple, lightweight, Easy to deploy
Weave Net	Overlay	Simple, fast setup
aws Amazon VPC CNI	Underlay (VPC Native)	AWS VPC integration, ENI per Pod

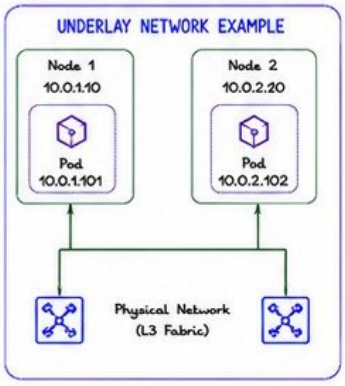
3. OVERLAY NETWORKS

- Virtual network built on top of the underlying network.
- Encapsulates packets to create a logical network.
- Works across nodes and even across subnets.

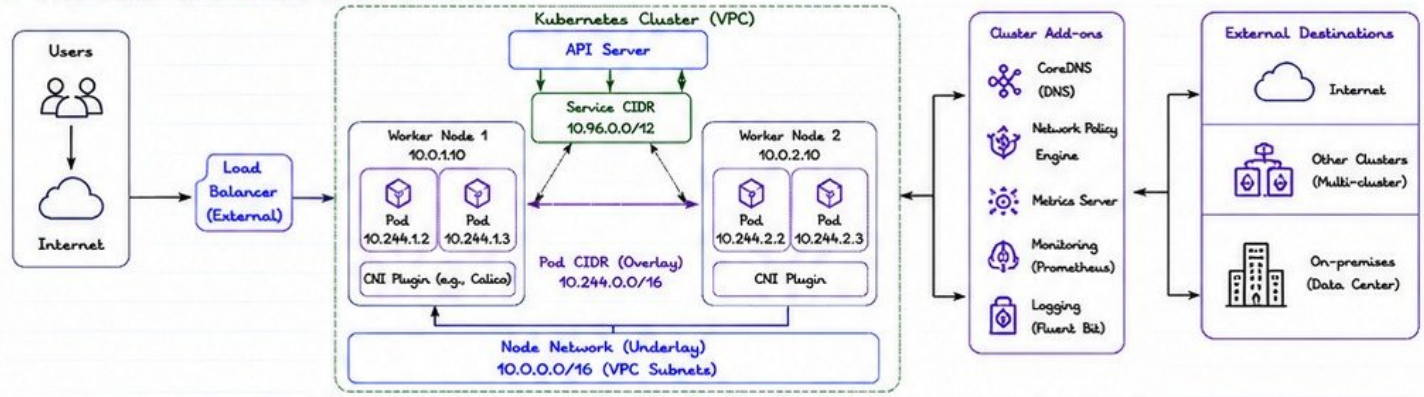


4. UNDERLAY NETWORKS

- Uses the physical network (L2/L3) provided by the infrastructure.
- No encapsulation overhead.
- Better performance and lower latency.
- Requires network fabric support.



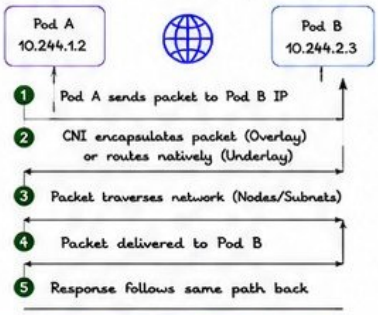
5. CLUSTER NETWORKING ARCHITECTURE (TYPICAL)



6. OVERLAY vs UNDERLAY COMPARISON

FEATURE	OVERLAY NETWORKS	UNDERLAY NETWORKS
Network Type	Virtual (Encapsulated)	Physical (Native)
Encapsulation	Yes (VXLAN, IPIP, etc.)	No
Performance	Lower (encapsulation overhead)	Higher (no encapsulation)
Complexity	Higher	Lower
Cross-Subnet	Yes	Depends on L2/L3 fabric
Use Cases	Multi-cloud, Multi-subnet, Cross-datacenter	High performance, Low latency environments
Examples	Flannel, Calico VXLAN, Weave Net	AWS VPC CNI, Cilium (Native) SR-IOV, BGP EVPN

7. NETWORK FLOW EXAMPLE (POD-TO-POD)



8. BEST PRACTICES

- ☑ Use a CNI that supports Network Policies.
- ☑ Allocate non-overlapping Pod CIDRs.
- ☑ Enable encryption for pod traffic (WireGuard/IPSec).
- ☑ Monitor network performance and latency.
- ☑ Use Calico or Cilium for advanced policy and observability.
- ☑ Design for scale: test with large pod counts.
- ☑ Document network architecture and CIDR plan.

KEY TAKEAWAY

Cluster networking is the backbone of Kubernetes. Choose the right CNI, understand overlay vs underlay, and design for performance, security, and scale.

9. USE CASES

- Microservices communication
- Multi-tenant isolation
- Hybrid and multi-cloud deployments
- High performance workloads
- Disaster recovery and DR sites

10. TROUBLESHOOTING QUICK CHECK

```
# Check CNI status on nodes
kubectl get pods -n kube-system
# Describe a pod network issue
kubectl describe pod <pod-name>
# Check pod IP and routes
ip addr
ip route
# Check connectivity between pods
kubectl exec -it <pod> -- ping <pod-ip>
```

11. INTERVIEW TIPS

- What is the role of CNI in Kubernetes?
- What is the difference between overlay and underlay networks?
- How does a Pod get its IP address?
- How does Network Policy work?
- How do you troubleshoot pod-to-pod connectivity?

VERIQTA

44. SERVICES

CLUSTERIP • NODEPORT • LOADBALANCER • EXTERNALNAME

"Kubernetes Services provide a stable endpoint for accessing your Pods."

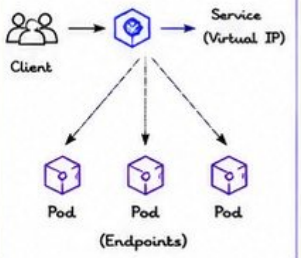
PRODUCTION NOTE

- ☑ Use the right Service type for the job.
- ☑ ClusterIP for internal communication.
- ☑ NodePort for testing or external nodes.
- ☑ LoadBalancer for production traffic.
- ☑ ExternalName for external DNS mapping.

1. SERVICES OVERVIEW

- Services provide a stable IP and DNS for accessing Pods.
- Pods are ephemeral; Services are stable.
- Services load balance traffic across healthy Pods.
- Kube-proxy implements Services on each node.
- Services work with Endpoints (Pod IPs) behind the scenes.

HOW IT WORKS

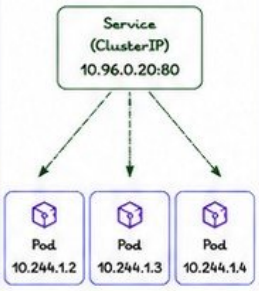


2. CLUSTERIP

- Default Service type.
- Exposes Service on a cluster-internal IP.
- Only reachable inside the cluster.
- Ideal for internal communication.

CLUSTERIP DIAGRAM

DNS: myapp.default.svc.cluster.local
IP: 10.96.0.20



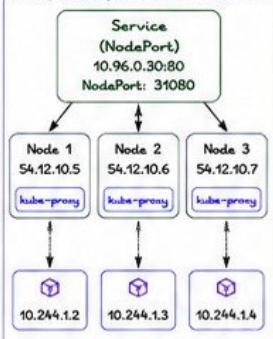
Reachable only inside the cluster.

3. NODEPORT

- Exposes Service on each Node's IP at a static port.
- Accessible from outside the cluster using <NodeIP>:<Port>.
- Port range: 30000 – 32767.
- Good for testing and small setups.

NODEPORT DIAGRAM

Access: http://<NodeIP>:<NodePort>
Example: http://54.12.10.5:31080

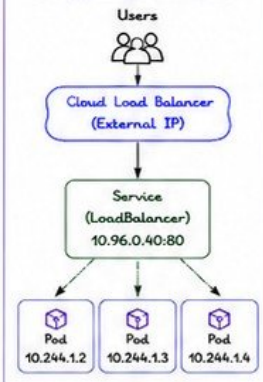


Traffic flows: Client → NodeIP:NodePort → kube-proxy → Pod

4. LOADBALANCER

- Provisions an external Load Balancer in cloud (AWS, GCP, Azure, etc).
- Provides a stable external IP for your application.
- Ideal for production workloads.

LOADBALANCER DIAGRAM



Traffic flows:
Internet → LB → Service → Pod

5. EXTERNALNAME

- Maps the Service to an external DNS name.
- No proxying or load balancing.
- Returns a CNAME record.
- Used to access external services.

EXTERNALNAME DIAGRAM

DNS: db.example.com

Kubernetes returns a CNAME record.
The client connects directly.

6. SERVICE TYPES COMPARISON

TYPE	CLUSTERIP	NODEPORT	LOADBALANCER	EXTERNALNAME
Exposure	Internal only	External via Node IP + Port	External via Cloud Load Balancer	External DNS name (CNAME)
IP	Cluster IP (10.96.x.x)	Node IP:NodePort	External IP (Cloud LB)	External DNS (no IP)
Reachable From	Inside Cluster	Inside + Outside (via Node IP)	Inside + Outside (via LB)	Inside + Outside
Use Case	Microservices communication	Testing, On-prem, Simple apps	Production, Internet-facing apps	Third-party services, DB, APIs
Port	Cluster IP only	NodePort (30000-32767)	Service Port (80/443 etc)	N/A
Load Balancing	Yes (kube-proxy)	Yes (kube-proxy)	Yes (Cloud LB + kube-proxy)	No
Example	Frontend → Backend	Access app on NodeIP:31080	Public website or API	Connect to Amazon RDS

7. YAML EXAMPLES

CLUSTERIP

```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-clusterip
spec:
  type: ClusterIP
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 8080
```

NODEPORT

```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-nodeport
spec:
  type: NodePort
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 31080
```

LOADBALANCER

```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-lb
spec:
  type: LoadBalancer
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 8080
```

EXTERNALNAME

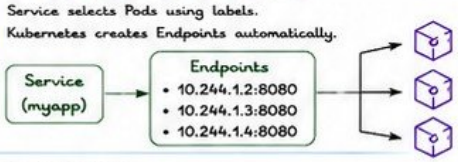
```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-external
spec:
  type: ExternalName
  externalName: db.example.com
  ports:
  - port: 80
    targetPort: 80
```

8. DNS NAMES

Service DNS Format:
<service>.<namespace>.svc.cluster.local
Examples:
myapp.default.svc.cluster.local
myapp.prod.svc.cluster.local

9. ENDPOINTS BEHIND SERVICE

Service selects Pods using labels.
Kubernetes creates Endpoints automatically.



10. BEST PRACTICES

- ☑ Use ClusterIP for internal microservices.
- ☑ Use NodePort only for testing or on-prem.
- ☑ Use LoadBalancer for production and internet traffic.
- ☑ Restrict access using Network Policies.
- ☑ Use meaningful labels for Service selectors.
- ☑ Monitor Service and Endpoint health.

11. COMMON ISSUES

- ⊗ Service has no Endpoints → Check Pod labels.
- ⊗ Wrong selector → No Pods matched.
- ⊗ Port mismatch → Connection refused.
- ⊗ NodePort blocked by firewall.
- ⊗ External Traffic not reaching Load Balancer.

12. INTERVIEW TIPS

- What is the difference between ClusterIP and NodePort?
- How does kube-proxy implement Services?
- When would you use ExternalName?
- How does LoadBalancer work internally?
- How do Services discover Pods?

VERIQTA

45. INGRESS

CONTROLLERS • TRAFFIC ROUTING • TLS TERMINATION

"Ingress exposes HTTP and HTTPS routes from outside the cluster to Services inside the cluster."

PRODUCTION NOTE

- ☑ Use Ingress for HTTP/HTTPS traffic.
- ☑ Use Ingress Controller (NGINX, ALB, etc.).
- ☑ Route based on host and/or path.
- ☑ Terminate TLS at the Ingress for security.
- ☑ Monitor, secure, and test regularly.

1. INGRESS OVERVIEW

- Ingress manages external HTTP/HTTPS access to Services.
- Defines rules for routing traffic.
- Works with an Ingress Controller.
- Supports host-based and path-based routing.
- Handles TLS termination.

2. INGRESS CONTROLLERS

- A controller watches Ingress resources.
- Configures the underlying proxy/load balancer.
- Popular controllers:
- NGINX Ingress Controller
- AWS ALB Ingress Controller
- Traefik, HAProxy, Istio Gateway

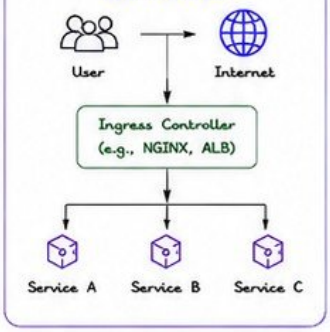
3. TRAFFIC ROUTING

- Route based on Host header.
- Route based on Path prefix.
- Supports exact path or prefix match.
- Routes traffic to backend Services.

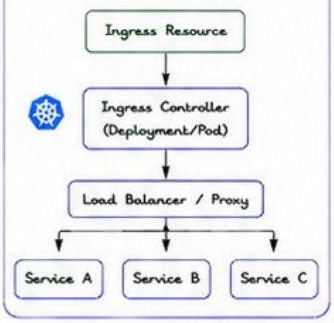
4. TLS TERMINATION

- Ingress terminates TLS at the edge.
- Decrypts traffic and routes internally as HTTP.
- Uses TLS certificates (Secrets).
- Improves security and offloads Pods.

HOW IT WORKS



INGRESS CONTROLLER ARCHITECTURE



ROUTING EXAMPLES

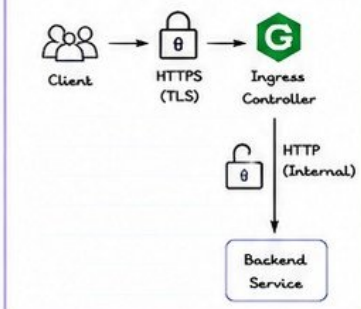
Host-based routing

app.example.com	Service A
api.example.com	Service B

Path-based routing (Host: app.example.com)

/	Service A (frontend)
/api	Service B (api)
/admin	Service C (admin)

TLS FLOW (TERMINATION AT INGRESS)



5. INGRESS RESOURCE EXAMPLE (NGINX)

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: veriqta-app-ingress
  namespace: default
  annotations:
    kubernetes.io/ingress.class: "nginx"
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  tls:
  - hosts:
    - app.example.com
    secretName: veriqta-tls
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80
```

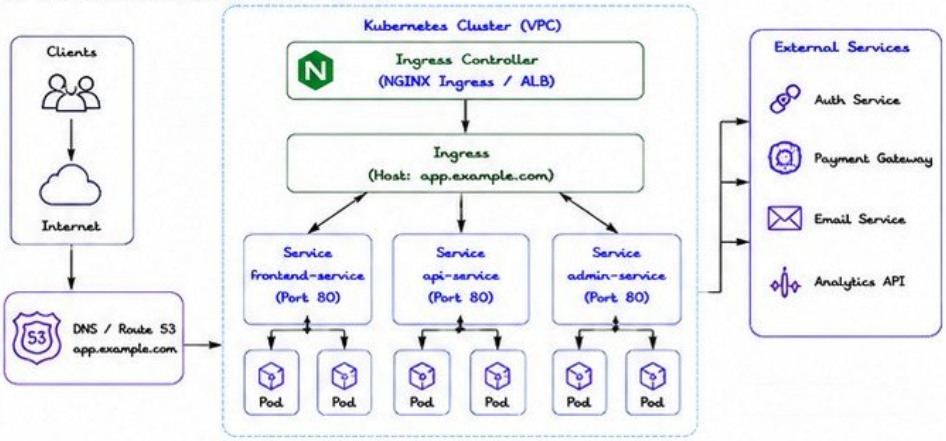
TLS configuration with secret

Host rule

Path "/" → frontend-service

Path "/api" → api-service

6. INGRESS ARCHITECTURE (HIGH LEVEL)



7. INGRESS ANNOTATIONS (COMMON)

ANNOTATION	PURPOSE
kubernetes.io/ingress.class	Defines the Ingress Controller to use.
nginx.ingress.kubernetes.io/ssl-redirect	Redirect HTTP to HTTPS.
nginx.ingress.kubernetes.io/rewrite-target	Rewrite URL before sending to backend.
nginx.ingress.kubernetes.io/limit-connections	Limit number of connections.
nginx.ingress.kubernetes.io/enable-cors	Enable CORS support.

8. TLS SECRET EXAMPLE

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: veriqta-tls
  type: kubernetes.io/tls
data:
  tls.crt: <base64-encoded-cert>
  tls.key: <base64-encoded-key>
```

9. BEST PRACTICES

- ☑ Use Ingress with a Controller (NGINX/ALB).
- ☑ Use TLS everywhere.
- ☑ Use host-based routing for clarity.
- ☑ Use path-based routing for microservices.
- ☑ Use annotations for advanced features.
- ☑ Monitor Ingress metrics and logs.
- ☑ Keep certificates updated (Let's Encrypt or ACM).

10. COMMON TROUBLESHOOTING

```
# Check Ingress
kubectl get ingress -A
# Describe Ingress
kubectl describe ingress veriqta-app-ingress
# Check Ingress Controller logs
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller
# Test host resolution
nslookup app.example.com
# Test from inside cluster
curl -I http://<service-ip>
```

11. INGRESS CHECKLIST

- ☑ DNS points to the Ingress Controller.
- ☑ Ingress resource created correctly.
- ☑ Ingress Controller is running.
- ☑ Routes are working (host/path).
- ☑ Services and Endpoints are healthy.
- ☑ TLS certificate is valid.

KEY TAKEAWAY

Ingress is the smart gateway into your cluster. With the right controller, routing rules, and TLS, you deliver secure, reliable, and scalable applications.

12. INTERVIEW TIPS

What is Ingress in Kubernetes?

- How does Ingress differ from Service?
- What is an Ingress Controller?
- How does TLS termination work in Ingress?
- How do you route traffic based on path and host?

VERIQTA

46. NETWORK POLICIES

ISOLATION • SECURITY • ZERO TRUST NETWORKING

"Network Policies control how Pods communicate."
Default is ALLOW ALL. Policies enforce Zero Trust."

PRODUCTION NOTE

- ☑ Default: allow all traffic.
- ☑ Apply NetworkPolicies per namespace.
- ☑ Deny by default, allow explicitly.
- ☑ Use labels for dynamic isolation.
- ☑ Monitor and test policies carefully.

1. NETWORK POLICIES OVERVIEW

- Operate at Layer 3/4 (IP, Port).
- Control traffic to and from Pods.
- Applied inside the cluster.
- Do not affect traffic to Nodes.
- Use labels to select Pods.
- Supported by CNI plugins (Calico, Cilium, Weave Net, etc.).
- Default behavior: ALLOW ALL.

2. ISOLATION PRINCIPLES

- Isolate workloads by namespace, app, or tier.
- Limit who can talk to whom.
- Control both Ingress and Egress.
- Reduce blast radius.
- Enforce least privilege.

3. SECURITY BENEFITS

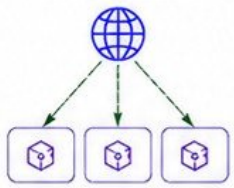
- Micro-segmentation.
- Prevent lateral movement.
- Protect sensitive services.
- Enforce Zero Trust model.
- Audit and compliance ready.
- Works with IAM, RBAC, and Secrets for strong security.

4. POLICY TYPES

- Ingress: controls incoming traffic to selected Pods.
- Egress: controls outgoing traffic from selected Pods.
- Both can use multiple rules.
- Combine for strong security.

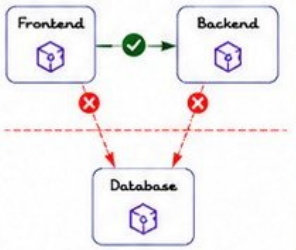
DEFAULT BEHAVIOR

No NetworkPolicy
=
All traffic allowed



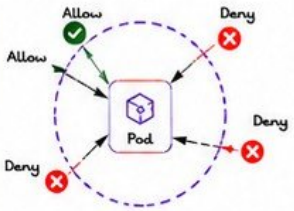
ISOLATION EXAMPLE

Frontend can talk to Backend, but not to Database directly.



ZERO TRUST MODEL

Verify every connection.
Trust nothing by default.



Everything denied unless allowed.

POLICY TYPES

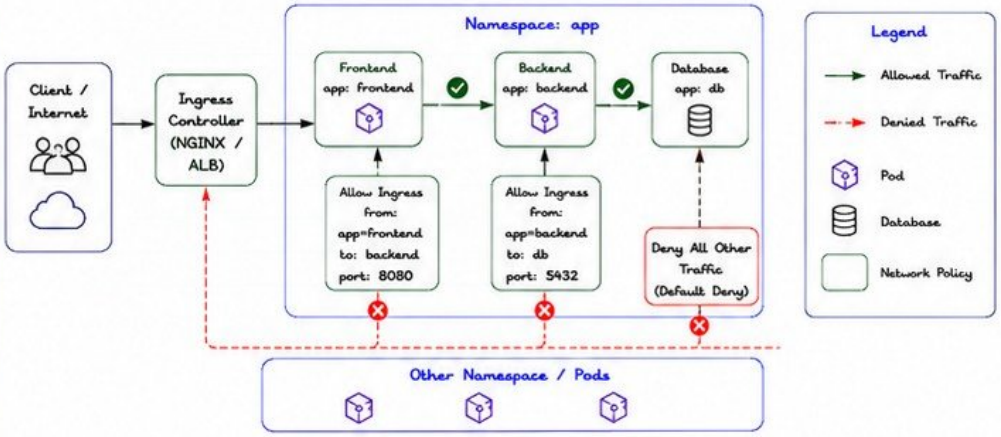
Can be applied together.

5. NETWORK POLICY EXAMPLE

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: app
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 8080
```

- `podSelector` → Selects backend Pods
- `policyTypes` → Applies to Ingress traffic
- `from` → Allow traffic FROM Pods with label app=frontend
- `port: 8080` → Allow TCP on port 8080

6. ARCHITECTURE WITH NETWORK POLICIES



7. COMMON POLICY PATTERNS

PATTERN	DESCRIPTION	USE CASES
Default Deny	Deny all by default, Allow specific.	All workloads
Allow by App	Allow traffic between specific apps.	Microservices
Namespace Isolation	Prevent traffic between namespaces.	Multi-tenant clusters
Egress Restriction	Allow only specific external destinations.	APIs, DB, SaaS
DB Protection	Allow only app tier to access DB.	Data security

8. ZERO TRUST NETWORKING

- ☑ Assume breach.
- ☑ Verify every connection.
- ☑ Least privilege access.
- ☑ Deny all by default.
- ☑ Encrypt traffic in transit.
- ☑ Monitor and log everything.
- ☑ Continuous validation.

8. BEST PRACTICES

- ☑ Always start with default deny.
- ☑ Apply policies per namespace.
- ☑ Use labels consistently.
- ☑ Test policies before enforcing.
- ☑ Monitor with Calico / Cilium tools.
- ☑ Document and version policies.
- ☑ Review and audit regularly.

KEY TAKEAWAY

Network Policies bring Zero Trust security to Kubernetes. Deny by default, allow explicitly, and isolate everything.

10. TROUBLESHOOTING TIPS

```
# List network policies
kubectl get networkpolicies -A
# Describe a policy
kubectl describe networkpolicy <name> -n <ns>
# Check CNI plugin status
kubectl -n kube-system get pods | grep calico
# Test connectivity between pods
kubectl run test --rm -it --image=busybox -- /bin/sh
# Inside pod: nc -vz <ip> <port>
```

11. COMMON ISSUES

- ⊗ Policy not applied → Check namespace.
- ⊗ Labels mismatch → Pods not selected.
- ⊗ Wrong ports/protocols → Traffic blocked.
- ⊗ CNI plugin not supporting policies.
- ⊗ Remember: Default is ALLOW ALL.

12. INTERVIEW TIPS

- What are Network Policies in Kubernetes?
- What is the default behavior?
- Difference between Ingress and Egress?
- How do you isolate namespaces?
- How does Network Policy enforce Zero Trust?

VERIQTA

47. SERVICE DISCOVERY

COREDNS • DNS RECORDS • INTERNAL COMMUNICATION

"Service Discovery lets Pods find Services without knowing IP addresses. CoreDNS is the brain."

PRODUCTION NOTE

- ☑ CoreDNS runs in every cluster.
- ☑ DNS is the foundation for microservices.
- ☑ Use stable Service names, not IPs.
- ☑ Keep DNS records clean and organized.
- ☑ Monitor DNS latency and errors.

1. SERVICE DISCOVERY OVERVIEW

- Allows Pods to find and connect to Services automatically.
- Uses DNS names instead of IPs.
- Built on CoreDNS.
- Handles internal and external name resolution.
- Essential for microservices communication.

2. COREDNS OVERVIEW

- CoreDNS is the DNS server in Kubernetes.
- Runs as a Deployment in kube-system namespace.
- Handles cluster DNS, service discovery, and forwarding.
- Highly extensible with plugins.
- Default DNS for all Pods.

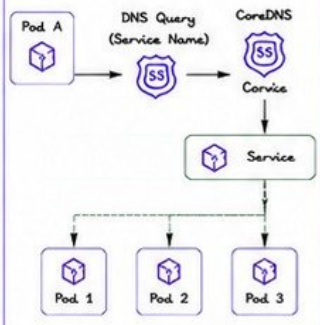
3. DNS RECORDS IN KUBERNETES

- CoreDNS automatically creates DNS records for Services.
- Names follow a predictable format.
- Records are updated in real-time.
- TTL is low for fast updates.

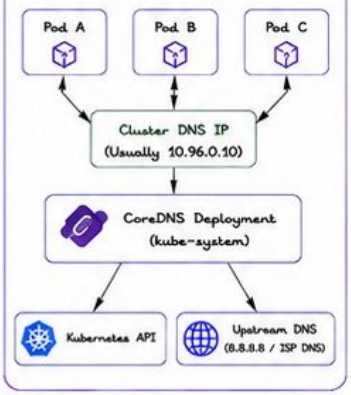
4. INTERNAL COMMUNICATION

- Pods communicate using Service names.
- No need to hardcode IP addresses.
- Works across namespaces.
- Load balancing handled automatically.
- Resilient to Pod IP changes.

HOW IT WORKS



COREDNS ARCHITECTURE



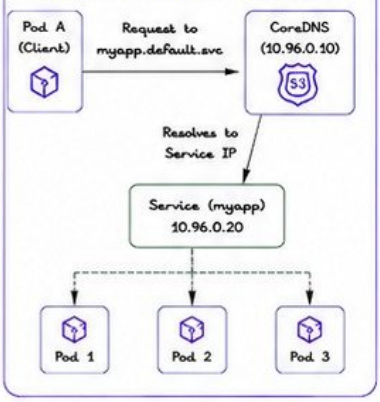
DNS RECORDS FORMAT

<service>.<namespace>.svc.cluster.local

Example:
myapp.default.svc.cluster.local

RECORD TYPE	PURPOSE
A Record	Resolves Service name to ClusterIP
SRV Record	Provides port and protocol details
PTR Record	Reverse DNS lookup
CNAME Record	Aliases and external name mapping

INTERNAL COMMUNICATION FLOW



5. DNS RECORDS EXAMPLES (KUBERNETES)

RESOURCE	DNS NAME	RECORD TYPE	RESOLVES TO
Service (myapp)	myapp.default.svc.cluster.local	A	10.96.0.20
Service (myapp)	myapp.default.svc.cluster.local	SRV	10 100 8080 myapp.default.svc.cluster.local.
Headless Service	db-headless.default.svc.cluster.local	A	Pod IPs
Service (other-ns)	api.otherns.svc.cluster.local	A	10.96.1.30
Kubernetes API	kubernetes.default.svc.cluster.local	A	10.96.0.1

6. COREDNS CONFIGURATION (SIMPLIFIED)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: coredns
  namespace: kube-system
data:
  Corefile: |
    .:53 {
      errors
      health
      kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        ttl 30
        fallthrough in-addr.arpa ip6.arpa
      }
      prometheus :9153
      forward . 8.8.8.8 1.1.1.1
      cache 30
      loop
      reload
      loadbalance
    }
```

7. SERVICE DISCOVERY FLOW

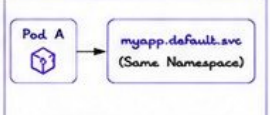
1. Pod sends DNS query for Service name.
2. Query is sent to CoreDNS.
3. CoreDNS checks cache and Kubernetes API.
4. CoreDNS returns Service ClusterIP.
5. Pod connects to Service IP.
6. kube-proxy routes traffic to Pods.

KEY BENEFITS

- ☑ Decouples Services from IP addresses.
- ☑ Automatic load balancing.
- ☑ Works across namespaces.
- ☑ High availability and fast updates.
- ☑ Foundation of cloud-native architecture.

8. NAMESPACE SCOPES

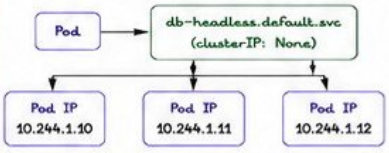
SAME NAMESPACE



CROSS NAMESPACE

9. HEADLESS SERVICES (POD DISCOVERY)

- Set clusterIP: None
- Returns Pod IPs directly.
- Used for databases, stateful sets, custom discovery.



10. BEST PRACTICES

- ☑ Use Service names, never Pod IPs.
- ☑ Keep Services and DNS names simple.
- ☑ Use namespaces to organize Services.
- ☑ Monitor CoreDNS performance.
- ☑ Set appropriate TTL for DNS records.
- ☑ Use headless Services only when needed.
- ☑ Enable DNS caching for performance.

11. COMMON TROUBLESHOOTING

```
# Test DNS from Pod
kubectl exec -it <pod> -- nslookup myapp.default.svc.cluster.local
# Check CoreDNS logs
kubectl logs -n kube-system deployment/coredns
# Test connectivity
kubectl exec -it <pod> -- curl http://myapp.default.svc
# Describe Service
kubectl describe svc myapp
# Check DNS config
kubectl get configmap -n kube-system coredns -o yaml
```

12. COMMON ISSUES

- ⊗ DNS query timeout → Check CoreDNS logs.
- ⊗ Service not found → Check Service name/namespace.
- ⊗ Pods can't resolve names → Check DNS config.
- ⊗ Intermittent failures → Increase cache/TTL.
- ⊗ High latency → Check upstream DNS and network.

13. INTERVIEW TIPS

- What is Service Discovery in Kubernetes?
- How does CoreDNS work?
- What is the DNS name format for a Service?
- Difference between ClusterIP and Headless Service?
- How does internal communication work?

VERIQTA

48. CONTAINER NETWORK TROUBLESHOOTING

DNS FAILURES • SERVICE FAILURES • CNI FAILURES • PACKET FLOW

"Great engineers troubleshoot faster by understanding how networking really works inside the cluster."

PRODUCTION NOTE

- ☑ Understand the full network path.
- ☑ Check DnS, Services, Endpoints, and Pods.
- ☑ Validate CNI and node networking.
- ☑ Always test from multiple points.
- ☑ Document findings and fix the root cause.

1. DNS FAILURES

- Pods cannot resolve Service names.
- CoreDNS is down or misconfigured.
- DNS records missing or wrong.
- Search domains or ndots issue.
- NetworkPolicy blocking DNS.

2. SERVICE FAILURES

- Service IP not reachable.
- Endpoints missing or not ready.
- Selector mismatch.
- NetworkPolicy blocking traffic.
- kube-proxy not working.

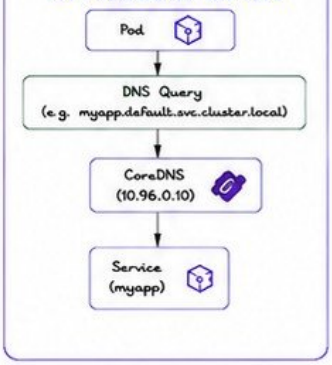
3. CNI FAILURES

- Pods cannot get IP addresses.
- CNI plugin not running.
- Pod to Pod communication broken.
- Network overlay issues.
- Node network routing problems.

4. PACKET FLOW DIAGNOSTIC

- Understand the full path.
- Verify hop-by-hop connectivity.
- Identify where traffic breaks.
- Use tools: ping, curl, traceroute, tcpdump, conntrack.

DNS TROUBLESHOOTING FLOW

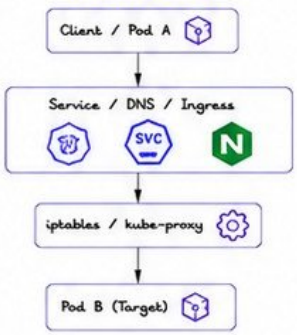


SERVICE TROUBLESHOOTING FLOW

CNI TROUBLESHOOTING CHECKLIST

- ☑ Check CNI pods on each node
  `kubectl -n kube-system get pods`
- ☑ Check CNI logs
  `kubectl -n kube-system logs <cni-pod>`
- ☑ Verify Pod IP address
  `kubectl get pod -o wide`
- ☑ Check CNI config on nodes
  `/etc/cni/net.d/`
- ☑ Verify node routing
  `ip route | grep <pod-cidr>`

PACKET FLOW PATH



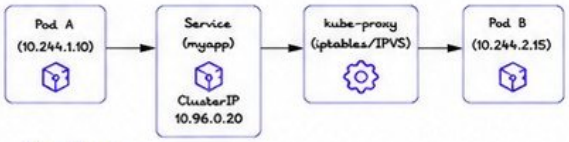
5. COMMON TROUBLESHOOTING COMMANDS

AREA	COMMAND	PURPOSE
DNS	kubectl exec -it <pod> -- nslookup <svc>	Test DNS resolution
DNS	kubectl -n kube-system logs -l k8s-app=kube-dns	Check CoreDNS logs
Service	kubectl get svc	List Services
Service	kubectl get endpoints <svc> -o wide	Check Endpoints
Pod	kubectl get pods -o wide	Check Pod IPs and nodes
CNI	kubectl -n kube-system get pods	Check CNI pods
CNI	ip addr	Check node interfaces
NetworkPolicy	kubectl get networkpolicies -A	List NetworkPolicies
Traffic	kubectl exec -it <pod> -- curl <svc>:<port>	Test connectivity
Traffic	tcpdump -i any -n host <ip>	Capture packets
Conntrack	conntrack -L | grep <ip>	Check connection tracking

6. END-TO-END TROUBLESHOOTING WORKFLOW

1. Identify the problem (From alert or user report)
2. Reproduce the issue (Pod to Service / External)
3. Check DNS resolution (nslookup / dig)
4. Check Service & Endpoints (kubectl get svc/endpoints)
5. Check Pod health (Readiness / Logs)
6. Check NetworkPolicies (kubectl get networkpolicies)
7. Check CNI and node networking (IPs, routes, CNI logs)
8. Capture packets if needed (tcpdump / Wireshark)
9. Identify root cause (Fix the underlying issue)
10. Validate and monitor (Confirm resolution)

7. PACKET FLOW EXAMPLE (POD A TO POD B)



Flow Steps:

1. Pod A sends packet to Service IP (10.96.0.20)
2. kube-proxy DNATs to one of the Endpoints (Pod B)
3. Packet routed through CNI overlay/underlay
4. Packet delivered to Pod B
8. Response follows reverse path

Key Tools

8. COMMON ROOT CAUSES

- ⊗ CoreDNS pods down or misconfigured
- ⊗ Wrong Service selector (no matching pods)
- ⊗ Endpoints not ready
- ⊗ NetworkPolicy blocking traffic
- ⊗ CNI plugin crash or misconfiguration
- ⊗ Node routing issues
- ⊗ iptables/kube-proxy rules missing
- ⊗ Firewall or cloud security group blocking

PRINCIPLE

Always validate each layer one by one. Don't assume.
Follow the path.
Find the break.

9. BEST PRACTICES

- ☑ Use consistent naming and labels.
- ☑ Monitor CoreDNS, CNI, and kube-proxy.
- ☑ Apply NetworkPolicies with care.
- ☑ Test connectivity in CI/CD pipelines.
- ☑ Document network architecture.
- ☑ Use observability tools (Prometheus, Grafana).
- ☑ Keep CNI, kube-proxy, and CoreDNS updated.

KEY TAKEAWAY

Most container network issues are caused by DNS, Services, Endpoints, NetworkPolicies, or CNI problems. Follow the flow, validate each layer, and fix the root cause.

10. QUICK CHECKLIST

- ☑ DNS works from Pod?
- ☑ Service has healthy Endpoints?
- ☑ Pod is Running and Ready?
- ☑ NetworkPolicy allows traffic?
- ☑ CNI pods are Running?
- ☑ Node routes are correct?
- ☑ Packets flow without drops?

11. COMMON ISSUES & FIX

ISSUE	FIX
DNS lookup fails	Check CoreDNS pods and logs
Service unreachable	Check Endpoints and selectors
Pod IP not assigned	Check CNI logs and node status
Pod to Pod unreachable	Check NetworkPolicies and CNI
Intermittent failures	Check conntrack, iptables, routes

12. TROUBLESHOOTING MINDSET

13. INTERVIEW TIPS

- What is the role of CoreDNS?
- How does kube-proxy work?
- How does a Service route traffic?
- How would you troubleshoot a DNS failure?
- How do NetworkPolicies affect traffic?
- How does CNI enable Pod networking?

VERIQTA

49. SERVICE MESH FUNDAMENTALS

ISTIO • LINKERD • MTLS • TRAFFIC MANAGEMENT

"Service Mesh gives you visibility, security, and control for service-to-service communication."

PRODUCTION NOTE

- ☑ Service Mesh is for internal traffic.
- ☑ Start simple, add policies gradually.
- ☑ mTLS is the foundation of security.
- ☑ Observe before you optimize.
- ☑ Always test policies in staging first.

1. SERVICE MESH OVERVIEW

- Dedicated infrastructure layer for service-to-service traffic.
- Provides security, observability, and traffic control.
- Works across languages, frameworks, and platforms.
- Data plane handles traffic.
- Control plane manages policy.

2. POPULAR SERVICE MESHES

- Istio – feature rich, enterprise ready, large ecosystem.
- Linkerd – lightweight, simple, fast, and easy to operate.
- Both use sidecar proxies.
- Both support mTLS, traffic management, observability.

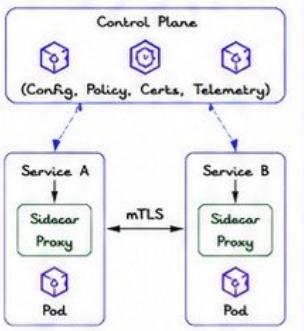
3. MTLS (MUTUAL TLS)

- Encrypts all service-to-service communication.
- Both sides present certificates.
- Prevents eavesdropping and impersonation.
- Automatic key rotation.

4. TRAFFIC MANAGEMENT CAPABILITIES

- Traffic routing (A/B, Canary, Blue/Green)
- Retries and timeouts
- Circuit breaking
- Fault injection
- Rate limiting
- Traffic mirroring
- Header based routing

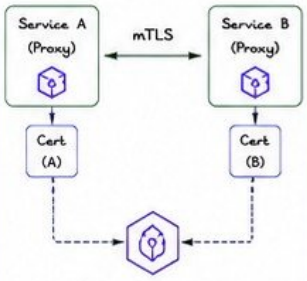
SERVICE MESH ARCHITECTURE



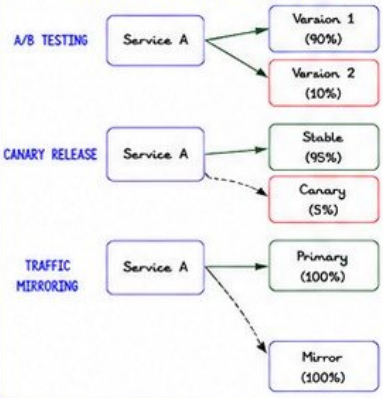
ISTIO vs LINKERD

Istio	Linkerd
Feature rich	Lightweight
Advanced traffic management	Easy to install
Large ecosystem	Low resource usage
Steeper learning curve	Great docs & UX
Control plane: Istiod	Control plane: linkerd
Data plane: Envoy	Data plane: Envoy

MTLS FLOW



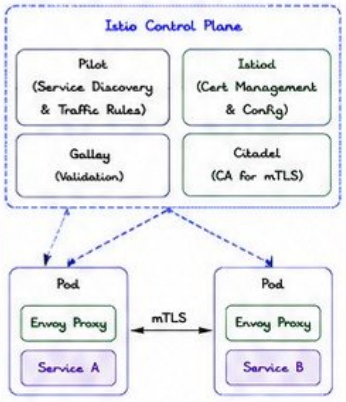
TRAFFIC MANAGEMENT EXAMPLES



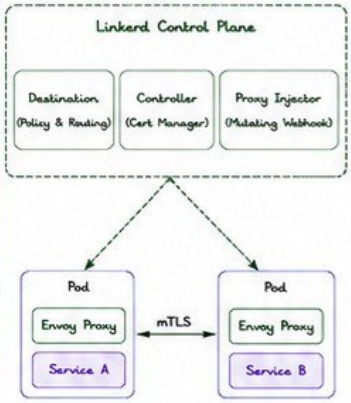
5. SERVICE MESH COMPONENTS

COMPONENT	DESCRIPTION
Sidecar Proxy (Envoy)	Runs alongside every service (Pod). Handles traffic, security, and telemetry.
Control Plane	Manages configuration, policies, certificates, and telemetry.
Certificate Authority	Issues and rotates certificates for mTLS.
Observability	Collects metrics, logs, and traces from the mesh.
Policies	Define traffic rules, security, and access control.

6. ISTIO ARCHITECTURE



7. LINKERD ARCHITECTURE



8. MTLS STATUS CHECK (ISTIO)

```
# Check MTLS authentication policies
istioctl authn tls-check pod/<pod-name> -n <ns>

# View workloads with MTLS enabled
istioctl authn policies -n <ns>

# Check connection security in logs
kubectl logs <pod> -n <ns> -c istio-proxy | grep "TLS"
```

9. COMMON ISTIO COMMANDS

```
# Install Istio (demo profile)
Istioctl install --set profile=demo-y
# Label namespace for injection
kubectl label namespace <ns> istio-injection=enabled
# Check mesh
istioctl check
# View services in mesh
istioctl proxy-status
# View traffic
istioctl proxy-config routes <pod> -n <ns>
```

10. COMMON LINKERD COMMANDS

```
# Install Linkerd
curl -sL https://run.linkerd.io/install | sh
# Check cluster
linkerd check --pre
# Get proxy status
linkerd -n <ns> stat pods
# View routes for a service
linkerd -n <ns> routes deploy/<service>
```

11. SERVICE MESH BENEFITS

- ☑ Secure by default (mTLS)
- ☑ Fine-grained traffic control
- ☑ Rich observability
- ☑ Faster incident detection
- ☑ Policy enforcement
- ☑ Works across environments

KEY TAKEAWAY

Service Mesh is the infrastructure for modern cloud-native applications. Start with security and observability, then use traffic management to deliver value.

12. BEST PRACTICES

- ☑ Enable mTLS from day one.
- ☑ Start with observability, then policies.
- ☑ Use namespaces to scope policies.
- ☑ Test traffic rules in staging.
- ☑ Monitor resource usage.

13. COMMON ISSUES

- ⊗ Sidecar injection not enabled.
- ⊗ Ports not included in proxy config.
- ⊗ MTLS handshake failures.
- ⊗ Destination not found (service discovery).
- ⊗ Policies blocking traffic.

14. INTERVIEW TIPS

- What is a Service Mesh?
- Why is mTLS important?
- Difference between Istio and Linkerd?
- How does traffic routing work?
- How do you debug mesh issues?

VERIQTA

50. PRODUCTION KUBERNETES NETWORKING

MULTI-CLUSTER • MULTI-REGION • HIGH AVAILABILITY • ENTERPRISE

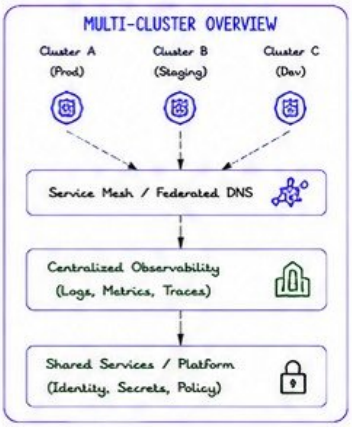
"Design networks that scale across clusters, regions, and teams.
Secure by default. Automate always."

PRODUCTION NOTE

- ☑ Design for failure, not for demos.
- ☑ Automate networking with GitOps.
- ☑ Use defense in depth.
- ☑ Observe everything.
- ☑ Document standards and runbooks.

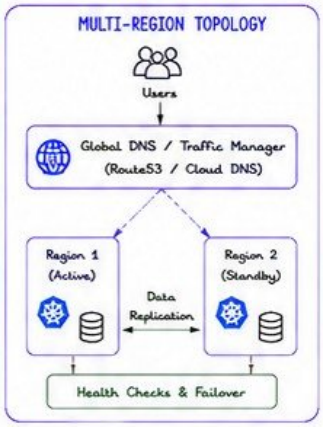
1. MULTI-CLUSTER ARCHITECTURE

- Multiple clusters for isolation and scale.
- Cluster per team / environment.
- Centralized networking and security.
- GitOps for consistent configuration.
- Federation or service mesh for service discovery.



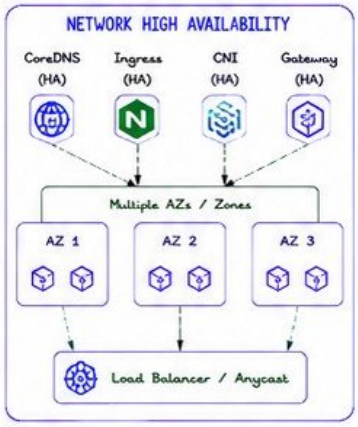
2. MULTI-REGION ARCHITECTURE

- Deploy clusters in multiple regions.
- Global DNS for traffic management.
- Replicate data where needed.
- Use region-local services.
- Plan for DR and failover.



3. HIGH AVAILABILITY NETWORKING

- No single points of failure.
- HA CNI, CoreDNS, Ingress, Gateways.
- Redundant NAT, IGW, Load Balancers.
- Pod disruption budgets for network pods.
- Continuous health monitoring.



4. ENTERPRISE RECOMMENDATIONS

- Standardize CIDR and IP management.
- Enforce NetworkPolicies everywhere.
- Use service mesh for security and observability.
- Centralize logs, metrics, and alerts.
- Automate with GitOps and IaC.

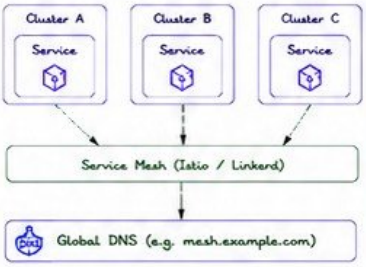
5. GLOBAL TRAFFIC MANAGEMENT

1. Primary region (Active)
2. Secondary region (Failover)
3. Edge locations / CDN

- ☑ DNS latency-based routing
- ☑ Health checks
- ☑ Automatic failover

6. CROSS-CLUSTER SERVICE COMMUNICATION

- Options: Service Mesh, DNS Federation, API Gateway, or VPC Peering.
- Prefer service mesh for mTLS and policies.
- Use Global DNS for stable service names.



7. IP ADDRESS MANAGEMENT

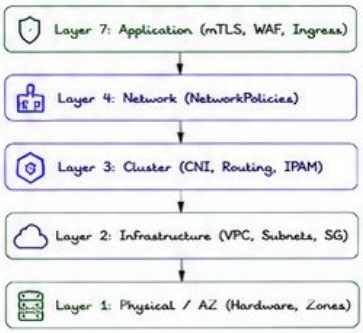
- Plan Pod CIDR per cluster.
- Avoid overlaps across clusters.
- Use IPAM (e.g. Infoblox, AWS VPC IPAM).
- Track and monitor IP usage.

IP PLAN EXAMPLE

CLUSTER	POD CIDR	SERVICE CIDR
prod-cluster-1	10.10.0.0/16	10.100.0.0/16
prod-cluster-2	10.11.0.0/16	10.101.0.0/16
staging-cluster	10.20.0.0/16	10.200.0.0/16
dev-cluster	10.30.0.0/16	10.300.0.0/16

Ensure no overlap across all networks.

8. NETWORK SECURITY LAYERS



9. CNI OPTIONS FOR PRODUCTION

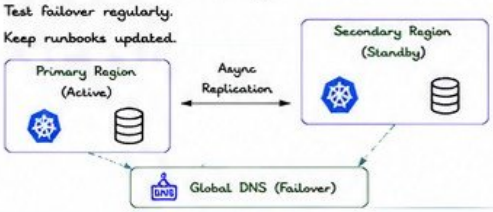
CNI	TYPE	FEATURES	BEST FOR
Calico	Underlay	NetworkPolicies, BGP, IPv4/IPv6, IPSec	Security, Performance
Cilium	eBPF	L7 Policies, Observability, High Performance	Modern, Security
AWS VPC CNI	Cloud	Native AWS, ENI, Security Groups	AWS EKS
Azure CNI	Cloud	Azure VNet Integration	Azure AKS
Google VPC CNI	Cloud	Native GCP Integration	GKE

10. OBSERVABILITY FOR NETWORKING

- Monitor DNS, Services, Ingress, CNI, Nodes.
- Capture metrics, logs, and traces.
- Use these tools:

11. DISASTER RECOVERY NETWORKING

- Multi-region failover with DNS.
- Backup CoreDNS, certs, and configs.
- Test failover regularly.
- Keep runbooks updated.



12. PRODUCTION NETWORKING CHECKLIST

- ☑ CIDR plan documented and non-overlapping
- ☑ CNI chosen, tested, and monitored
- ☑ CoreDNS HA and monitored
- ☑ Ingress/Load Balancers are HA
- ☑ NetworkPolicies enforced and tested
- ☑ Service Mesh deployed (if required)
- ☑ Observability covering full network stack
- ☑ Backup and DR plans tested
- ☑ Runbooks created and team trained

13. USEFUL COMMANDS

```
# DNS lookup from a pod
kubectl exec -it <pod> -- nslookup myapp.my-namespace.svc.cluster.local

# Check services
kubectl get svc -A

# Check network policies
kubectl get networkpolicies -A

# Capture packets (Cilium)
cilium monitor --type drop

# Trace network path
toolbox trace myapp.my-namespace.svc.cluster.local
```

14. COMMON FAILURE SIGNS

- ⊗ Pod cannot resolve service name
- ⊗ Service unreachable from other pods
- ⊗ Intermittent timeouts
- ⊗ High latency across regions
- ⊗ CNI pods crashing or NotReady
- ⊗ DNS slow or failing
- ⊗ NetworkPolicies blocking traffic

KEY TAKEAWAY

Production Kubernetes networking is about consistency, security, and resilience. Design for scale, automate everything, observe continuously, and prepare for failure before it happens.

VERIQTA

51. MONITORING NETWORK HEALTH

LATENCY • THROUGHPUT • PACKET LOSS • JITTER • ERROR RATES

"You can't fix what you don't measure."
Monitor continuously, alert early, and act before users feel it."

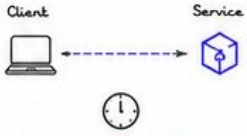
PRODUCTION NOTE

- ☑ Baseline everything.
- ☑ Monitor from multiple angles.
- ☑ Alert on trends, not just thresholds.
- ☑ Correlate metrics with real user impact.
- ☑ Document SLOs and response playbooks.

1. LATENCY

- Time taken for a packet to travel from source to destination.
- Measured in milliseconds (ms).
- High latency = slow applications.

EXAMPLE



Response Time: 120 ms

GOOD	< 100ms
WARNING	100 – 250ms
CRITICAL	> 250ms

2. THROUGHPUT

- Amount of data successfully transferred over the network per unit time.
- Measured in Mbps / Gbps.
- Shows the capacity and utilization of the network.

EXAMPLE

Throughput: 850 Mbps

GOOD	> 500 Mbps
WARNING	200 – 500 Mbps
CRITICAL	< 200 Mbps

3. PACKET LOSS

- Percentage of packets lost during transmission.
- Causes retries, timeouts, and poor performance.
- Measured in %.

EXAMPLE

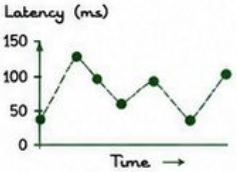
Sent: 1000 packets
Received: 980 packets
Packet Loss: 2%

GOOD	< 1%
WARNING	1% – 3%
CRITICAL	> 3%

4. JITTER

- Variation in latency between packets.
- High jitter affects voice, video, and real-time apps.
- Measured in milliseconds (ms).

EXAMPLE



Jitter: 25 ms

GOOD	< 20ms
WARNING	20 – 50ms
CRITICAL	> 50ms

5. ERROR RATES

- Percentage of failed requests / connections.
- Indicates application or network issues.
- Measured in %.

EXAMPLE

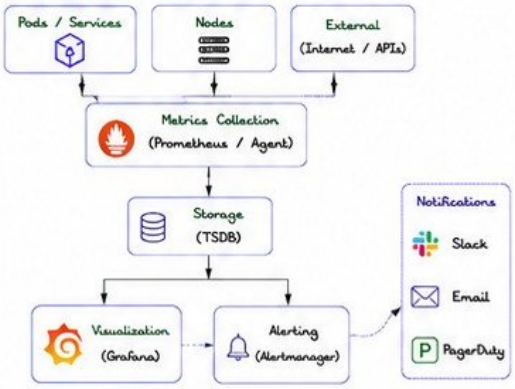
Total Requests: 10,000
Failed Requests: 120
Error Rate: 1.2%

GOOD	< 1%
WARNING	1% – 3%
CRITICAL	> 3%

6. WHAT TO MONITOR (NETWORK LAYERS)

- **L3 / Network** (Latency, Packet Loss, Throughput)
- **L4 / Transport** (TCP Retransmissions, Connection Errors)
- **L7 / Application** (Response Time, Error Rate, DNS Time)
- **Infrastructure** (Node, CNI, Links, Devices)
- **External Dependencies** (APIs, Databases, Internet)

7. MONITORING ARCHITECTURE



8. TOOLS FOR NETWORK MONITORING

- **Prometheus + Grafana** — Metrics, dashboards, alerting
- **kube-state-metrics** — Kubernetes network metrics
- **Cilium / Hubble** — eBPF based network visibility
- **Calico** — Network policy and flow logs
- **Ping / MTR / traceroute** — Connectivity and path analysis
- **Wireshark / tcpdump** — Packet capture and deep analysis
- **Blackbox Exporter** — External endpoint monitoring

9. SAMPLE PROMETHEUS METRICS

node_network_receive_bytes_total	Bytes received
node_network_transmit_bytes_total	Bytes transmitted
node_network_receive_errs_total	Receive errors
node_network_transmit_errs_total	Transmit errors
node_network_receive_drop_total	Dropped packets
node_network_transmit_drop_total	Dropped packets
http_request_duration_seconds_bucket	Request latency
up	Target up/down

10. SAMPLE ALERT RULES

High Latency	avg latency > 250ms for 5m
High Packet Loss	packet loss > 3% for 5m
High Error Rate	error rate > 3% for 5m
High Jitter	jitter > 50ms for 5m
Low Throughput	throughput < 200Mbps for 10m

11. BEST PRACTICES

- ☑ Establish baselines for all metrics.
- ☑ Use multiple monitoring points.
- ☑ Monitor both internal and external paths.
- ☑ Alert on rate of change, not only thresholds.
- ☑ Correlate metrics with user experience.
- ☑ Automate incident response where possible.
- ☑ Review dashboards regularly.

15. QUIFUL COMMANDS

- ☑ ping -c 5 <host>
- ☑ Throughput
- ☑ iperf3 -c <server>
- ☑ curl -w "time_total: %{time_total}s\n" <url>
- ☑ dig <domain> @<dns-server>
- ☑ kubectl exec -it <pod> -- nc -zv <host> <port>
- ☑ kubectl get --raw /metrics | grep network

13. INTERVIEW TIPS

- How do you measure network latency?
- What causes packet loss in Kubernetes?
- How do you troubleshoot high latency?
- How do you detect network saturation?
- What tools do you use for network monitoring?

KEY TAKEAWAY

Network health is the foundation of application reliability. Monitor latency, throughput, packet loss, jitter, and error rates continuously to deliver fast, reliable, and resilient systems.

14. COMMON ISSUES

- ⊗ Monitoring from only one location.
- ⊗ Ignoring packet loss and jitter.
- ⊗ Alert fatigue (too many alerts).
- ⊗ No baseline or SLO defined.
- ⊗ No correlation between metrics.

15. USEFUL COMMANDS

- ping -c 5 <host>
- mtr <host>
- iperf3 -c <server>
- curl -w "time_total: %{time_total}s\n" <url>
- dig <domain> @<dns-server>
- kubectl exec -it <pod> -- nc -zv <host> <port>
- kubectl get --raw /metrics | grep network

VERIQTA

52. OBSERVABILITY FOR NETWORKS

PROMETHEUS • GRAFANA • SNMP • FLOW MONITORING
LOGS • METRICS • TRACES

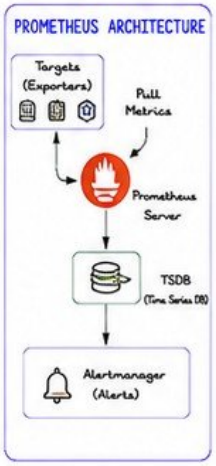
"You can't improve what you don't observe."
Full visibility. Real insights. Fast response."

PRODUCTION NOTE

- ☑ Observe everything, not just failures.
- ☑ Correlate metrics, logs, and traces.
- ☑ Alert early before users complain.
- ☑ Baselines first, then set alerts.
- ☑ Document dashboards and runbooks.

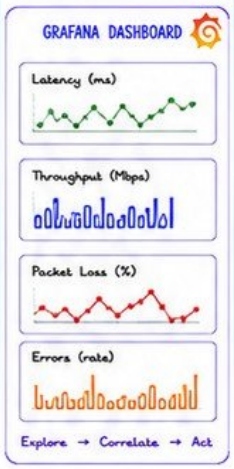
1. PROMETHEUS

- Open-source monitoring system.
- Pulls metrics from targets.
- Multi-dimensional time series data.
- Powerful query (PromQL).



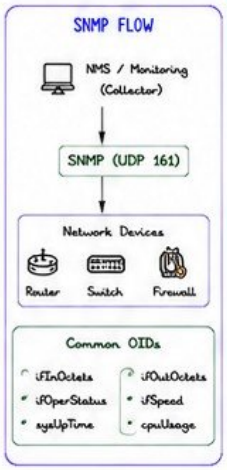
2. GRAFANA

- Visualization and dashboards.
- Connects to Prometheus and other data sources.
- Alerting and annotations.
- Share dashboards with teams.



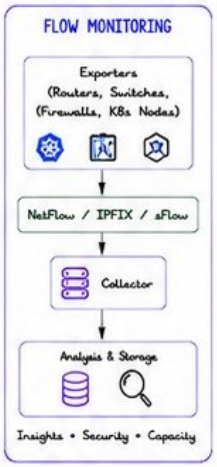
3. SNMP MONITORING

- Simple Network Management Protocol.
- Monitor devices (routers, switches, firewalls).
- Metrics via SNMP OIDs.
- Uses SNMP Walk / Get.



4. FLOW MONITORING

- NetFlow, IPFIX, sFlow.
- Understand who is talking to whom.
- Top talkers, protocols, applications.
- Bandwidth and sessions.

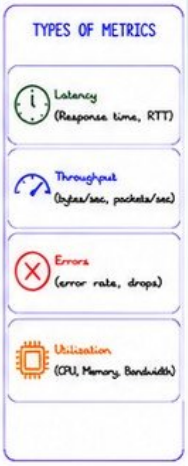


5. LOGS

- Capture events and activities.
- Essential for troubleshooting and audits.
- Centralize and search.
- Correlate with metrics.

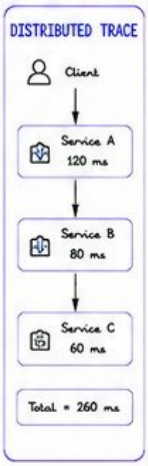
6. METRICS

- Numeric measurements over time.
- Trends, rates, percentages.
- Use labels for context.
- Alert on thresholds.

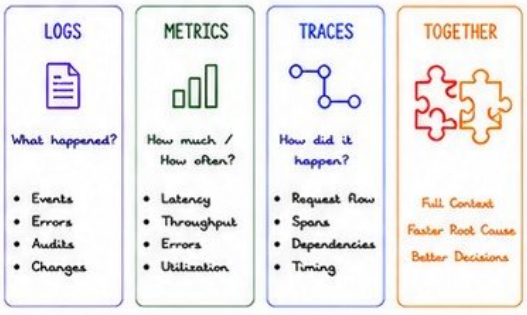


7. TRACES

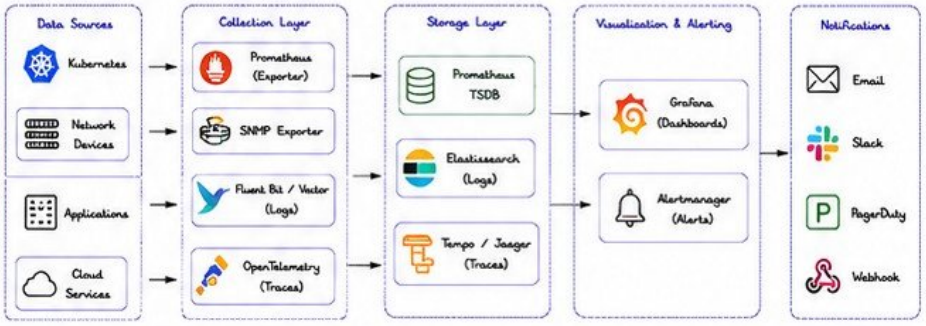
- Follow a request across services.
- Identify latency and bottlenecks.
- Distributed tracing.
- Correlate with logs and metrics.



8. OBSERVABILITY DATA PILLARS



9. OBSERVABILITY STACK EXAMPLE



10. USEFUL PROMQL EXAMPLES

```
# Request rate (per second)
rate(http_requests_total[5m])
# Error rate (%)
100 * rate(http_requests_total{status=~"5..."}[5m]) /
  rate(http_requests_total[5m])
# Average latency (ms)
rate(http_request_duration_seconds_sum[5m]) /
  rate(http_request_duration_seconds_count[5m])
# Packet loss (%)
100 * (rate(node_network_receive_drop_total[5m]) /
  rate(node_network_receive_packets_total[5m]))
```

11. SAMPLE ALERT RULES

⚠ Latency High	avg latency > 250ms for 5m
⚠ Packet Loss High	packet loss > 5% for 5m
⚠ Error Rate High	error rate > 2% for 5m
⚠ High Jitter	jitter > 30ms for 5m
⚠ Interface Down	ifOperStatus = down
⚠ High Bandwidth	utilization > 80% for 5m

12. KEY NETWORK KPIs

- ☑ Availability (%)
- ☑ Latency (P50 / P95 / P99)
- ☑ Throughput (Mbps / Gbps)
- ☑ Packet Loss (%)
- ☑ Jitter (ms)
- ☑ Error Rate (%)
- ☑ Connection / Session Count
- ☑ Interface Utilization (%)

13. BEST PRACTICES

- ☑ Monitor from multiple points.
- ☑ Use time sync (NTP) everywhere.
- ☑ Set meaningful baselines.
- ☑ Correlate logs, metrics, traces.
- ☑ Automate alerts and responses.
- ☑ Review dashboards regularly.

14. COMMON ISSUES & WHAT TO CHECK

- ⊗ High latency → Check congestion, routing, QoS
- ⊗ Packet loss → Check drops, MTU, errors
- ⊗ High jitter → Check wireless, bufferbloat
- ⊗ Service unreachable → DNS, routing, firewall
- ⊗ Flow gaps → Exporter down, sampling, timeout
- ⊗ Missing metrics → Exporter, scrape config

15. QUICK CLI / TOOLS

```
ping                # Latency & packet loss
traceroute          # Path & hops
mtr                 # Real-time path analysis
iperf3              # Throughput test
netstat -s          # Network stats
ss -tupan           # Connections & sockets
ethtool -S eth0     # Interface counters
tcpdump -i any      # Packet capture
```

KEY TAKEAWAY

Observability turns data into clarity.
Use logs, metrics, and traces together to understand, act, and improve your network continuously.

16. INTERVIEW TIPS

- How do you monitor network latency?
- How do you detect packet loss?
- What is NetFlow and why use it?
- How do you troubleshoot high errors?
- How do you correlate logs & metrics?
- How do you trace a slow request?

VERIQTA

53. NETWORK PERFORMANCE OPTIMIZATION

MTU • TCP TUNING • CONNECTION OPTIMIZATION • KERNEL TUNING

"Fast networks are not just about bandwidth.
They are about tuning every layer for performance."

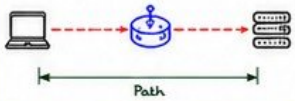
PRODUCTION NOTE

- ☑ Measure before you tune.
- ☑ Change one parameter at a time.
- ☑ Test in a staging environment first.
- ☑ Monitor after every change.
- ☑ Document all tuning values.

1. MTU OPTIMIZATION

- MTU affects throughput and latency.
- Too large = fragmentation.
- Too small = overhead.
- Find the optimal MTU for your path.

MTU PATH DISCOVERY



FIND OPTIMAL MTU

```
ping -M do -s <size> <destination>
```

Increase size until it fails.
Optimal MTU = last successful size + 28 (IP+ICMP header)

BEST PRACTICES

- Cloud VPC: 9001
- VPN/IPsec: 1436
- Docker overlay: 1450
- Avoid fragmentation

2. TCP TUNING

- TCP parameters control speed, reliability, and concurrency.
- Tune buffers, congestion control, and retries.

KEY TCP PARAMETERS

PARAMETER	PURPOSE
tcp_rmem	Receive buffer sizes
tcp_wmem	Send buffer sizes
tcp_congestion_control	Congestion algorithm
tcp_window_scaling	Window scaling
tcp_syn_retries	SYN retry attempts
tcp_fin_timeout	Time to keep closed connections
tcp_tw_reuse	Reuse TIME_WAIT

VIEW CURRENT SETTINGS

```
sysctl -a | grep "^net.ipv4.tcp"
```

3. CONNECTION OPTIMIZATION

- Reduce connection setup time.
- Reuse connections.
- Optimize keepalive and timeouts.
- Use connection pooling.

OPTIMIZATION TECHNIQUES

Connection Reuse
Reuse existing connections (HTTP keep-alive)

Connection Pooling
Maintain a pool of open connections

Keepalive Settings
Detect dead connections faster

Timeout Tuning
Remove idle connections quickly

4. KERNEL TUNING

- Kernel parameters affect network performance.
- Tune buffers, queues, and interrupts.
- Optimize for your workload.

IMPORTANT KERNEL PARAMETERS

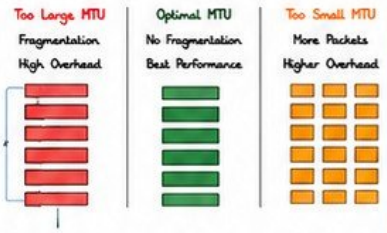
PARAMETER	PURPOSE
net.core.rmem_max	Max receive buffer
net.core.wmem_max	Max send buffer
net.core.netdev_max_backlog	Packet backlog queue
net.ipv4.ip_forward	IP forwarding
net.ipv4.tcp_max_syn_backlog	SYN backlog size
net.ipv4.tcp_max_tw_buckets	TIME_WAIT buckets
net.ipv4.tcp_fin_timeout	FIN-WAIT-2 timeout
net.core.somaxconn	Listen backlog size

APPLY TEMPORARILY

```
sysctl -w <parameter>=<value>
```

MAKE PERMANENT
Add to /etc/sysctl.conf

5. MTU OPTIMIZATION EXAMPLE



Goal: Maximize payload per packet without fragmentation.

6. TCP TUNING EXAMPLE (HIGH THROUGHPUT)

```
# /etc/sysctl.conf
net.ipv4.tcp_rmem = 4096 87380 16777216
net.ipv4.tcp_wmem = 4096 65536 16777216
net.ipv4.tcp_congestion_control = bbr
net.ipv4.tcp_window_scaling = 1
net.ipv4.tcp_max_syn_backlog = 4096
net.ipv4.tcp_fin_timeout = 15
net.ipv4.tcp_tw_reuse = 1
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
```

Apply: sysctl -p

7. CONNECTION OPTIMIZATION EXAMPLE (NGINX)

```
http {
    keepalive_timeout 65s;          # Keep connections alive
    keepalive_requests 1000;        # Reuse connections
    sendfile on;                    # Zero-copy file transfer
    tcp_nopush on;                  # Packet coalescing
    tcp_nodelay on;                 # Reduce latency
    client_body_buffer_size 128k;   # Buffer uploads
    client_max_body_size 100m;      # Large uploads
}
```

8. KERNEL TUNING EXAMPLE (LOW LATENCY)

```
# /etc/sysctl.conf
net.core.netdev_max_backlog = 50000
net.core.somaxconn = 65535
net.ipv4.ip_forward = 1
net.ipv4.tcp_max_syn_backlog = 8192
net.ipv4.tcp_max_tw_buckets = 262144
net.ipv4.tcp_fin_timeout = 15
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
```

Apply: sysctl -p

9. MEASURING NETWORK PERFORMANCE

TOOL	PURPOSE	EXAMPLE
ping	Latency, packet loss	ping -c 100 server
iperf3	Throughput testing	iperf3 -c server -P 4
mtr	Path analysis	mtr -rwc 100 server
ss	Socket statistics	ss -tatna
ethtool	Interface stats	ethtool -S eth0
netat	Kernel network stats	netat -ax
sar -n DEV	Interface statistics	sar -n DEV 1 10

10. COMMON NETWORK BOTTLENECKS

High Latency
Causes: Long distance, congestion, slow DNS, overloadad devices

Packet Loss
Causes: Congestion, errors, overload, bad cables, wireless issues

Low Throughput
Causes: Small buffers, CPU limits, suboptimal TCP settings

High Jitter
Causes: Variable congestion, wireless interference, QoS issues

11. TCP CONGESTION ALGORITHMS

ALGORITHM	BEST FOR	CHARACTERISTICS
cubic	General purpose	Stable, default in Linux
bbr	High bandwidth low latency	Uses bandwidth & RTT estimation
reno	Legacy networks	Old, less efficient
vegas	Low latency networks	Delay-based, conservative

12. CHECKLIST

- ☑ Optimize MTU for your path.
- ☑ Tune TCP buffers for your workload.
- ☑ Use modern congestion control (BBR).
- ☑ Enable connection reuse and pooling.
- ☑ Tune kernel parameters.
- ☑ Monitor metrics before and after tuning.
- ☑ Test under real production load.
- ☑ Document all changes.

13. QUICK COMMANDS

```
# MTU Discovery
ping -M do -s 1472 server

# Throughput Test
iperf3 -c server -P 4 -t 30

# TCP Stats
ss -s

# Interface Stats
ethtool -S eth0

# Kernel Network Stats
netat -ax
```

14. COMMON ISSUES & FIXES

ISSUE	FIX
High latency	Check path, DNS, QoS, congestion
Packet loss	Check errors, queues, cables
Low throughput	Tune buffers, TCP, check CPU
High CPU in softirq	Increase backlog, optimize drivers
Many TIME_WAIT	Reduce tcp_fin_timeout, reuse
Retransmissions	Check congestion, packet loss

15. PERFORMANCE OPTIMIZATION MINDSET

- **Measure** — Collect baseline metrics
- **Analyze** — Identify bottlenecks
- **Tune** — Adjust one parameter at a time
- **Validate** — Test and compare results
- **Monitor** — Continuously observe
- **Iterate** — Repeat and improve

16. INTERVIEW TIPS

- How does MTU affect performance?
- What is the role of TCP congestion control?
- How do you tune TCP for high throughput?
- What kernel parameters impact networking?
- How do you optimize connections in applications?
- How do you troubleshoot high latency issues?

KEY TAKEAWAY

Network performance is a combination of proper MTU, TCP, connection management, and kernel tuning. Measure, tune, validate, and monitor continuously for the best results.

VERIQTA

54. PRODUCTION INCIDENT RESPONSE

DETECTION • INVESTIGATION • CONTAINMENT • RECOVERY • POSTMORTEM

"Fast detection, clear investigation, decisive action, safe recovery, and continuous learning."

PRODUCTION NOTE

- ☑ Minutes matter. Data decides.
- ☑ Follow the process, not panic.
- ☑ Communicate early and clearly.
- ☑ Contain first, then fix.
- ☑ Learn and make it better.

1. DETECTION

- Alerts trigger an incident.
- Automated monitoring detects anomalies.
- Users may report issues.
- Validate the alert.

DETECTION SOURCES

- Prometheus Alerts
- Grafana Alerts
- Kubernetes Events
- Application Logs
- Synthetic / Health Checks
- User Reports

KEY ACTIONS

- Acknowledge the alert
- Validate the severity
- Create incident ticket
- Assemble the right team

2. INVESTIGATION

- Understand the blast radius.
- Collect data and evidence.
- Identify symptoms and patterns.
- Find the root cause.

INVESTIGATION WORKFLOW

Review Alert → Check Dashboard / Metrics → Inspect Logs → Check System / Service → Identify Root Cause

KEY QUESTIONS

- What is impacted?
- When did it start?
- What changed?
- What is the error?
- Who else is affected?

3. CONTAINMENT

- Stop the impact from spreading.
- Isolate the failing component.
- Protect users and critical systems.
- Maintain service availability.

CONTAINMENT STRATEGIES

- Scale down / Pause traffic
- Remove bad pods / instances
- Enable maintenance mode
- Block offending IP / traffic
- Limit access (NetworkPolicy)
- Disable failing feature

CONTAINMENT GOAL

Stabilize the system and prevent further damage while you fix the root cause.

4. RECOVERY

- Fix the root cause.
- Restore full functionality.
- Verify system health.
- Monitor closely after recovery.

RECOVERY STEPS

Fix the Issue → Deploy / Rollback → Warm Up / Scale Up → Verify Health Checks → Monitor & Validate

RECOVERY CHECKLIST

- ☑ Root cause fixed
- ☑ Health checks passing
- ☑ No errors in logs
- ☑ Metrics back to normal
- ☑ Users can access normally

5. POSTMORTEM

- Learn from the incident.
- Document what happened.
- Find root causes and improvements.
- Prevent recurrence.

POSTMORTEM OUTLINE

- **Summary** — What happened?
- **Impact** — Who was affected?
- **Timeline** — What happened when?
- **Root Cause** — Why did it happen?
- **What Went Well** — What helped?
- **What Didn't** — What can improve?
- **Action Items** — What will we change?

POSTMORTEM GOAL

Turn incidents into improvements. Make the next incident smaller and shorter.

INCIDENT RESPONSE LIFECYCLE

DETECTION — Know fast → INVESTIGATION — Find the cause → CONTAINMENT — Stop the impact → RECOVERY — Restore service → POSTMORTEM — Learn & improve

SEVERITY LEVELS (EXAMPLE)

P1 - CRITICAL	System down, major impact
P2 - HIGH	Degraded service, partial impact
P3 - MEDIUM	Minor impact, workaround available
P4 - LOW	Minimal impact, cosmetic issues

COMMUNICATION PLAN

- Acknowledge early — Internal + customers
- Use one channel — (Slack / Teams / PagerDuty)
- Provide updates — Be clear and honest
- Close the loop — Share resolution

USEFUL COMMANDS (K8S)

```
# Check pods
kubectl get pods -A
# Describe failing pod
kubectl describe pod <pod> -n <ns>
# View events
kubectl get events -n <ns> --sort-by=.lastTimestamp
# Logs
kubectl logs <pod> -n <ns> --tail=200
# Check nodes
kubectl get nodes
# Top resources
kubectl top pod -A
```

USEFUL COMMANDS (SYSTEM)

```
# Check CPU & RAM
top
# Check disk
df -h
# Check connections
ss -tulnp
# Check network
ping <host>
traceroute <host>
# View logs
journalctl -xe
tail -f /var/log/syslog
```

ESCALATION MATRIX (EXAMPLE)

Level 1	On-call Engineer
Level 2	Team Lead / SRE
Level 3	Engineering Manager
Level 4	CTO / Stakeholders

DATA TO COLLECT

- ☑ Logs
- ☑ Metrics
- ☑ Traces
- ☑ Events
- ☑ Configurations
- ☑ Screenshots
- ☑ User reports
- ☑ Time of incident

BEST PRACTICES

- ☑ Monitor everything that matters.
- ☑ Automate detection and alerting.
- ☑ Runbooks for common incidents.
- ☑ Test your incident response.
- ☑ Blameless culture encourages learning.
- ☑ Keep runbooks updated.

COMMON INCIDENT TYPES

- Service Down
- High Latency
- Error Rate Spike
- Database Issues
- Network Outage
- Resource Exhaustion
- Deployment Failure
- Security Incident

KEY TAKEAWAY

A strong incident response process minimizes impact, restores trust, and builds a reliable system.

Detect fast. Act smart. Learn always.

VERIQTA

55. COMMON PRODUCTION NETWORKING FAILURES

DNS OUTAGES • ROUTING FAILURES • FIREWALL ISSUES
LOAD BALANCER FAILURES • PACKET LOSS

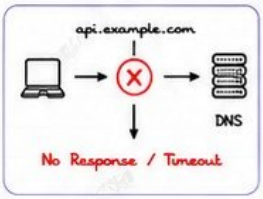
"Most outages are not caused by the application.
They are caused by the network the application depends on."

PRODUCTION NOTE

- [x] Assume the network can fail.
- [x] Validate every hop.
- [x] Have visibility before failure.
- [x] Automate detection and rollback.
- [x] Document and learn from every incident.

1. DNS OUTAGES

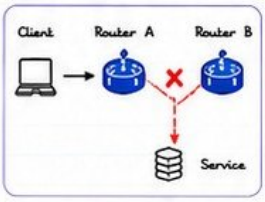
SYMPTOMS
- Applications cannot resolve names
- Intermittent timeouts
- Services appear down
- High latency on new connections



COMMON CAUSES
- DNS server failure
- Misconfigured DNS records
- TTL too high / stale cache
- Network ACL blocking DNS
- Resolver recursion failures

2. ROUTING FAILURES

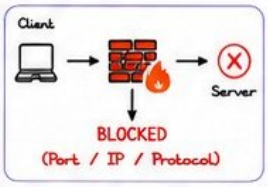
SYMPTOMS
- Services unreachable
- Some AZs / regions isolated
- Traffic takes wrong path
- High latency or drops



COMMON CAUSES
- Incorrect route tables
- BGP session down
- Missing return route
- Route propagation delay
- Misconfigured peering / TGW

3. FIREWALL ISSUES

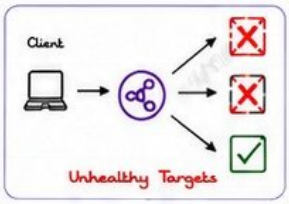
SYMPTOMS
- Connection timed out
- Intermittent access
- Specific ports blocked
- Some users can connect, others cannot



COMMON CAUSES
- Incorrect security group rules
- NACL blocking traffic
- Firewall rule change
- Missing ephemeral port range
- IP not allowed in allowlist

4. LOAD BALANCER FAILURES

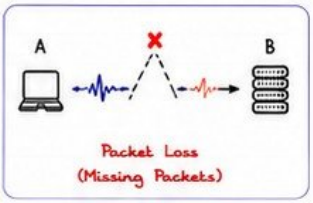
SYMPTOMS
- 502 / 503 / 504 errors
- No healthy targets
- High latency spikes
- Uneven traffic distribution



COMMON CAUSES
- All targets unhealthy
- Health check misconfiguration
- SSL certificate issues
- Idle timeout too low
- Cross-zone LB disabled (AZ fail)

5. PACKET LOSS

SYMPTOMS
- High retransmissions
- Slow application performance
- Timeouts / unstable connections
- VoIP / video quality issues



COMMON CAUSES
- Network congestion
- Faulty NIC / cables
- MTU mismatch / fragmentation
- Wireless interference
- ISP / upstream issues

DETECTION SIGNALS

- Detect from monitoring tools (Prometheus, Grafana, CloudWatch)
- Increase in error rates (4xx, 5xx, timeouts)
- Latency / packet loss spikes
- User reports / ticket influx
- Application & network logs indicating failures

QUICK CHECK WORKFLOW

Detect → Investigate → Isolate → Fix → Validate

1. Check user impact and scope
2. Validate DNS resolution
3. Check routing and reachability
4. Review firewall / security rules
5. Validate load balancer & targets
6. Check packet loss / latency
7. Implement fix or rollback
8. Monitor and confirm recovery

ESSENTIAL COMMANDS

```
# DNS
nslookup example.com
dig example.com +short
# Connectivity
ping -c 5 <host>
traceroute <host>
mtr -rw <host>
# Ports
nc -vz <host> <port>
telnet <host> <port>
# Packet Loss
mtr -rw <host>
ping -i 0.2 -c 100 <host>
# Routes
ip route
ip route get <host>
```

FAILURE VS OSI LAYER

FAILURE	OSI LAYER	EXAMPLE	IMPACT
DNS Outage	Application (L7)	Cannot resolve api.example.com	No connection established
Routing Failure	Network (L3)	No route to destination	Unreachable networks
Firewall Issue	Network (L3/L4)	Port 443 blocked	Connection timeout
Load Balancer Failure	Transport (L4/L7)	No healthy targets	503 / 502 errors
Packet Loss	Data Link (L2/L3)	High loss on path	High latency, retransmits

MITIGATION TIPS

- [x] Implement redundant DNS resolvers.
- [x] Use multiple routes and gateways.
- [x] Apply least privilege firewall rules.
- [x] Ensure load balancer health checks match real app behavior.
- [x] Monitor packet loss and latency continuously.
- [x] Test failover and rollback regularly.

PREVENTION BEST PRACTICES

- [x] Design for failure (HA, Multi-AZ/Region).
- [x] Monitor everything (metrics, logs, flows).
- [x] Automate validation (synthetic checks).
- [x] Implement change control & testing.
- [x] Keep runbooks updated.
- [x] Conduct regular game days.
- [x] Document network architecture and dependencies.

USEFUL TOOLS

- Prometheus / Grafana - Metrics & Alerts
- CloudWatch - Logs & Metrics
- Pingdom / UptimeRobot - External Monitoring
- Wireshark / tcpdump - Packet Analysis
- VPC Flow Logs / NetFlow - Traffic Visibility

INCIDENT RESPONSE STAGES

Detection (Know fast) → Investigation (Find root cause) → Containment (Stop the impact) → Recovery (Restore service) → Postmortem (Learn & improve)

REMEMBER
The network is the foundation.
When the network fails, everything fails.

KEY TAKEAWAY

Most production incidents are network-related. Fast detection, systematic investigation, and strong mitigation reduce impact and protect users.

VERIQTA

56. NETWORK SECURITY OPERATIONS

DDoS PROTECTION • IDS • IPS • WAF • THREAT DETECTION

Secure networks are not about trust.
They are about visibility, control, and continuous defense.

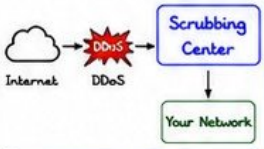
PRODUCTION NOTE

- [x] Assume breach.
- [x] Monitor everything.
- [x] Detect early.
- [x] Respond fast.
- [x] Improve continuously.

1. DDOS PROTECTION

HOW IT WORKS

- Absorbs and mitigates malicious traffic.
- Uses scrubbing centers and rate limiting.
- Keeps legitimate traffic flowing to your services.



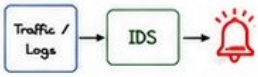
COMMON THREATS

- Volumetric attacks
- SYN floods
- UDP floods
- Application layer attacks
- Reflection / amplification

2. IDS (INTRUSION DETECTION)

HOW IT WORKS

- Monitors network or host activity.
- Detects suspicious patterns and policy violations.
- Alerts security team for further investigation.



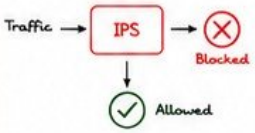
COMMON THREATS

- Port scanning
- Malware communication
- Policy violations
- Brute force attempts
- Anomalous behavior

3. IPS (INTRUSION PREVENTION)

HOW IT WORKS

- Actively blocks malicious traffic.
- Uses signatures and behavioral analysis.
- Prevents attacks before they reach systems.



COMMON THREATS

- Exploits and vulnerabilities
- SQL injection
- Remote access attempts
- Malware delivery
- Policy violations

4. WAF (WEB APPLICATION FIREWALL)

HOW IT WORKS

- Filters and inspects HTTP/HTTPS traffic.
- Blocks attacks targeting web applications.
- Protects against OWASP Top 10 threats.

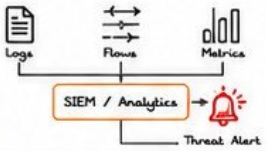
COMMON THREATS

- SQL injection
- Cross-site scripting (XSS)
- Cross-site request forgery (CSRF)
- Remote file inclusion
- Bot attacks

5. THREAT DETECTION

HOW IT WORKS

- Collects logs, metrics, and network flows.
- Uses correlation and behavior analytics.
- Detects advanced persistent threats (APTs).



COMMON THREATS

- Insider threats
- Lateral movement
- Command & control (C2)
- Data exfiltration
- Zero-day exploits

SECURITY OPERATIONS WORKFLOW

1. **IDENTIFY** — Monitor and collect data from all sources.
2. **DETECT** — Identify suspicious or malicious activities.
3. **ANALYZE** — Investigate alerts, correlate events, determine impact.
4. **RESPOND** — Contain threats, block traffic, remove access.
5. **RECOVER** — Restore services, validate integrity, monitor closely.
6. **REVIEW** — Conduct postmortem, update controls, improve detection.

NETWORK SECURITY BEST PRACTICES

- [x] Use defense in depth.
- [x] Keep systems and firmware updated.
- [x] Apply least privilege access.
- [x] Encrypt data in transit and at rest.
- [x] Segment networks and isolate critical assets.
- [x] Monitor 24/7 with automated alerts.
- [x] Regularly test and validate security controls.
- [x] Conduct security drills and tabletop exercises.

ESSENTIAL SECURITY TOOLS

- Cloudflare / AWS Shield (DDoS Protection)
- Snort / Suricata (IDS / IPS)
- AWS WAF / ModSecurity (Web Application Firewall)
- Security Onion / Zeek (Network Monitoring)
- ELK / Splunk / Graylog (SIEM & Log Analysis)
- Nmap / Masscan (Network Scanning)

COMMON ATTACK TYPES

- **DDoS Attacks** — Overwhelm bandwidth/resources
- **Man-in-the-Middle (MitM)** — Intercept and modify traffic
- **Exploitation** — Target vulnerabilities
- **Credential Attacks** — Brute force / password spraying
- **Malware / Ransomware** — Encrypt or steal data
- **Botnets** — Compromised systems at scale

USEFUL COMMANDS

```
# Check open ports
nmap -sS -Pn 10.10.10.0/24
# Monitor connections
ss -tulnp
# View firewall rules
sudo iptables -L -n -v
# Check system logs
sudo journalctl -xe
# Real-time packet capture
sudo tcpdump -i eth0 -nn
# Analyze web requests (nginx)
sudo tail -f /var/log/nginx/access.log
```

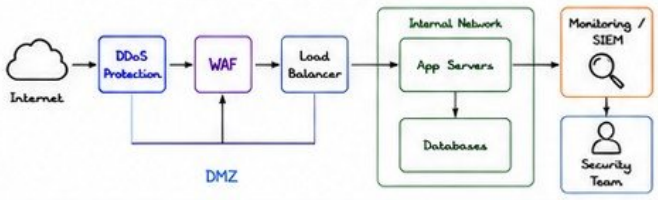
INCIDENT RESPONSE CHECKLIST

- [x] Identify and validate the alert.
- [x] Determine the scope and impact.
- [x] Isolate affected systems.
- [x] Block malicious IPs / traffic.
- [x] Preserve evidence.
- [x] Eradicate the threat.
- [x] Recover and verify systems.
- [x] Document and conduct postmortem.

KEY METRICS TO MONITOR

- Inbound / Outbound Traffic (Baseline & spikes)
- Blocked vs Allowed Connections (Security effectiveness)
- Top Attacked Ports / IPs (Threat targeting)
- Alert Volume & Response Time (Operational efficiency)

SECURITY ZONES EXAMPLE



KEY TAKEAWAY

Strong network security is a combination of technology, process, and people working together 24/7 to prevent, detect, and respond.

VERIQTA

57. ENTERPRISE TROUBLESHOOTING FRAMEWORK

LAYER-BY-LAYER INVESTIGATION • DECISION TREES • ROOT CAUSE ANALYSIS • ESCALATION STRATEGY

"Great engineers don't guess. They follow a system, eliminate possibilities, find the root cause, and prevent recurrence."

PRODUCTION NOTE

- ☑ Follow the process, not panic.
- ☑ Verify before you assume.
- ☑ Collect evidence at every step.
- ☑ Communicate clearly.
- ☑ Solve the problem and prevent it from returning.

1. LAYER-BY-LAYER INVESTIGATION

Investigate from the top down. Don't skip layers.

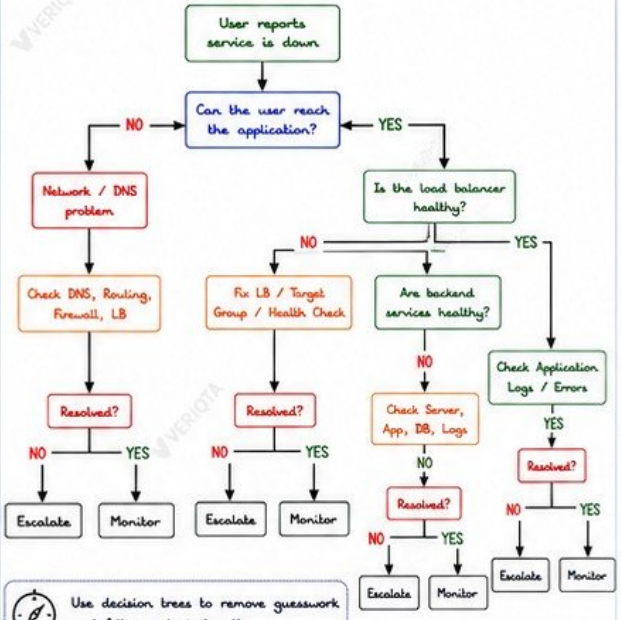
TOP

7. APPLICATION LAYER — App, Code, Logic, Errors
6. PRESENTATION LAYER — HTTP/HTTPS, SSL, Headers, API
5. SESSION LAYER — Sessions, Tokens, Auth, Timeouts
4. TRANSPORT LAYER — TCP/UDP, Ports, Connections
3. NETWORK LAYER — IP, Routing, Subnets, ACLs
2. DATA LINK LAYER — MAC, Switches, VLANs, ARP
1. PHYSICAL LAYER — Cables, NICs, Hardware, Power

BOTTOM

Rule: Start at Layer 7 and work down until you find the break.

2. DECISION TREE EXAMPLE (SERVICE DOWN)



Use decision trees to remove guesswork and follow a logical path.

3. ROOT CAUSE ANALYSIS (RCA)

1. Define the problem clearly.
2. Collect all facts and evidence.
3. Identify contributing factors.
4. Determine the root cause.
5. Implement corrective action.
6. Prevent recurrence.

RCA METHODS

- 5 Whys
- Fishbone Diagram
- Fault Tree Analysis
- Timeline Analysis
- Change Correlation

4. ESCALATION STRATEGY

CRITICAL

- L3 / Engineering Manager — Technical ownership, final decision
- L2 / Senior Engineer — Deep investigation, RCA lead
- L1 / Engineer — Initial investigation, mitigation
- Support / NOC — Monitoring, alerting, first response
- Automation / Tools — Auto-remediation, self-healing

LOW

5. ENTERPRISE TROUBLESHOOTING WORKFLOW

DETECT	VERIFY	INVESTIGATE	ANALYZE	RESOLVE	VALIDATE	DOCUMENT
Alert, User report, Monitoring	Reproduce issue, Confirm impact	Layer-by-layer, Collect evidence	Identify cause, Validate theory	Fix issue, Restore service	Test, Monitor, Confirm stable	RCA, Timeline, Lessons learned

Communicate clearly at every step.

6. EVIDENCE COLLECTION CHECKLIST

- ☑ Timestamps (use UTC)
- ☑ Screenshots / Logs
- ☑ Command outputs
- ☑ Configuration / Changes
- ☑ Affected users / Impact
- ☑ Metrics / Graphs
- ☑ Network captures (if needed)
- ☑ Timeline of events

7. USEFUL INVESTIGATION COMMANDS

ping <host>	Check connectivity
traceroute <host>	Trace network path
nslookup <domain>	Check DNS resolution
dig <domain>	Advanced DNS lookup
curl -I <url>	Check HTTP/HTTPS
netstat -tulnp	Check ports & services
ss -tulnp	Modern netstat
iptables -L -n -v	Check firewall rules
tcpdump -i any	Capture packets
ip route	Check routing table

8. COMMON QUESTIONS TO ASK

- When did the issue start?
- What exactly is failing?
- Who is affected?
- Any recent changes or deployments?
- Any related alerts or errors?
- Is it reproducible?
- What have we already tried?
- What is the business impact?

9. RCA DOCUMENT TEMPLATE

1. Summary
2. Impact
3. Timeline
4. Root Cause
5. Contributing Factors
6. Corrective Actions
7. Preventive Actions
8. Lessons Learned

10. TROUBLESHOOTING MINDSET

- Stay calm and structured.
- Follow the process.
- Eliminate, don't speculate.
- Communicate early and often.
- Learn and improve every time.

11. QUICK REFERENCE

Start Point	Layer 7 (Application)
Direction	Top Down
Goal	Find the Break
Method	Verify → Isolate → Fix
Key Tool	Evidence
Outcome	Restore & Prevent

12. TROUBLESHOOTING PRINCIPLES

- ☑ Verify before you change.
- ☑ Change one thing at a time.
- ☑ Document everything.
- ☑ Automate what you can.
- ☑ Prevent recurrence, not just resolution.

KEY TAKEAWAY

The best engineers solve problems systematically, find the root cause, and build systems that prevent the same issue from happening again.

VERIQTA

58. PRODUCTION CASE STUDIES

REAL INCIDENTS • REAL IMPACT • REAL LESSONS • REAL ENGINEERING

"Learn from incidents. Build systems that prevent recurrence."

PRODUCTION NOTE

- [x] Every incident teaches.
- [x] Document everything.
- [x] Understand the blast radius.
- [x] Communicate clearly.
- [x] Prevent the next one.

1 KUBERNETES OUTAGE

Cluster became unavailable across multiple namespaces.

WHAT HAPPENED
- Control plane API latency spiked.
- etcd disk I/O saturation.
- API server timeouts.
- New pods could not be scheduled.

IMPACT
- Apps unreachable
- Deployments failed
- Monitoring alerts fired
- Business disruption

ROOT CAUSE

etcd disk filled up due to excessive logs and old events.

RESOLUTION
- [x] Freed up etcd storage.
- [x] Compacted etcd.
- [x] Restarted API server.
- [x] Recovered controllers.
- [x] Restored normal ops.

LESSONS LEARNED
- Monitor etcd metrics.
- Set alerts on disk usage.
- Implement log rotation.
- Review retention policies.
- Run periodic etcd maintenance.

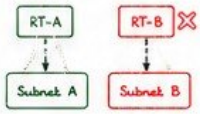
2 AWS NETWORKING FAILURE

WHAT HAPPENED
- Route table misconfiguration during deployment.
- 0.0.0.0/0 route pointed to wrong target.
- Instances lost internet access.

IMPACT
- EC2 instances offline
- Updates/downloads failed
- Users impacted
- Alarms triggered

ROOT CAUSE

Incorrect route table associated with private subnets.



RESOLUTION
- [x] Corrected route table.
- [x] Associated with subnets.
- [x] Restored NAT access.
- [x] Validated connectivity.
- [x] Monitored traffic.

LESSONS LEARNED
- Validate network changes.
- Use change reviews.
- Automate route table checks.
- Enable flow logs.
- Document network architecture.

3 DNS OUTAGE

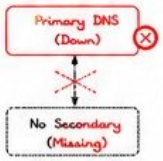
Internal DNS resolution failed for critical services.

WHAT HAPPENED
- DNS server crashed.
- No secondary resolver available.
- All internal lookups failed.
- Services could not discover dependencies.

IMPACT
- Service discovery failed
- Apps returned errors
- Traffic failed over incorrectly
- Incident escalated

ROOT CAUSE

Single point of failure DNS architecture.



RESOLUTION
- [x] Restored DNS server.
- [x] Cleared stale records.
- [x] Validated zones.
- [x] Implemented DNS redundancy.
- [x] Added health checks.

LESSONS LEARNED
- Never run single DNS.
- Use Route53 / Infoblox high availability.
- Implement TTL tuning.
- Monitor resolution latency.
- Test failover regularly.

4 HYBRID CONNECTIVITY FAILURE

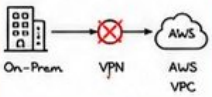
On-prem users lost access to cloud applications.

WHAT HAPPENED
- IPsec VPN tunnel dropped.
- BGP route flapping.
- Traffic blackholed.
- Applications became unreachable.

IMPACT
- Cloud apps inaccessible
- Files & data transfers failed
- Remote teams impacted
- Business slowdown

ROOT CAUSE

ISP instability caused tunnel flapping.



RESOLUTION
- [x] Re-established VPN.
- [x] Adjusted DPD settings.
- [x] Stabilized BGP routes.
- [x] Added backup tunnel.
- [x] Tested failover.

LESSONS LEARNED
- Use dual VPN tunnels.
- Monitor BGP stability.
- Implement automatic failover.
- Test disaster scenarios.
- Document runbooks.

5 LOAD BALANCER INCIDENT

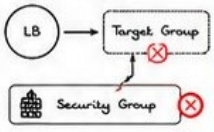
Application traffic returned 502/503 errors.

WHAT HAPPENED
- Health checks marked all targets unhealthy.
- Security group blocked health check IP.
- Load balancer had no healthy targets.

IMPACT
- 502 / 503 errors
- Users unable to access app
- Revenue impacted
- SLA breached

ROOT CAUSE

Health check traffic blocked by SG rules.



RESOLUTION
- [x] Updated SG rules.
- [x] Allowed health check IPs.
- [x] Targets became healthy.
- [x] Verified ALB behavior.
- [x] Added alerting.

LESSONS LEARNED
- Always allow health check IP.
- Monitor target health.
- Test deployments safely.
- Automate SG validations.
- Review change impact.

INCIDENT MANAGEMENT PRINCIPLES

- [x] Detect fast, respond faster.
- [x] Communicate early and often.
- [x] Contain the blast radius.
- [x] Document everything.
- [x] Learn and improve systems.

USEFUL TROUBLESHOOTING COMMANDS

```
# Kubernetes
kubectl get nodes -o wide
kubectl get pods -A
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events --sort-by=.metadata.creationTimestamp
# AWS
aws ec2 describe-instances
aws ec2 describe-route-tables
aws elbv2 describe-target-health --target-group-arn <arn>
aws cloudwatch get-metric-statistics --metric-name <metric>
# Network
dig <domain> @<dns-server>
mtr -rw <destination>
ping -c 10 <ip>
traceroute <destination>
```

INCIDENT RESPONSE CHECKLIST

- [x] Identify the issue.
- [x] Assess impact and severity.
- [x] Communicate with stakeholders.
- [x] Contain the issue.
- [x] Investigate root cause.
- [x] Fix and validate.
- [x] Monitor for stability.
- [x] Document and review.

VERIQTA

59. SENIOR ENGINEER NETWORKING CHECKLIST

DESIGN REVIEW • DEPLOYMENT VALIDATION • MONITORING CHECKLIST
SECURITY CHECKLIST • TROUBLESHOOTING CHECKLIST

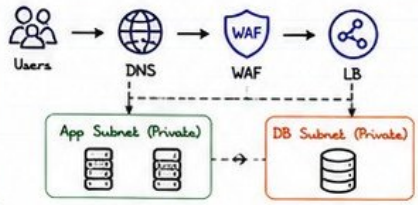
"Excellence in networking is not doing things right once.
It is doing the right checks every single time."

PRODUCTION NOTE

- ☑ Check before you deploy.
- ☑ Validate before you trust.
- ☑ Monitor before users complain.
- ☑ Secure before attackers find you.
- ☑ Document before you forget.

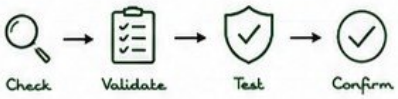
1 DESIGN REVIEW CHECKLIST

- ☑ Requirements understood and documented.
- ☑ Network architecture diagram is complete.
- ☑ IP addressing plan is approved.
- ☑ Subnet sizing is appropriate.
- ☑ Routing strategy is defined.
- ☑ High availability is designed.
- ☑ Security controls are defined.
- ☑ DNS strategy is defined.
- ☑ Load balancing strategy reviewed.
- ☑ Monitoring and logging planned.
- ☑ Failure scenarios reviewed.
- ☑ Cost impact reviewed.
- ☑ Design reviewed with stakeholders.



2 DEPLOYMENT VALIDATION CHECKLIST

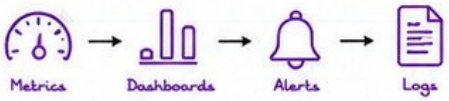
- ☑ VPC and subnets created correctly.
- ☑ Route tables validated.
- ☑ Internet / NAT connectivity verified.
- ☑ Security Groups follow least privilege.
- ☑ Network ACLs configured correctly.
- ☑ DNS resolution validated.
- ☑ Load balancer health checks pass.
- ☑ TLS certificates are valid.
- ☑ Application ports are open and tested.
- ☑ Inter-service connectivity tested.
- ☑ Database connectivity verified.
- ☑ High availability tested.
- ☑ Failover scenario tested.
- ☑ Deployment documented.



Validate everything before going live.

3 MONITORING CHECKLIST

- ☑ Network metrics collection enabled.
- ☑ Latency monitoring configured.
- ☑ Packet loss monitoring enabled.
- ☑ Throughput monitoring enabled.
- ☑ Error rate monitoring configured.
- ☑ DNS monitoring enabled.
- ☑ Load balancer metrics enabled.
- ☑ Firewall / WAF logs collected.
- ☑ Flow logs enabled.
- ☑ Alerts configured for critical metrics.
- ☑ Dashboards created in Grafana.
- ☑ Logs centralized.
- ☑ Retention policies configured.
- ☑ On-call notifications tested.



You can't fix what you can't see.

4 SECURITY CHECKLIST

- ☑ Principle of least privilege applied.
- ☑ Security Groups restrict traffic.
- ☑ Network ACLs configured.
- ☑ WAF rules enabled and tested.
- ☑ DDoS protection enabled.
- ☑ TLS everywhere (in transit).
- ☑ Secrets stored securely.
- ☑ Flow logs enabled.
- ☑ Intrusion detection enabled (IDS/IPS).
- ☑ Vulnerability scanning scheduled.
- ☑ Patching strategy in place.
- ☑ Access logs reviewed.
- ☑ Security changes logged.
- ☑ Periodic security review scheduled.

Secure by design. Defend by default.

5 TROUBLESHOOTING CHECKLIST

- ☑ Confirm the scope of the issue.
- ☑ Check recent changes / deployments.
- ☑ Verify DNS resolution.
- ☑ Test end-to-end connectivity.
- ☑ Check routing tables.
- ☑ Validate security groups / ACLs.
- ☑ Check load balancer health.
- ☑ Review firewall / WAF logs.
- ☑ Analyze application logs.
- ☑ Check network metrics.
- ☑ Identify root cause.
- ☑ Implement fix.
- ☑ Validate resolution.
- ☑ Document incident and lessons.

Observe → Analyze → Fix → Validate → Learn

QUICK COMMANDS

```
# Connectivity
ping <host>
traceroute <host>
mtr -rw <host>
nc -vz <host> <port>
# DNS
nslookup <domain>
dig <domain>
# Network
ip a
ip r
ss -tulnp
tcpdump -i <iface>
# AWS
aws ec2 describe-route-tables
aws ec2 describe-security-groups
```

LEGEND

- ☑ Must Have
- ☑ Should Have
- ☑ Good to Have
- ! Critical

SENIOR ENGINEER PRACTICAL RULES

1. **Verify Assumptions** — Never assume. Always verify.
2. **Document Everything** — If it's not documented, it didn't happen.
3. **Automate Checks** — Automate repetitive validations and monitoring.
4. **Plan for Failure** — Design for failure. Test your disaster plans.
5. **Monitor Continuously** — Continuous monitoring prevents surprise incidents.
6. **Learn & Improve** — Every incident teaches you something.

REMEMBER

Great networks are designed with intention, built with validation, monitored with discipline, secured with layers, and run with excellence.

VERIQTA

60. NETWORKING MASTERY ROADMAP

FROM CONCEPTS TO PRODUCTION EXCELLENCE

LEARN. PRACTICE. OPERATE. IMPROVE. LEAD.

"Networking is not about commands.
It is about understanding systems, solving problems,
and keeping businesses connected."

PRODUCTION NOTE

- [x] Master the fundamentals.
- [x] Understand the why, not just the how.
- [x] Design for failure.
- [x] Monitor everything.
- [x] Automate wisely.
- [x] Keep learning, always.

1 KEY CONCEPTS REVIEW

- [x] OSI Model and real-world mapping
- [x] IP Addressing, Subnetting, CIDR
- [x] Routing (Static, Dynamic, Policy)
- [x] Switching (VLANs, STP, EtherChannel)
- [x] Network Services (DNS, DHCP, NTP)
- [x] Network Security (Firewall, ACL, VPN)
- [x] Load Balancing and Proxies
- [x] Monitoring, Logs, Metrics, Traces
- [x] High Availability and Redundancy
- [x] Troubleshooting Methodologies

Understand → Remember → Apply → Master

2 PRODUCTION MINDSET

- [x] Design for failure, not for success.
- [x] Assume something will break.
- [x] Observe before you change.
- [x] Validate before you deploy.
- [x] Document everything.
- [x] Communicate clearly.
- [x] Measure what matters.
- [x] Automate repetitive tasks.
- [x] Simplify, standardize, secure.
- [x] Take ownership. End-to-end.

Observe → Validate → Automate → Secure

3 CONTINUOUS LEARNING PATH

- [x] Follow networking communities.
- [x] Read vendor and RFC documentation.
- [x] Study real-world outage reports.
- [x] Build projects and run labs.
- [x] Explore new technologies.
- [x] Attend webinars and conferences.
- [x] Contribute to open source.
- [x] Teach what you learn.
- [x] Stay curious. Stay consistent.
- [x] Level up every quarter.

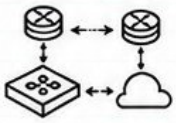
Learn → Practice → Build → Lead

4 RECOMMENDED LABS

- [x] Subnetting & IP Planning Lab
- [x] VLANs & Inter-VLAN Routing Lab
- [x] OSPF Multi-Area Lab
- [x] BGP Peering Lab
- [x] NAT, PAT & Port Forwarding Lab
- [x] Firewall Rules & Security Lab
- [x] Site-to-Site VPN Lab
- [x] HA Network Design Lab
- [x] Load Balancer Lab (NGINX / ALB)
- [x] Monitoring with Prometheus & Grafana
- [x] Kubernetes Networking Lab
- [x] Troubleshooting Scenarios Lab

Lab Tips

- Build it.
- Break it.
- Observe it.
- Fix it.
- Document it.
- Repeat it.



5 ENTERPRISE NETWORKING PRINCIPLES

Security First	Security is built-in, not bolted-on. Zero trust, least privilege, always.
High Availability	Redundancy at every layer. No single point of failure.
Performance	Design for speed, efficiency, and scalability.
Observability	You cannot improve what you cannot see.
Automation	Automate the boring. Focus on engineering.
Standardization	Consistent designs, naming, and documentation.
Resilience	Assume failure. Test often. Recover fast.

6 FINAL PRODUCTION ENGINEER GUIDANCE

- [x] Think like an architect, act like an operator.
- [x] Understand the business, not just the technology.
- [x] Solve the right problem, not just any problem.
- [x] Own your systems end-to-end.
- [x] Make your network boring and reliable.
- [x] Leave everything better than you found it.
- [x] Mentor others and build strong teams.
- [x] Stay humble. Keep learning.
- [x] Excellence is a habit.
- [x] You are now a Production Network Engineer.

Learn → Practice → Operate → Optimice → Lead

Learn	Practice	Operate	Optimice	Lead
Foundations Concepts Labs	Hands-on Projects Scenarios	Production Monitoring Reliability	Performance Security Automation	Design Mentor Improve

REMEMBER

The best networks are invisible.
They just work... all the time.